【A23】DocFusion 基于大语言模型的文档理解与多源数据融合系统项目详细方案

目 录

目 录 II

二、问题分析与项目立项依据 1

2.1 文档处理痛点 1

2.2 信息抽取痛点 1

2.3 模板填表痛点 1

2.4 智能交互与集成痛点 2

三、核心价值主张 2

四、系统各角色功能模块 2

4.1 系统管理员（Web 后台） 3

4.2 业务用户（Web 端） 3

4.3 对接与扩展（OpenAPI / MCP） 4

五、用例规约 4

5.1 用例规约目的与说明 4

5.2 角色（Actor）定义 4

5.3 用例图 5

5.4 用例总览 7

5.5 核心用例规约 8

5.5.1 UC-01 用户登录与 JWT 鉴权 8

5.5.2 UC-02 文档上传与异步解析 9

5.5.3 UC-03 单文档与批量 LLM 抽取 11

5.5.4 UC-04 抽取结果编辑与多源融合查看 12

5.5.5 UC-05 模板上传与字段识别 14

5.5.6 UC-06 模板智能填表 smart_fill 15

5.5.7 UC-07 智能对话与 Agent 多轮工具调用 17

5.5.8 UC-08 会话内附件上传与多文件勾选 19

5.5.9 UC-09 系统配置维护(LLM / Embedding / MCP) 20

5.5.10 UC-10 用户管理与操作日志审计 21

5.5.11 UC-11 OpenAPI 调用(X-API-Key) 22

5.5.12 UC-12 MCP 工具调用(JSON-RPC + SSE) *24*
5.5.13 UC-13 主题与个性化偏好持久化 *25*
5.6 用例与接口/数据表追溯关系 26
六、系统的设计思路与结构 **27**
6.1 设计思路 27
6.2 系统架构 27
6.3 典型业务流程与数据流 28
6.4 安全与非功能性设计 29
七、业务流程图 **30**
7.1 用户登录与 JWT 鉴权流程 30
7.2 文档上传与异步解析流程 32
7.3 信息抽取（HYBRID_EXTRACT）流程 33
7.4 模板上传与字段识别流程 34
7.5 模板自动填表（SMART_FILL）流程 35
7.6 智能对话 AGENT 多轮工具调用流程 36
7.7OPENAPI 调用流程（N8N / COZE 集成） 37
7.8MCP 工具调用流程（JSON-RPC + SSE） 37
八、系统建模(UML 补充) **39**
8.1 类图 / 领域模型 39
8.2 时序图(SEQUENCE DIAGRAMS) 40
8.2.1 SD-01 用户登录与 JWT 鉴权 *40*
8.2.2 SD-02 文档上传与异步解析 *41*
8.2.3 SD-03 信息抽取(hybrid_extract) *42*
8.2.4 SD-04 模板智能填表 smart_fill *43*
8.2.5 SD-05 智能对话 Agent 多轮工具调用 *44*
8.2.6 SD-06 MCP 工具调用(JSON-RPC + SSE) *45*
8.3 状态图(STATE DIAGRAMS) 47
8.3.1 ST-01 file_info.parse_status 状态机 *47*
8.3.2 ST-02 task_info.status 状态机 *47*
8.3.3 ST-03 chat_session 生命周期 *48*

8.4 数据流图(DFD) 49
8.4.1 DFD-0 上下文图 *50*
8.4.2 DFD-1 1 层分解 *50*
8.5 组件图 / 包依赖 51
九、数据库设计说明 51
9.1 概念模型与实体关系（ER） 51
9.2 数据表清单（物理模型） 52
9.3 数据表字段定义 53
9.3.1 用户表 users *53*
9.3.2 文件信息表 file_info *54*
9.3.3 文档切片表 doc_chunk *54*
9.3.4 信息抽取结果表 extract_result *55*
9.3.5 模板表 template_info *55*
9.3.6 模板字段映射表 template_mapping *56*
9.3.7 任务表 task_info *56*
9.3.8 操作日志表 operation_log *57*
9.3.9 字段字典表 field_dict *58*
9.3.10 系统配置表 system_config *58*
9.3.11 对话会话表 chat_session *59*
9.3.12 对话消息表 chat_message *59*
9.4 索引设计 59
9.5 枚举值与级联策略 60
9.6 容量与性能设计 62
9.7 大字段存储与向量索引协同 63
9.8 数据安全与脱敏 64
9.9 备份与恢复策略 65
9.10 数据库版本管理与迁移 65
9.11 审计列与软删除规范 67
9.12 数据字典与字段归一 68
9.13 ER 关系完整说明 68

十、API 接口规格说明 69

10.1 模块概览 69

10.2 认证与用户（/API/AUTH） 70

2.接口名称：login 71

3.接口名称：me 71

4.接口名称：upload_avatar 72

5.接口名称：get_avatar 73

6.接口名称：change_password 73

7.接口名称：change_nickname 74

10.3 文档管理（/API/DOCUMENTS） 74

1.接口名称：upload 74

2.接口名称：list_docs 75

3.接口名称：detail 76

4.接口名称：delete 76

5.接口名称：batch_delete 77

6.接口名称：reparse 77

7.接口名称：extract 78

8.接口名称：extract_all 78

9.接口名称：download 79

10.4 信息抽取结果管理（/API/EXTRACTION） 80

1.接口名称：get_results 80

2.接口名称：get_fused 80

3.接口名称：get_files_with_fields 81

4.接口名称：get_file_fields 81

5.接口名称：update 82

6.接口名称：batch_delete_results 82

10.5 模板与填表（/API/TEMPLATES） 83

1.接口名称：upload 83

2.接口名称：list_tpl 84

3.接口名称：delete_tpl 84

4.接口名称：update_requirement 85
5.接口名称：mappings 85
6.接口名称：fill 86
7.接口名称：fill_history 87
8.接口名称：delete_history 87
9.接口名称：download_fill_by_task 88
10.接口名称：download 88
10.6 智能对话与 AGENT（/API/CHAT） 89
1.接口名称：list_sessions 89
2.接口名称：get_session_messages 89
3.接口名称：delete_session 90
4.接口名称：ask 90
5.接口名称：chat_upload 91
6.接口名称：chat_file_list 92
7.接口名称：download_file 92
10.7 仪表盘统计（/API/DASHBOARD） 93
1.接口名称：stats 93
10.8 系统设置（/API/SETTINGS） 94
1.接口名称：get_config 94
2.接口名称：set_config 94
3.接口名称：list_users 95
4.接口名称：update_user 96
5.接口名称：delete_user 96
6.接口名称：get_logs 97
10.9 任务中心（/API/TASKS） 97
1.接口名称：list_tasks 97
2.接口名称：task_stats 98
10.10 对外开放接口（X-API-KEY）（/API/OPEN） 99
1.接口名称：api_info 99
2.接口名称：upload 99

3.接口名称：extract *100*
4.接口名称：extract_text *101*
5.接口名称：get_fields *101*
6.接口名称：get_fused *102*
7.接口名称：semantic_search *102*
8.接口名称：chat *103*
9.接口名称：list_files *104*
10.11 MCP 协议端点（/API/MCP） 104
1.接口名称：jsonrpc *104*
2.接口名称：sse *105*
3.接口名称：message *105*
4.接口名称：tools *106*
十一、关键技术 107
11.1 后端技术 107
11.2 前端技术 107
11.3 数据库与存储技术 108
11.4 系统集成技术 108
11.5 DEVOPS 与部署 109
十二、具体功能实现与当前使用情况 110
12.1 功能实现情况 110
12.1.1 认证与用户模块 *110*
12.1.2 文档管理模块 *110*
12.1.3 信息抽取模块 *110*
12.1.4 模板与填表模块 *110*
12.1.5 智能对话模块 *110*
12.1.6 仪表盘与设置模块 *111*
12.1.7 任务与扩展模块 *111*
12.1.8 界面与个性化模块 *111*
12.2 当前使用情况与界面截图 112
十三、核心代码示例 134

13.1 后端核心代码 134
13.1.1 FastAPI 应用装配与路由注册（节选） *134*
13.1.2 JWT 签发（节选） *134*
13.2 前端核心代码 134
13.2.1 路由守卫（节选） *134*
13.2.2 开发代理（节选） *135*
13.3 数据库与连接（节选） 135
13.4 智能对话与 AGENT 工具（节选） 135
13.4.1 Agent 多轮工具调用（节选） *135*
13.4.2 Excel 图表与数据有效性（节选） *135*
13.5 开放接口与 MCP（节选） 136
13.5.1 OpenAPI X-API-Key 鉴权（节选） *136*
13.5.2 MCP JSON-RPC 派发（节选） *136*
13.6 智能填表 SMART_FILL（节选） 136
13.7 前端主题与个性化（节选） 137
13.7.1 CSS 变量主题切换（节选） *137*
13.7.2 自定义鼠标跟随（节选） *137*
十四、项目开发说明 138
十五、创新价值 138
15.1 模式创新 138
15.2 功能与体验创新 138
15.3 技术应用创新 138
十六、商业与社会价值 139
16.1 商业价值 139
16.2 社会价值 139
十七、总结与后续优化改进 139
17.1 项目总结 139
17.2 后续优化改进 139
17.2.1 功能扩展 *139*
17.2.2 技术升级 *139*

17. 2. 3 业务拓展 139

十八、术语与名词定义 140

一、项目背景

各级组织与企业在日常工作中积累了大量公文、统计公报、行业报告、合同及附件等材料，存在格式多样、篇幅长、信息分散等问题。人工摘录与反复填报不仅效率低，而且容易出现遗漏与版本不一致，难以支撑领导决策与业务协同。

传统“文件夹+表格”的管理方式，难以将非结构化文本转化为可检索、可复用、可审计的结构化知识；同时，对外部大模型能力的引入又需要兼顾配置灵活性、数据边界与可持续运维。基于上述痛点，DocFusion（文档理解与多源数据融合系统）以 Web 化、模块化方式构建端到端能力：从文档接入、解析与抽取，到模板成稿、对话核验与运营看板，形成可落地的文档智能处理闭环。

二、问题分析与项目立项依据

立项前对四类典型痛点做了梳理，作为本系统功能边界与技术选型的直接依据。

2.1 文档处理痛点

1. 公文 / 统计 / 合同等材料格式混杂（pdf、docx、xlsx、xls、doc(OLE)等），缺乏统一入口；

2. 信息分散在长文档中，人工摘录效率低、易遗漏、版本不一致；

3. 旧版 .doc 与 WPS 私有特性（如 DISPIMG 嵌入图）导致传统 Office 工具链无法直接处理。

对应方案:document_parser 多格式分发 + legacy_doc 走 LibreOffice headless 转换 + image_filler 同时兼容 DISPIMG 与浮动图片。

2.2 信息抽取痛点

1. 单纯规则抽取覆盖窄、维护成本高；纯 LLM 抽取又存在幻觉、JSON 格式不稳；

2. 同一字段在多份材料中表达不一致（别名、单位、口径），需要去重与置信度排序。

对应方案：extractor.hybrid_extract = 多类正则（电话/邮箱/日期/身份证/金额/统计句/键值对/增速）+ LLM 强制 JSON 输出；fusion 服务按 ALIASES 归一、按来源加权、多值投票。

2.3 模板填表痛点

1. 真实模板大量使用合并单元格、复杂表头与数字格式，传统脚本一动就乱；

2. 数据源中混杂英文表头行、空行、分组行（如「省份」），需要智能跳过；

3. 「合计、平均」等汇总列需要按表头自动识别并计算。

对应方案：smart_fill 在保留模板合并单元格、表头样式、数字格式的前提下做模糊列匹配；并对汇总列做自动求和/均值。

2.4 智能交互与集成痛点

1. 通用对话产品无法直接操作用户本地的 Word / Excel 文件；
2. 自动化平台（n8n / Coze / Cursor 等）需要规范化、可发现的接入方式；
3. 多轮工具调用过程不透明，难以审计。

对应方案：doc_editor 暴露 12 项 Agent 工具，chat.py 以 12 轮 tool_use 循环驱动；同时提供 OpenAPI（X-API-Key）与 MCP（JSON-RPC + SSE）双通道；操作日志写入 operation_log。

三、核心价值主张

DocFusion 以“可控、融合、成稿”为核心，为不同角色提供针对性价值：

业务用户：一站式完成文档上传、抽取结果查看、模板填表与智能问答，减少重复劳动，缩短从材料到可用信息的路径。

管理人员：通过仪表盘掌握处理规模与关键指标，结合系统配置统一管理模型与扩展能力，降低运维与沟通成本。

技术对接方：提供规范化 REST 接口及可配置的 MCP 等扩展方式，便于与现有办公、自动化流程或第三方工具集成。

组织层面：数据可本地化部署与存储（默认 SQLite 与本地文件目录），在满足试用与中小规模生产的同时，保留后续替换为集中式数据库与对象存储的演进空间。

四、系统各角色功能模块

系统主要面向系统管理员、业务用户及外部对接三类角色，功能模块协同完成“文档—知识—成稿—对话—运营”的全流程。

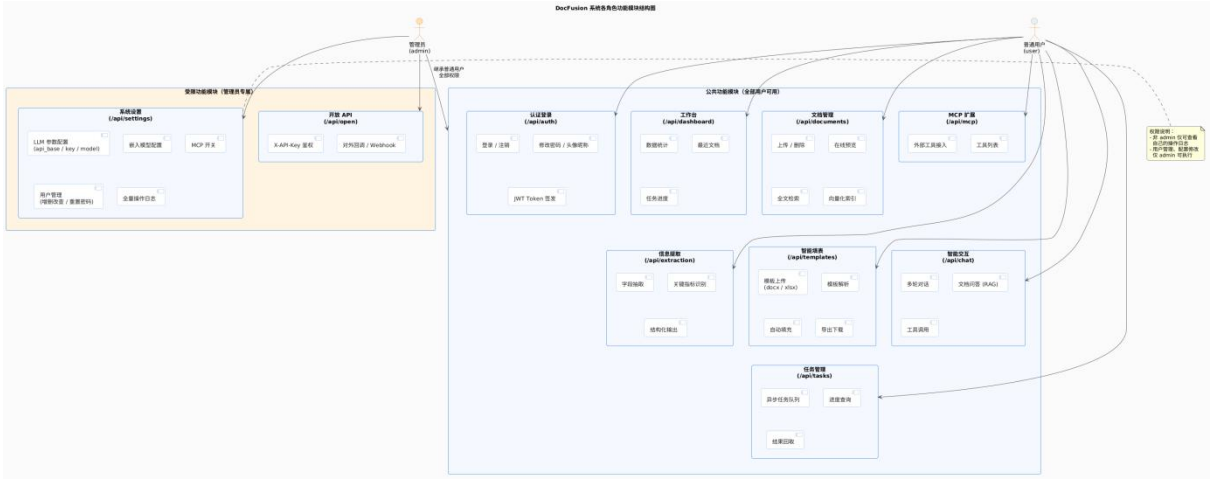


图 4-1 功能模块结构图

4.1 系统管理员（Web 后台）

1. 用户与权限：账户维护、登录认证与角色区分（如管理员与普通用户场景可扩展）。

2. 系统配置：大模型 API（地址、密钥、模型名、格式等）与 MCP 开关等参数的可视化配置与持久化。

3. 运营与数据：仪表盘统计、任务与导出等环节的支撑能力（随版本迭代可继续增强）。

4. 安全审计：关键操作日志、导出记录与配置变更留痕（可在后续版本落地）。

5. 用户与运维：管理员可维护用户列表、角色与密码，删除用户（内置超级管理员除外）；操作日志支持分页查询关键操作记录，非管理员通常仅可见与本人相关条目（以当前实现为准）。

4.2 业务用户（Web 端）

业务用户通过 Web 端与系统交互，主要功能与能力如下：

1. 文档中心

核心功能：上传与管理文档，查看文档的解析与处理状态。

业务侧能力：

(1) 支持对文档进行批量删除操作。

(2) 支持对已上传文档发起重新解析。

(3) 支持单文件或批量触发基于大语言模型（LLM）的智能信息提取任务。

2. 信息抽取

核心功能：对文档内容进行结构化信息抽取，并浏览抽取结果，为后续的数据融合与引用提供支撑。

业务侧能力：

(1) 支持用户对自动抽取的结果进行手动编辑与修正。

(2) 支持对抽取出的结构化条目进行批量删除。

3. 模板填表

核心功能：基于预置模板与用户的需求描述，引导用户完成填表流程，并最终生成目标文档成稿。

业务侧能力：

(1) 填表过程支持录入和参考用户要求文本。

(2) 支持查看历史填表记录。

(3) 支持按填表任务维度，下载任务相关的输出文件。

4. 智能对话

核心功能：结合用户已处理的文档内容与系统知识库能力，进行智能问答与内容核验。

业务侧能力：

(1) 支持会话持久化，用户可随时返回历史对话。

(2) 支持在会话内上传附件，针对新文件进行交互。

(3) 支持基于多文件上下文进行关联问答与分析。

5. 个人与任务管理

核心功能：用户登录与身份会话管理，查看个人任务进度，获取已完成的输出文件等。

4.3 对接与扩展（OpenAPI / MCP）

1. **OpenAPI（路径前缀 /api/open）：**对外提供规范化开放 HTTP 接口描述与调用方式，便于二次开发与系统集成。使用 Header「X-API-Key」鉴权；提供上传并解析、对已上传文件执行 LLM 深度提取、对纯文本直接提取、查询已提取字段、获取融合数据、语义搜索、轻量问答、列举文件等能力；GET /api/open/ 返回 JSON 形式接口索引，便于 n8n、Coze 等编排集成。

2. **MCP 端点：**提供 JSON-RPC（/api/mcp/jsonrpc）、SSE（/api/mcp/sse）、消息（/api/mcp/message）、工具列表（/api/mcp/tools）等，与外部智能体或工具链对接；具体工具集合以服务端实现为准。

五、用例规约

5.1 用例规约目的与说明

本章基于 DocFusion 已实现的功能,结合 backend/routers 与 frontend/src 的真实代码流,梳理出 13 个核心用例;每个用例规约统一描述用例编号、名称、参与者、前置条件、后置条件、触发条件、主成功场景、扩展事件流、异常流、特殊需求,以及关联的接口、数据表与界面。

5.2 角色（Actor）定义

1. **ADM 系统管理员**：拥有系统配置、用户与日志管理权限;通常由内置超级管理员或被授权账号承担。对应 users.role='admin'。

2. **USR 业务用户**：面向材料/统计/合同等岗位,使用 Web 端完成上传、抽取、模

板填表、智能对话等业务操作。对应 users.role='user'。

3. **API 开放接口调用方**：通过 X-API-Key 调用 /api/open/* 端点的外部低代码平台,如 n8n、Coze、Dify 等。

4. **MCP MCP 客户端**：通过 JSON-RPC + SSE 调用 /api/mcp/* 端点的智能体客户端,如 Claude Desktop、Cursor 等。

5. **LLM 大语言模型服务**：Anthropic / OpenAI 双格式兼容的外部推理服务,作为次要参与者出现在抽取与对话用例中。

6. **FS 文件系统/文档转换器**：包含本地落盘目录与 LibreOffice headless,作为次要参与者支持解析与导出。

5.3 用例图

下图展示 DocFusion 主要 Actor 与用例的关系。

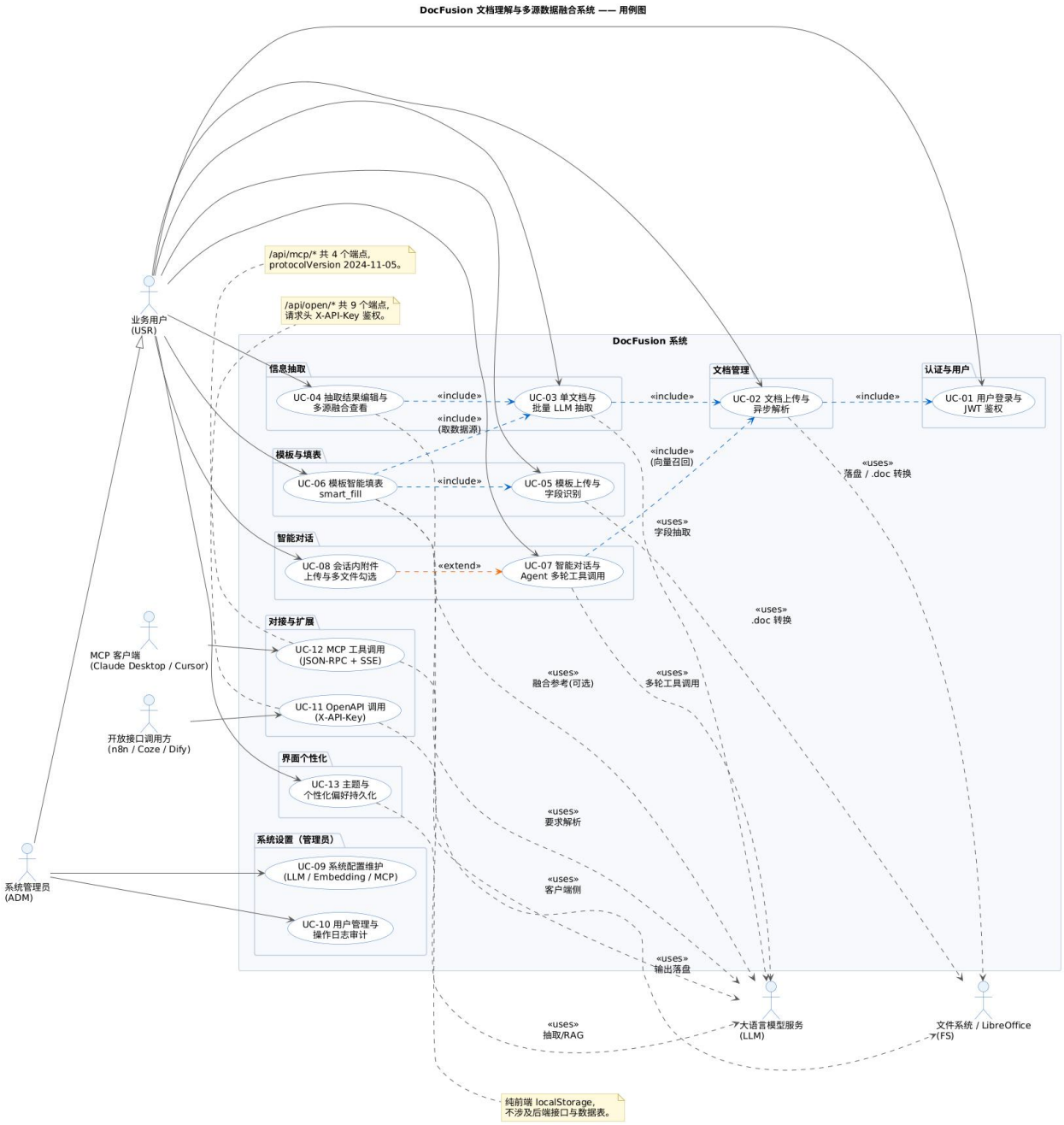


图 5-1 DocFusion 用例图

Actor 与用例的对应关系：

- **业务用户 USR**：UC-01 登录、UC-02 上传解析、UC-03 LLM 抽取、UC-04 抽取结果编辑、UC-05 模板上传识别、UC-06 智能填表、UC-07 智能对话、UC-08 会话附件、UC-13 主题个性化。
- **系统管理员 ADM**：上述 USR 全部用例 + UC-09 系统配置、UC-10 用户管理与日志审计。
- **开放接口调用方 API**：UC-11 OpenAPI 调用。
- **MCP 客户端 MCP**：UC-12 MCP 工具调用。

- **大语言模型服务 LLM**：在 UC-03/04/06/07/11 中作为次要参与者参与。
- **文件系统/转换器 FS**：在 UC-02/05/06 中作为次要参与者参与（落盘、.doc→.docx 转换）。

5.4 用例总览

本章共定义 13 个核心用例,清单如下：

表 5-1 核心用例清单

用例编号	用例名称	主参与者	优先级	接口列表
UC-01	用户登录与 JWT 鉴权	USR / ADM	高	/api/auth/login, /api/auth/me
UC-02	文档上传与异步解析	USR / ADM	高	/api/documents/upload, /reparse
UC-03	单文档与批量 LLM 抽取	USR / ADM	高	/api/documents/{fid}/extract, /extract-all
UC-04	抽取结果编辑与多源融合查看	USR / ADM	高	/api/extraction/results, /fused, /files/{fid}/fields
UC-05	模板上传与字段识别	USR / ADM	高	/api/templates/upload, /list, /mappings
UC-06	模板智能填表 smart_fill	USR / ADM	高	/api/templates/{tid}/fill, /history, /download
UC-07	智能对话与 Agent 多轮工具调用	USR / ADM	高	/api/chat/ask, /sessions, /messages
UC-08	会话内附件上传与多文件勾选	USR / ADM	中	/api/chat/upload, /files, /files/{fid}/download
UC-09	系统配置维护（LLM / Embedding / MCP）	ADM	高	/api/settings/config (GET/PUT)
UC-10	用户管理与操作日志审计	ADM	高	/api/settings/users, /logs
UC-11	OpenAPI 调用（X-API-Key）	API	中	/api/open/* 共 9 个端点

UC-12	MCP 工具调用（JSON-RPC + SSE）	MCP	中	/api/mcp/jsonrpc, /sse, /message, /tools
UC-13	主题与个性化偏好持久化	USR / ADM	低	前端 localStorage(无后端接口)

5.5 核心用例规约

5.5.1 UC-01 用户登录与 JWT 鉴权

用例编号：UC-01

用例名称：用户登录与 JWT 鉴权

主参与者：业务用户（USR）/ 系统管理员（ADM）

次要参与者：无

优先级：高

简要描述：用户在登录页输入账号密码,后端 bcrypt 校验通过后签发 HS256 JWT(默认 1440 分钟有效),前端写入 sessionStorage 的 token 字段;之后所有受保护接口由 axios 拦截器自动注入 Authorization Header。

前置条件：1) 账号已存在于 users 表;2) 前端可访问后端 /api/auth/login。

后置条件（成功）：1) 用户获得有效 JWT;2) 前端进入工作台页;3) 后续接口请求 Header 携带 Bearer Token。

后置条件（失败）：用户停留在登录页,前端展示错误提示;不写入任何令牌。

触发条件：用户在登录页点击「登录」按钮。

主成功场景：

1. 用户在登录页输入 username、password。
2. 前端 POST /api/auth/login,请求体含 {username, password}。
3. 后端按 username 查 users 表,取出 password 哈希。
4. bcrypt.checkpw 校验密码通过。
5. 后端用 SECRET_KEY 以 HS256 签发 JWT(payload 含 user_id、exp=now+1440min)。
6. 后端返回 {token, user: {id, username, nickname, role, avatar}}。
7. 前端将 token 写入 sessionStorage,user 写入 Vuex/Pinia store,跳转 /dashboard。

8. axios 请求拦截器从 sessionStorage 读 token,在每次请求 Header 注入 Authorization: Bearer <token>。

扩展事件流:

3a. 账号不存在: 后端返回 401「账号或密码错误」,前端弹出提示,流程返回步骤 1。

4a. 密码不匹配: 后端返回 401「账号或密码错误」,前端弹出提示,流程返回步骤 1。

8a. 任意请求返回 401: 前端响应拦截器清空 token,跳转登录页。

异常流:

- 后端不可用 / 网络中断: axios 统一捕获,弹出「服务不可用」提示,不修改本地状态。

- JWT 过期: 服务端校验失败返回 401,触发 8a 跳转。

特殊需求: 1) password 字段以 bcrypt 哈希存储,严禁明文;2) 建议生产环境对 /api/auth/login 启用限流;3) JWT 通过 HTTPS 传输。

涉及接口: POST /api/auth/login;GET /api/auth/me。

涉及数据表: users。

涉及界面: 前端 src/views/Login.vue;axios 拦截器 src/utils/request.ts。

5.5.2 UC-02 文档上传与异步解析

用例编号: UC-02

用例名称: 文档上传与异步解析

主参与者: 业务用户(USR)/ 系统管理员(ADM)

次要参与者: 文件系统/LibreOffice(FS)

优先级: 高

简要描述: 用户在「文档管理」页选择 1~N 份文件,上传至后端;后端落盘并写入 file_info(parse_status=pending),随后由 BackgroundTasks 触发 document_parser 按文件类型分发解析,生成 doc_chunk 与向量索引,完成后置 parse_status=done。

前置条件: 1) 用户已登录;2) 文件类型属于 pdf/docx/xlsx/xls/doc/jpg/png 之一。

后置条件（成功）: 1) 文件落盘 backend/uploads/;2) file_info 记录 parse_status=done,chunk_count>0;3) doc_chunk 表写入文本切片;4) 向量索引同步生成。

后置条件（失败）: file_info.parse_status=failed,前端显示「重新解析」按钮;原文件保留以便重试。

触发条件：用户点击「上传文档」并确认文件选择。

主成功场景：

1. 用户在文档管理页点击「上传文档」选择 1~N 份文件。

2. 前端 POST /api/documents/upload(multipart/form-data,可携带多个 files)。

3. 后端校验大小、类型、文件名安全。

4. 后端落盘到 backend/uploads/<uuid>_<原名>。

5. 后端为每份文件写入 file_info 记录,parse_status='pending',upload_user=<当前用户 id>。

6. 后端通过 BackgroundTasks 异步触发 document_parser.parse(file_id)。

7. document_parser 按 file_type 分发：pdf→pdfplumber/pymupdf,docx→python-docx,xlsx→openpyxl,xls→xlrd,img→OCR 适配器。

8. 解析得到的文本被切分为多个 chunk 写入 doc_chunk(file_id, chunk_index, content, char_count)。

9. vector_store 同步生成 embedding 索引(若 sentence-transformers 可用),否则降级为关键词索引。

10. 解析完成后更新 file_info.parse_status='done'、chunk_count、parse_time。

11. 前端轮询 /api/documents/list 或 WebSocket 推送(若启用),界面状态从「解析中」变为「已完成」。

扩展事件流：

3a. 文件超过 MAX_UPLOAD_SIZE：返回 413,前端提示「文件过大」。

3b. 文件类型不在白名单：返回 415,前端提示「不支持的文件类型」。

7a. 文件类型为 .doc(OLE)：进入 legacy_doc 分支,调用 LibreOffice headless 转换为 .docx 后再走 docx 分支。

7b. 解析单个 chunk 失败：跳过该 chunk 记录到日志,不阻塞其它 chunk。

异常流：

- 落盘失败(磁盘满)：回滚 file_info,返回 500。

- 解析整体失败：置 parse_status='failed',记录错误信息;用户可在文档管理页点「重新解析」(POST /reparse) 触发重试。

- LibreOffice 不可用：legacy_doc 抛错,parse_status='failed',提示「无法处理 .doc 旧版文件」。

特殊需求：1) 单次上传支持多文件;2) 解析为异步,不阻塞 HTTP 响应;3) 同名文件不冲突(以 uuid 区分);4) 解析进度可通过 file_info.parse_status 查询。

涉及接口：POST /api/documents/upload;GET /api/documents/list;POST /api/documents/{fid}/reparse;DELETE /api/documents/{fid};POST /api/documents/batch-delete。

涉及数据表：file_info,doc_chunk,task_info(可选)。

涉及界面：前端 src/views/Documents/Index.vue。

5.5.3 UC-03 单文档与批量 LLM 抽取

用例编号：UC-03

用例名称：单文档与批量 LLM 抽取

主参与者：业务用户(USR)/ 系统管理员(ADM)

次要参与者：大语言模型服务(LLM)

优先级：高

简要描述：对已解析完成的文档执行 hybrid_extract:先以多类正则做粗抽,再调用 LLM 强制 JSON 输出做细抽,fusion 服务按字段别名归一 + 来源加权 + 多值投票,结果写入 extract_result 表。支持单文件与批量两种入口。

前置条件：1) 文件已上传且 file_info.parse_status='done';2) doc_chunk 中已存在切片;3) 系统配置中 LLM API 已正确填写。

后置条件（成功）：extract_result 表新增/更新该文件的字段记录;file_info.field_count 同步更新。

后置条件（失败）：task_info 记录失败状态,前端显示错误,extract_result 保持原状。

触发条件：用户在「文档管理」页点击单文件「LLM 抽取」按钮 或 顶部「批量提取」按钮。

主成功场景：

1. 用户点击「LLM 抽取」(单文件) 或「批量提取」(对所有 parse_status='done' 文件)。

2. 前端 POST /api/documents/{fid}/extract 或 /extract-all。

3. 后端创建 task_info 记录(type='extract',status='running')。

4. 读取 doc_chunk,合并文本(必要时分窗口)。

5. extractor.rule_extract 用多类正则抽取(电话/邮箱/日期/身份证/金额/统计句/键值对/增速)。

6. extractor.llm_extract 调用 llm_service.extract_fields(强制 JSON 输出 schema)。

7. fusion 服务对规则结果与 LLM 结果按字段别名归一(基于 field_dict),按 source 加权,多值投票得到最终值与置信度。

8. 写入 extract_result(file_id,field_name,field_value,confidence,source,extract_method)。

9. 更新 file_info.field_count;关闭 task_info(status='done')。

10. 前端切换到「信息抽取」页查看结果。

扩展事件流:

2a. 文件未解析完成: 返回 409「请等待文件解析完成」。

6a. LLM 返回非合法 JSON: 自动重试 1 次;再失败则只保留规则抽取结果,confidence=0.5。

7a. 同字段多份证据冲突: 取置信度最高者作为 final_value,其余进入 candidates。

异常流:

- LLM 服务不可达: task_info.status='failed',记录错误;前端可手动重试。

- LLM 限流(429): 自动按指数退避重试 ≤3 次,仍失败则 failed。

- 单文件失败不影响 /extract-all 中其他文件;最终任务状态汇总为 partial_success。

特殊需求: 1) LLM 输出必须为合法 JSON(强制 schema);2) 对超长文档自动分窗口抽取;3) 抽取过程可被中断/取消(后续迭代);4) 字段命名遵循 field_dict 标准名。

涉及接口: POST /api/documents/{fid}/extract;POST /api/documents/{fid}/extract-all 或 /api/extraction/extract-all。

涉及数据表: extract_result,file_info,doc_chunk,field_dict,task_info,system_config(读取 LLM 配置)。

涉及界面: 前端「文档管理」页的行级与顶部按钮;「任务中心」可查看抽取任务状态。

5.5.4 UC-04 抽取结果编辑与多源融合查看

用例编号: UC-04

用例名称: 抽取结果编辑与多源融合查看

主参与者: 业务用户(USR)/ 系统管理员(ADM)

次要参与者: 无

优先级: 高

简要描述：用户在「信息抽取」页分页浏览/筛选已抽取的字段记录，可就地编辑、批量删除；另提供「多源融合视图」聚合同一字段在多份文件中的候选值与置信度，以及「按文件查看字段」视图。

前置条件：1）用户已登录；2）extract_result 表中已有该用户文件的抽取记录。

后置条件（成功）：1）编辑后字段值持久化到 extract_result；2）批量删除后对应记录被移除；3）file_info.field_count 同步刷新。

后置条件（失败）：数据库回滚，前端显示错误。

触发条件：用户进入「信息抽取」页。

主成功场景：

1. 用户进入「信息抽取」页，前端 GET /api/extraction/results（分页 + 可选 file_id/field_name 过滤）。

2. 后端返回结果列表，按字段分组卡片渲染。

3. 用户点击某字段值的「编辑」按钮，就地修改值。

4. 前端 PUT /api/extraction/results/{rid}，请求体含 {field_value，confidence?}。

5. 后端校验字段所属用户，持久化更新。

6. 用户切换到「多源融合视图」查看同一字段跨文件的候选值。

7. 前端 GET /api/extraction/fused，后端返回 fusion 后的 final_value + candidates 列表。

8. 用户切换到「按文件查看」，GET /api/extraction/files/{fid}/fields。

扩展事件流：

4a. 字段值校验不通过（类型/长度/格式）：返回 400，前端高亮提示。

5a. 当前用户非该记录所有者且非 admin：返回 403。

6a. 用户勾选多个字段后点「批量删除」：POST /api/extraction/batch-delete-results。

异常流：

- 数据库连接异常：返回 500，前端提示「保存失败，请稍后重试」。

- 并发编辑同一字段：后写入者覆盖前写入者（MVP 版本暂不做乐观锁，后续迭代补）。

特殊需求：1）用户数据隔离（只能看到自己上传文件的字段；admin 可看全部）；2）编辑操作建议写 operation_log。

涉及接口：GET /api/extraction/results；PUT /api/extraction/results/{rid}；POST /api/extraction/batch-delete-results；GET

/api/extraction/fused;GET /api/extraction/files-with-fields;GET /api/extraction/files/{fid}/fields。

涉及数据表：extract_result,file_info,field_dict,operation_log。

涉及界面：前端 src/views/Extraction/Index.vue。

5.5.5 UC-05 模板上传与字段识别

用例编号：UC-05

用例名称：模板上传与字段识别

主参与者：业务用户(USR)/ 系统管理员(ADM)

次要参与者：文件系统/LibreOffice(FS)

优先级：高

简要描述：用户上传 .xlsx/.xls/.docx 模板,后端 template_filler.identify_fields 自动识别 table_fill(表头横向、数据下行)与 field_fill(字段名+值竖向)两种模式,并将字段写入 template_info 与 template_mapping。

前置条件：1) 用户已登录;2) 模板文件类型属于 .xlsx/.xls/.docx 之一。

后置条件（成功）：1) 模板文件落盘 backend/templates/;2) template_info 写入(含 template_mode、field_count);3) template_mapping 写入字段清单(含位置、列名、单元格地址等)。

后置条件（失败）：返回错误,template_info 与 template_mapping 不写入。

触发条件：用户在「模板与填表」页点击「上传模板」并选择文件。

主成功场景：

1. 用户点击「上传模板」选择文件,可填写「填写要求」描述。

2. 前端 POST /api/templates/upload(multipart/form-data,含 file + requirement?)。

3. 后端校验大小、类型,落盘到 backend/templates/<uuid>_<原名>。

4. 后端调用 template_filler.identify_fields(path):

- .xlsx → openpyxl 加载,扫描首行/首列,启发式判断 table_fill / field_fill。
- .xls → xlrd 扫描表头(避免 openpyxl 不支持二进制流)。
- .docx → python-docx 扫描表格 + 段落,识别字段占位符。

5. 后端写入 template_info(name,template_mode,field_count,requirement,upload_user)。

6. 后端写入 template_mapping(每字段一

条:field_name,field_type,position/cell_address)。

7. 前端跳转「模板列表」展示新模板。

扩展事件流:

4a. 模板为 .doc 旧版: 先经 legacy_doc 转换为 .docx,再走 .docx 分支。

4b. 字段识别为 0: 仍创建 template_info(field_count=0),前端提示「未自动识别到字段,请手动维护或重新上传」。

4c. 文件含合并单元格: identify_fields 跨合并单元格识别,首格为字段名。

异常流:

- 落盘失败: 回滚 template_info,返回 500。

- 模板被加密 / 损坏: 返回 400「模板文件无法解析」。

特殊需求: 1) 模板文件保留原版,不在识别阶段做任何修改;2) field_dict 中已有的标准字段名优先匹配以利后续填表。

涉及接口: POST /api/templates/upload;GET /api/templates/list;DELETE /api/templates/{tid};PUT /api/templates/{tid}/requirement;GET /api/templates/{tid}/mappings。

涉及数据表: template_info,template_mapping,field_dict。

涉及界面: 前端 src/views/Templates/Index.vue。

5.5.6 UC-06 模板智能填表 smart_fill

用例编号: UC-06

用例名称: 模板智能填表 smart_fill

主参与者: 业务用户(USR)/ 系统管理员(ADM)

次要参与者: 大语言模型服务(LLM)、文件系统(FS)

优先级: 高

简要描述: 用户选择目标模板与可选数据源文件,触发 smart_fill 引擎:在保留模板合并单元格、表头样式、数字格式的前提下,按列名做模糊匹配 + 跳过英文表头/空行/分组行 + OCR 类脏字符清洗 + 自动汇总「合计、平均」列;如有图片字段则调用 image_filler(DISPIMG 或浮动图);输出文件路径写入 task_info.output_file。

前置条件: 1) 模板已上传且 template_info 存在;2) 已抽取的字段在 extract_result 中可用,或用户提供 source_file_ids。

后置条件 (成功): 1) 生成填好的 .xlsx/.docx 输出文件,落盘 backend/outputs/;2) task_info 写入(type='fill',status='done',output_file=<路径>);3) 前端可按 task_id 下载文件。

后置条件 (失败): task_info.status='failed',记录错误;不产出文件。

触发条件：用户在模板填表面板点击「执行填表」。

主成功场景：

1. 用户进入「模板与填表」，选择目标模板，粘贴或上传「用户要求」(Markdown 多段约束)，勾选 source_file_ids(可选)。

2. 前端 POST /api/templates/{tid}/fill，请求体含 {source_file_ids?, requirement?}。

3. 后端创建 task_info(type='fill',status='running')。

4. 后端聚合数据源：从 extract_result 取出指定文件的字段(若 source_file_ids 为空则取该用户全部已抽取字段)。

5. 调用 smart_fill.run(template_path, data_rows, requirement)：

   - 复制模板到 outputs/<uuid>_<原名>。
   - 按列名做模糊匹配(SequenceMatcher + 同义词字典)。
   - 跳过英文表头、空行、省份分组行(如「省份」「合计」标识行)。
   - 数据写入，保留单元格样式与数字格式。
   - 自动汇总「合计、平均」列(隐式或显式)。
   - 若字段为图片：image_filler 按模板风格写入(DISPIMG 或浮动图)。

6. 写出文件，task_info.output_file=<相对路径>,status='done'。

7. 前端展示填表历史，用户点击下载按钮 GET /api/templates/history/{task_id}/download。

扩展事件流：

1a. 用户要求中含筛选条件(如「常住人口 500 万以上」)：smart_fill + LLM 共同生效，LLM 解析筛选规则后参与数据筛选。

5a. 模板含 WPS DISPIMG 函数：image_filler 解析 cellimages.xml 后写入 =_xlfn.DISPIMG。

5b. 模板列名与数据字段无任何匹配项：留空并在 task_info 备注中记录，前端高亮提示。

异常流：

- 模板文件被占用 / 写入失败：task 失败，返回 500。

- LLM 配置缺失但用户要求需要 LLM 解析：跳过 LLM 处理，只做基础列匹配。

- 数据为空：输出文件为模板原样副本，task 标记 partial_success。

特殊需求：1) 输出文件名按 task_id 编址，避免中文文件名 URL 兼容问题；2) 输出文件保留模板的合并单元格、表头样式与数字格式；3) 大数据集(>1 万行)需在后续迭代中分批写入避免内存溢出。

涉及接口：POST /api/templates/{tid}/fill;

GET /api/templates/{tid}/fill-history;

DELETE /api/templates/history/{task_id};

GET /api/templates/history/{task_id}/download;

GET /api/templates/{tid}/download。

涉及数据表：
template_info,template_mapping,extract_result,task_info,file_info,field_dict。

涉及界面：前端 src/views/Templates/Fill.vue 与历史 Drawer。

5.5.7 UC-07 智能对话与 Agent 多轮工具调用

用例编号：UC-07

用例名称：智能对话与 Agent 多轮工具调用

主参与者：业务用户(USR)/ 系统管理员(ADM)

次要参与者：大语言模型服务(LLM)、文件系统(FS)

优先级：高

简要描述：用户在「智能对话」页发起提问;后端先以 vector_store.search(user_id=...) 召回相关切片+ 拼接已抽取字段摘要写入 system 提示;然后进入 tool_use 循环(MAX_TOOL_ROUNDS=12),每轮 LLM 可请求 12 项 Agent 工具中的任意一项(search_documents / extract_info / list_files / edit_document / analyze_document / format_document / create_spreadsheet / format_spreadsheet / add_chart / validate_spreadsheet / fill_template_smart / get_extract_results),tool_result 回灌后继续直到 LLM 不再请求工具,输出最终 Markdown 答复。

前置条件：1) 用户已登录;2) 已配置可用的 LLM API;3) 至少有一个 chat_session(若无,后端自动创建)。

后置条件（成功）：1) 用户消息与最终回答写入 chat_message;2) 涉及文件操作时输出文件落盘并可下载;3) chat_session.updated_at 刷新。

后置条件（失败）：失败的对话以 assistant 角色记录错误信息;chat_session 仍刷新。

触发条件：用户在对话框输入文字并发送。

主成功场景：

1. 用户输入问题,可选地勾选会话内已上传的多个 file_ids。
2. 前端 POST /api/chat/ask,含 {session_id, content, file_ids?}。
3. 后端写入 chat_message(role='user', session_id, content)。

4. 后端 vector_store.search(user_id=current_user_id, query=content) 召回 top-k 切片。

5. 后端拼接 system 提示: 角色定义 + 已抽取字段摘要 + 召回切片 + 可用工具说明。

6. 进入 tool_use 循环(round=1..12):

a. LLM 返回 tool_use(tool_name + input)。

b. 后端 execute_tool 在 doc_editor / smart_fill / vector_store / extractor 等服务内执行。

c. tool_result 回灌进对话历史。

d. 若 LLM 返回 final_text 则跳出循环。

7. 写入 chat_message(role='assistant', content=final_text, tool_trace=<工具调用轨迹>)。

8. 若期间产生输出文件(如 edit_document / fill_template_smart),file_info 同步更新,前端显示「下载」入口。

扩展事件流:

1a. 用户勾选了图片类附件: build_image_content_blocks 把图片以 base64 嵌入 Anthropic 多模态消息块。

6a. 工具执行抛错: 把 stack 摘要作为 tool_result(is_error=true) 回灌,LLM 自行决定是否换工具或终止。

6b. 达到 12 轮上限仍未终止: 强制终止,assistant 输出「已达最大轮次」并附已完成步骤摘要。

异常流:

- LLM 服务不可达: chat_message 写入 assistant 错误信息,前端弹出提示。

- vector_store 不可用: 跳过召回,只做纯 LLM 问答(降级)。

- session_id 不存在或非本人会话: 返回 403。

特殊需求: 1) 用户数据隔离(vector_store.search 强制按 user_id 过滤);2) 工具调用必须带审计(写 operation_log 或 chat_message.tool_trace);3) 12 轮工具上限可由系统配置调整。

涉及接口: POST /api/chat/ask;GET /api/chat/sessions;GET /api/chat/sessions/{sid}/messages;DELETE /api/chat/sessions/{sid}。

涉及数据表: chat_session,chat_message,file_info,extract_result,doc_chunk,system_config。

涉及界面: 前端 src/views/Chat/Index.vue。

5.5.8 UC-08 会话内附件上传与多文件勾选

用例编号：UC-08

用例名称：会话内附件上传与多文件勾选

主参与者：业务用户(USR)/ 系统管理员(ADM)

次要参与者：无

优先级：中

简要描述：在智能对话会话内,用户可上传文档或图片;文档触发后台解析任务(异步 done 后才进入「可勾选」),图片直接 done;勾选的多个 file_ids 在下一次 /api/chat/ask 中作为 Agent 的可操作上下文。

前置条件：1) 用户已登录;2) 当前会话已存在。

后置条件（成功）：附件文件落盘并写入 file_info(scope='chat',session_id),前端可勾选用于下一次提问。

后置条件（失败）：前端展示上传错误,不写入 file_info。

触发条件：用户点击对话框「+」选择文件。

主成功场景：

1. 用户点击「+」选择 1~N 份文件。

2. 前端 POST /api/chat/upload(multipart/form-data + session_id)。

3. 后端落盘 backend/uploads/chat/<sid>/,写入 file_info(scope='chat',session_id)。

4. 文档型：触发 BackgroundTasks 解析,与 UC-02 共用 document_parser;图片型:直接 parse_status='done'。

5. 前端轮询 GET /api/chat/files?session_id=<sid> 获取可选附件清单(含 parse_status)。

6. 用户在会话内勾选文件 file_ids,作为下一次 /api/chat/ask 的上下文。

7. 用户可点「下载」：GET /api/chat/files/{fid}/download。

扩展事件流：

4a. 文档解析失败：file_info.parse_status='failed',前端禁用勾选并显示错误。

6a. 用户勾选图片：在 UC-07 主成功场景 5 中,build_image_content_blocks 将其嵌入多模态消息。

异常流：

- session_id 非本人或不存在：返回 403。

- 文件类型不被支持：返回 415。

特殊需求：1) 会话内文件与「文档管理」中的文件共享 file_info 表，通过 scope 区分；2) 删除会话时建议级联归档（以避免误删用户主仓文件）。

涉及接口：POST /api/chat/upload；GET /api/chat/files；GET /api/chat/files/{fid}/download。

涉及数据表：file_info，chat_session，doc_chunk（若文档型）。

涉及界面：前端 src/views/Chat/Index.vue 的附件 Drawer。

5.5.9 UC-09 系统配置维护（LLM / Embedding / MCP）

用例编号：UC-09

用例名称：系统配置维护（LLM / Embedding / MCP）

主参与者：系统管理员（ADM）

次要参与者：无

优先级：高

简要描述：管理员在「系统设置」配置大模型 API（Base URL、Key、Model、Format=anthropic/openai）、Embedding 模型、MCP 开关等；保存到 system_config 表后由 config.py 热加载，无需重启即可对后续请求生效。

前置条件：当前用户为 admin。

后置条件（成功）：1) system_config 持久化；2) llm_service / vector_store / mcp_service 在下一次调用时读取新配置。

后置条件（失败）：数据库回滚，前端显示错误，旧配置仍生效。

触发条件：管理员进入「系统设置」并修改任一配置项后保存。

主成功场景：

1. 管理员进入「系统设置」页。
2. 前端 GET /api/settings/config 拉取当前配置。
3. 管理员修改字段（LLM Base URL、Key、Model、Format，Embedding Model，MCP_ENABLED 等）。
4. 前端 PUT /api/settings/config 提交改动。
5. 后端校验权限（role=='admin'）、字段格式。
6. 后端写入 system_config（key/value），写 operation_log。
7. config.py 热加载新值。
8. 前端刷新展示。

扩展事件流：

5a. 当前用户非 admin: 返回 403。

6a. LLM Key 字段建议加密存储(后续迭代),当前以明文 + 数据库访问控制保障。

异常流:

- 配置项不存在: 自动 INSERT 而非 UPDATE。

- 数据库写失败: 回滚,旧值保留。

特殊需求: 1) LLM API Key、SECRET_KEY 等敏感配置不入版本库;2) 修改 MCP_ENABLED 后, /api/mcp/* 在下一次请求时启停。

涉及接口: GET /api/settings/config;PUT /api/settings/config。

涉及数据表: system_config,operation_log。

涉及界面: 前端 src/views/Settings/System.vue。

5.5.10 UC-10 用户管理与操作日志审计

用例编号: UC-10

用例名称: 用户管理与操作日志审计

主参与者: 系统管理员(ADM)

次要参与者: 无

优先级: 高

简要描述: 管理员在「系统设置-用户管理」页查看用户列表,可改角色、重置密码、删除用户(内置超级管理员受保护);在「操作日志」页分页浏览 operation_log。普通用户仅可见与本人相关的日志。

前置条件: 当前用户为 admin。

后置条件（成功）: 1) 用户记录被修改/删除;2) 关键操作写入 operation_log。

后置条件（失败）: 返回错误,数据库回滚。

触发条件: 管理员进入「用户管理」或「操作日志」页。

主成功场景:

1. 管理员进入「系统设置-用户管理」页。

2. 前端 GET /api/settings/users(分页 + 关键词)。

3. 管理员对某用户点「修改角色」: 前端 PUT /api/settings/users/{uid},含 {role}。

4. 后端校验:不可降级 SUPER_ADMIN_USERNAME(若配置)、不可改本人。

5. 后端写入 users 表,写 operation_log(action='update_user_role',target_id=uid,operator=current_user_id)。

6. 管理员点「重置密码」: PUT /api/settings/users/{uid} 含 {password=<新密码>},服务端 bcrypt 哈希后保存。

7. 管理员点「删除」: DELETE /api/settings/users/{uid}; 后端检查不可删除内置超级管理员;级联或软删除按设计。

8. 管理员进入「操作日志」: GET /api/settings/logs(分页 +user_id 过滤)。

扩展事件流:

4a. 试图修改超级管理员: 返回 403「内置超级管理员受保护」。

7a. 试图删除自己: 返回 400「不可删除当前登录用户」。

8a. 非 admin 调用 /api/settings/logs: 后端按 user_id=current_user_id 过滤(只看本人相关)。

异常流:

- 用户不存在: 返回 404。

- 数据库异常: 回滚,前端提示「操作失败」。

特殊需求: 1) 关键写操作必须写 operation_log;2) 删除用户建议软删除以保留审计链路(后续迭代)。

涉及接口: GET /api/settings/users;PUT /api/settings/users/{uid};DELETE /api/settings/users/{uid};GET /api/settings/logs。

涉及数据表: users,operation_log。

涉及界面: 前端 src/views/Settings/Users.vue;Logs.vue。

5.5.11 UC-11 OpenAPI 调用(X-API-Key)

用例编号: UC-11

用例名称: OpenAPI 调用(X-API-Key)

主参与者: 开放接口调用方(API,如 n8n / Coze / Dify)

次要参与者: 大语言模型服务(LLM)

优先级: 中

简要描述: 外部低代码平台在请求头加 X-API-Key 调用 /api/open/* 共 9 个端点(上传 → 解析 → LLM 抽取 → 语义搜索 → 轻量 RAG 问答 → 文件列表),返回 JSON。GET /api/open/ 提供索引,便于 n8n 节点的 schema 自动发现。

前置条件: 调用方已获得有效的 X-API-Key(与服务端 WEBHOOK_KEY 或 SECRET_KEY 之一匹配)。

后置条件（成功）: 请求处理完成并返回 JSON;涉及上传/抽取的请求会写入 file_info / extract_result。

后置条件（失败）: 返回标准错误 JSON({error_code, message});不写入业务数

据。

触发条件：外部 HTTP 客户端发起请求至 /api/open/*。

主成功场景：

1. 调用方在请求头加 X-API-Key: <key>。

2. 中间件 verify_api_key 校验 key 与允许的密钥列表。

3. 路由分发到具体端点：

- GET /api/open/ → API 索引(便于 schema 自动发现)。
- POST /api/open/upload → 上传并触发解析。
- POST /api/open/extract → 对已上传文件抽取。
- POST /api/open/extract-text → 对纯文本抽取。
- GET /api/open/files/{fid}/fields → 查询已抽取字段。
- GET /api/open/fused → 查询融合数据。
- POST /api/open/semantic-search → 语义搜索。
- POST /api/open/chat → 轻量 RAG 问答(单轮,不进入 12 轮 Agent)。
- GET /api/open/files → 文件列表。

4. 端点完成业务后返回标准 JSON。

扩展事件流：

2a. X-API-Key 缺失：返回 401。

2b. X-API-Key 不匹配：返回 403。

3a. /api/open/upload 同 UC-02 主成功场景 3-7。

3b. /api/open/extract 同 UC-03 主成功场景 4-9。

异常流：

- 上游 LLM 限流：返回 503,前端可触发外部重试。

- 文件类型不支持：返回 415。

特殊需求：1) Key 通过环境变量配置,严禁入版本库;2) 建议为不同调用方使用不同 Key 并支持轮换(后续迭代);3) 所有 /api/open 调用建议写 operation_log 以便审计。

涉及接口：/api/open/* 共 9 个端点。

涉及数据表：file_info,doc_chunk,extract_result,operation_log。

涉及界面：无(供外部平台直接调用);n8n 节点 / Coze 工作流配置面板。

5.5.12 UC-12 MCP 工具调用(JSON-RPC + SSE)

用例编号：UC-12

用例名称：MCP 工具调用(JSON-RPC + SSE)

主参与者：MCP 客户端(MCP,如 Claude Desktop / Cursor)

次要参与者：大语言模型服务(LLM,客户端侧)

优先级：中

简要描述:MCP 客户端通过 JSON-RPC 调用 /api/mcp/jsonrpc 完成 initialize / tools/list / tools/call;流式场景使用 /api/mcp/sse 推送事件、/api/mcp/message 上行消息(Bearer 鉴权)。tools/call 由 mcp_service 派发到 13 个具体工具(与对话内 Agent 工具集大体一致)。

前置条件：1) 服务端 MCP_ENABLED=true;2) 客户端持有有效的 MCP_TOKEN(用于 SSE Bearer)。

后置条件（成功）：客户端收到 JSON-RPC 响应或 SSE 事件;涉及业务的工具会写入相应数据表。

后置条件（失败）：返回标准 JSON-RPC 错误对象;不写业务数据。

触发条件：MCP 客户端发起 JSON-RPC 请求或建立 SSE 长连接。

主成功场景：

1. 客户端发 initialize: POST /api/mcp/jsonrpc {method:'initialize',params:{protocolVersion:'2024-11-05'}}。

2. 服务端返回 protocolVersion=2024-11-05 与 serverInfo。

3. 客户端发 tools/list,服务端返回 13 个工具的 schema。

4. 客户端发 tools/call {name,arguments},mcp_service 派发到对应工具实现。

5. 工具执行结果以 JSON-RPC result 返回。

6. 流式场景：客户端连接 GET /api/mcp/sse(Header: Authorization: Bearer <MCP_TOKEN>),服务端持续推送 ping/进度事件;

客户端通过 POST /api/mcp/message 上行消息。

扩展事件流：

1a. MCP_ENABLED=false: 所有 /api/mcp/* 返回 503「MCP 未启用」。

4a. tools/call 名称不存在：返回 JSON-RPC -32601 method not found。

6a. SSE Bearer 校验失败：返回 401。

异常流：

- 工具内部抛错：返回 JSON-RPC -32000 server error,error.data 含 stack

摘要。

- 长连接被代理超时切断：客户端按指数退避重连。

特殊需求：1) JSON-RPC 严格遵循 2.0;2) protocolVersion 与 Anthropic MCP 规范保持一致(2024-11-05);3) SSE 心跳间隔 ≤ 30s 以穿透常见反向代理。

涉及接口:POST /api/mcp/jsonrpc;GET /api/mcp/sse;POST /api/mcp/message;GET /api/mcp/tools。

涉及数据表：依赖具体工具(可能涉及 file_info / extract_result / template_info / task_info)。

涉及界面：无(MCP 客户端侧 UI,如 Claude Desktop 设置中的 MCP Server 配置)。

5.5.13 UC-13 主题与个性化偏好持久化

用例编号：UC-13

用例名称：主题与个性化偏好持久化

主参与者：业务用户(USR)/ 系统管理员(ADM)

次要参与者：无

优先级：低

简要描述：用户在右下角悬浮入口的「个性化中心」中切换 8 套预设主题或编辑自定义主题(基础色/四色背景渐变/卡片毛玻璃强度/图表调色板),并可一键开关自定义鼠标跟随;所有偏好通过 localStorage 持久化,跨会话生效。响应式断点 <768px 自动切换 el-drawer 抽屉导航。

前置条件：用户已进入任意已登录页面。

后置条件（成功）：1) localStorage 中持久化主题键值与开关;2) CSS 变量更新,页面立即生效;3) 下次登录或刷新仍生效。

后置条件（失败）：若 localStorage 不可用(隐私模式),则当前会话临时生效,无持久化。

触发条件：用户点击右下角「个性化中心」入口并修改任一项。

主成功场景：

1. 用户点击右下角悬浮入口,弹出「个性化中心」面板。
2. 用户切换预设主题或拖动颜色编辑器调整自定义主题。
3. 前端 themeManager 修改 :root 上的 CSS 变量。
4. 同步写入 localStorage(key='docfusion_theme', value=<JSON>)。
5. 用户开关「自定义鼠标跟随」,前端注册/注销 mousemove 监听并写入 localStorage。
6. 用户关闭面板,主题保持生效。

7. 下次进入或刷新时,index.html 内联脚本提前读取 localStorage 注入 CSS 变量,避免主题闪烁。

扩展事件流:

5a. 触控设备(无 mouse 事件): 自动隐藏鼠标跟随开关。

1a. 视口宽度 < 768px: 顶部菜单收起为 el-drawer,适配移动端。

异常流:

- localStorage 不可用(隐私模式 / 配额满): 仅在当前会话生效,提示用户「主题偏好暂不能保存」。

特殊需求: 1) 偏好为前端纯本地状态,不进入后端 users 表(后续若需多端同步可迁移到 users 扩展字段);2) 主题切换不触发整页重渲染,仅 CSS 变量更新。

涉及接口: 无(纯前端)。

涉及数据表: 无(localStorage)。

涉及界面: 前端 src/components/PersonalCenter.vue;src/utils/themeManager.ts。

5.6 用例与接口/数据表追溯关系

下表汇总 13 个用例与数据表的对应关系,便于评审与回溯。

表 5-2 用例与接口/数据表追溯关系表

用例编号	接口列表	关联数据表
UC-01	/api/auth/login, /me	users
UC-02	/api/documents/upload, /reparse, /list, /batch-delete	file_info, doc_chunk, task_info
UC-03	/api/documents/{fid}/extract, /extract-all	extract_result, file_info, doc_chunk, field_dict, task_info, system_config
UC-04	/api/extraction/results, /fused, /files/{fid}/fields, /batch-delete-results	extract_result, file_info, field_dict, operation_log
UC-05	/api/templates/upload, /list, /mappings, /requirement	template_info, template_mapping, field_dict
UC-06	/api/templates/{tid}/fill, /fill-history, /history/{task_id}/download	template_info, template_mapping, extract_result, task_info, file_info, field_dict

UC-07	/api/chat/ask, /sessions, /messages	chat_session, chat_message, file_info, extract_result, doc_chunk, system_config
UC-08	/api/chat/upload, /files, /files/{fid}/download	file_info, chat_session, doc_chunk
UC-09	/api/settings/config (GET/PUT)	system_config, operation_log
UC-10	/api/settings/users, /logs	users, operation_log
UC-11	/api/open/* (9 个端点)	file_info, doc_chunk, extract_result, operation_log
UC-12	/api/mcp/jsonrpc, /sse, /message, /tools	依赖具体工具（可能涉及 file_info /extract_result/template_info /task_info）
UC-13	无后端接口	无（localStorage）

六、系统的设计思路与结构

6.1 设计思路

1. 以“文档进、知识出、可成稿、可运营”为目标，遵循模块化、可配置、可扩展原则：

2. 针对非结构化数据：解析与抽取管道将文本转为可计算对象，降低人工复制粘贴。

3. 针对业务成稿：模板与填表链路把抽取结果映射到版式与字段约束，提升一致性。

4. 针对核验与解释：对话与检索能力帮助用户定位依据、补充上下文。

5. 针对落地运维：配置项入库、接口清晰、前后端分离，便于分期迭代与对接。

6.2 系统架构

系统采用前后端分离架构，逻辑上可分为展示层、接口层、领域服务层与数据层。

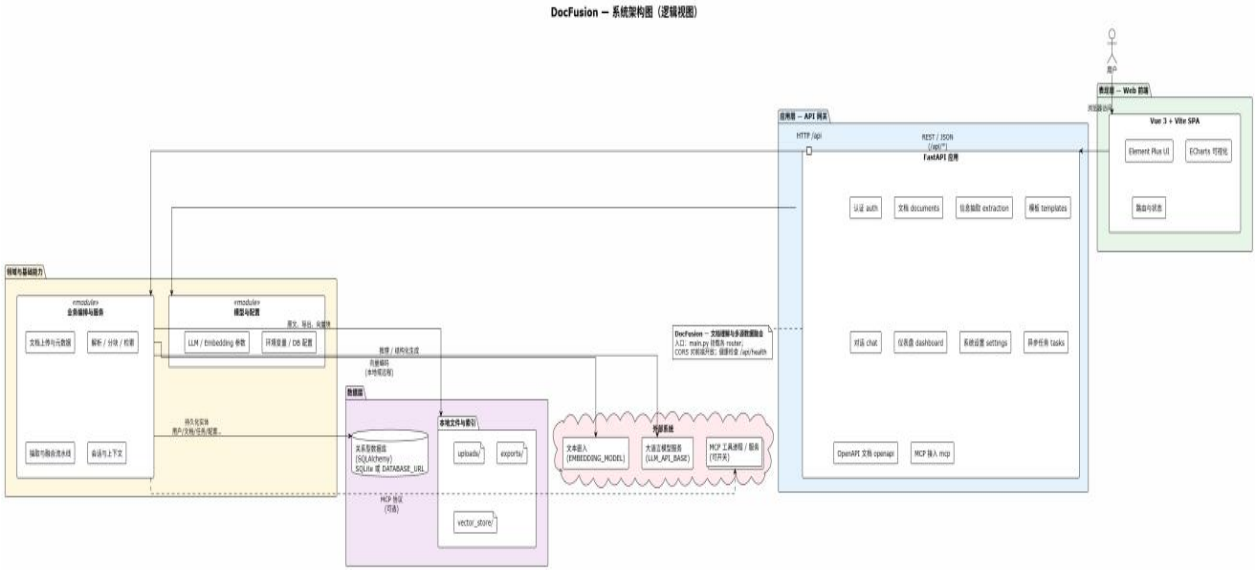


图 6-1 系统架构图

6.3 典型业务流程与数据流

以“上传文档 ➜ 抽取 ➜ 模板填表 ➜ 导出/任务回传”为例，数据流要点如下：

1. 用户上传文档至后端，文件落盘并写入业务记录；解析服务生成可供抽取的文本视图。

2. 抽取服务结合规则、模型与过滤策略输出结构化字段，前端分页或详情展示。

3. 用户在模板模块绑定需求与素材，填表服务调用大模型等能力生成或修订内容，并可触发导出。

4. 任务模块可跟踪长时操作状态，便于异步完成与结果下载。

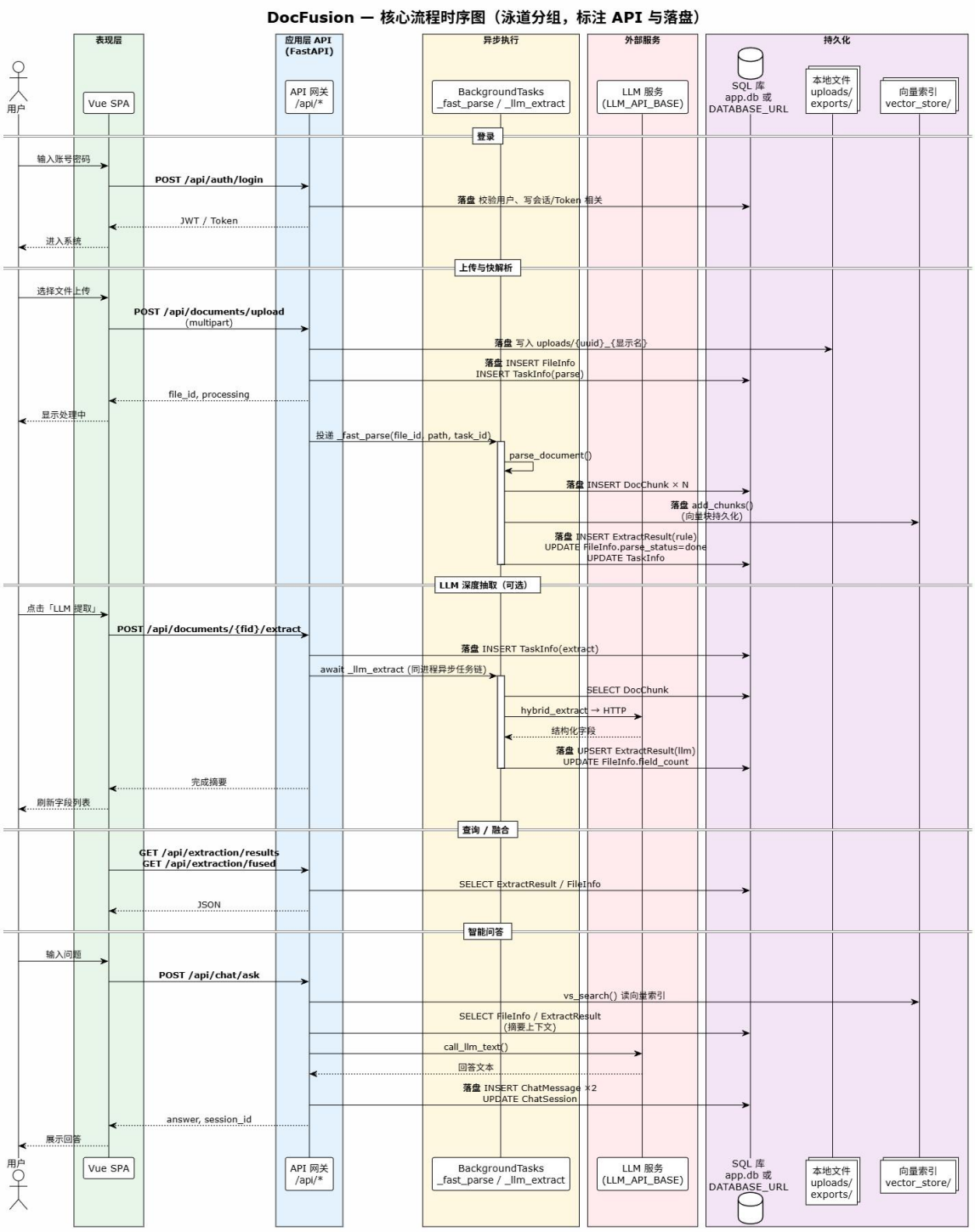


图 6-2 典型业务流程图

6.4 安全与非功能性设计

1. 认证与授权：登录后颁发 JWT；接口按用户身份隔离数据（需在生产环境完善细粒度权限模型）。

2. 传输安全：生产环境建议全站 HTTPS 与网关限流。

3. 数据安全：敏感配置不入库明文于版本库；定期备份数据库与上传目录。

4. 可观测性：后续迭代补充结构化日志、审计表与监控告警。

七、业务流程图

依据真实代码流（routers + services）梳理 8 张关键流程图。

7.1 用户登录与 JWT 鉴权流程

用户在登录页输入账号密码 → 前端 POST /api/auth/login → 后端 bcrypt 校验 → 通过后签发 JWT(HS256, 1440 分钟有效) → 前端写入 sessionStorage 的 token 字段 → axios 拦截器在每个请求 Header 自动注入 Authorization: Bearer <token>；401 时自动跳转登录页。

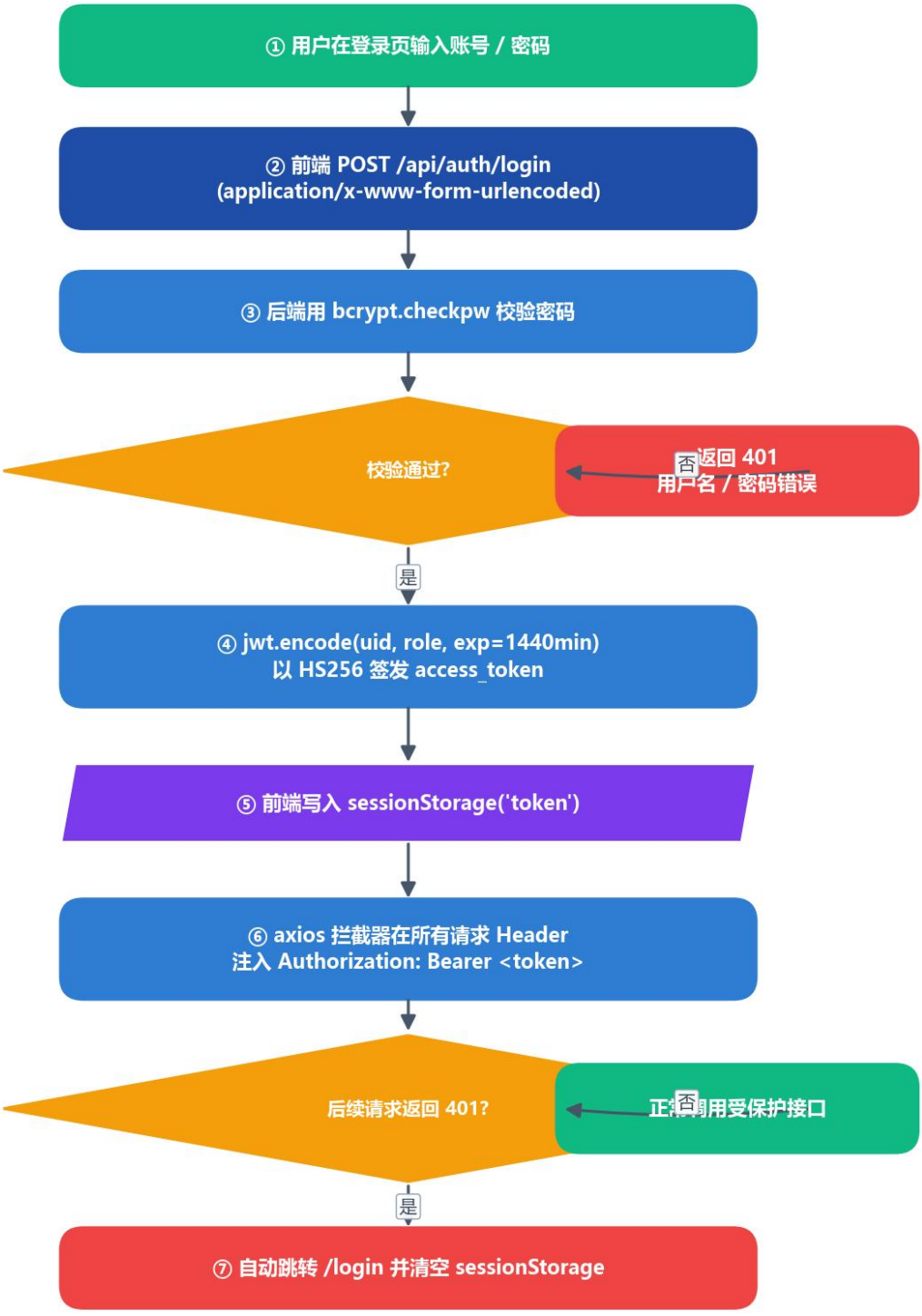


图 7-1 用户登录与 JWT 鉴权流程

7.2 文档上传与异步解析流程

用户选择文件 → POST /api/documents/upload（支持多文件） → 落盘到 backend/uploads/ + 写入 file_info 记录（parse_status=pending） → 触发 BackgroundTasks 解析 → document_parser 按文件类型分发(pdf/docx/xlsx/xls/img) → 切片写入 doc_chunk → 同步生成向量索引 → 解析完成后将 parse_status 置为 done，更新 chunk_count。

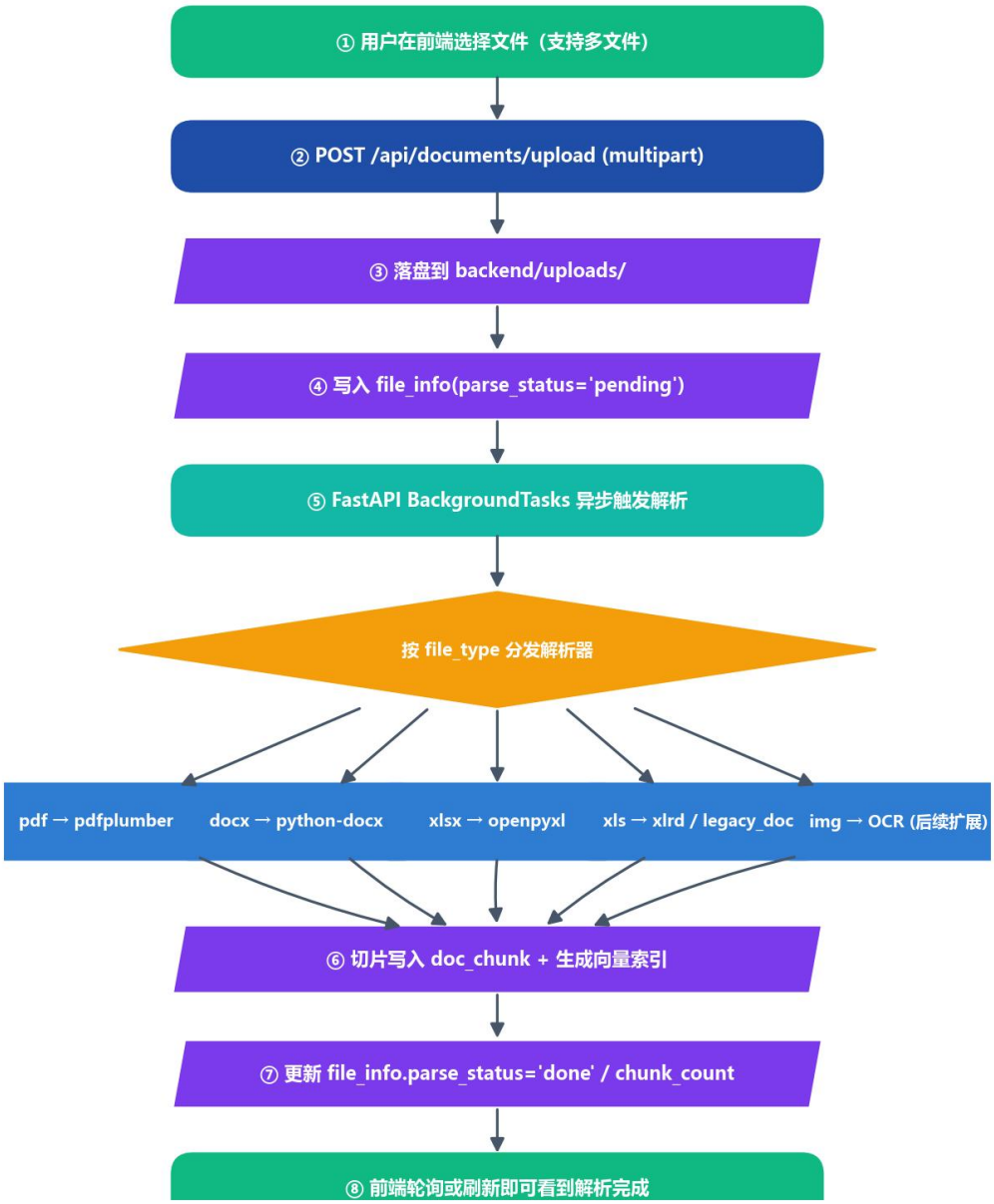


图 7-2 文档上传与异步解析流程

7.3 信息抽取（hybrid_extract）流程

POST /api/documents/{fid}/extract → 读取该文件全部 doc_chunk → 调用 extractor.hybrid_extract：①rule_extract 用多类正则（电话/邮箱/日期/身份证/金额/统计句/键值对/增速 等）做粗抽 ②llm_service.extract_fields 用 LLM 强制输出 JSON 做细抽 → fusion 服务按字段别名归一 + 来源加权 + 多值投票 → 写入 extract_result 表 → 前端在「信息抽取」页可查看、编辑、批量删除。

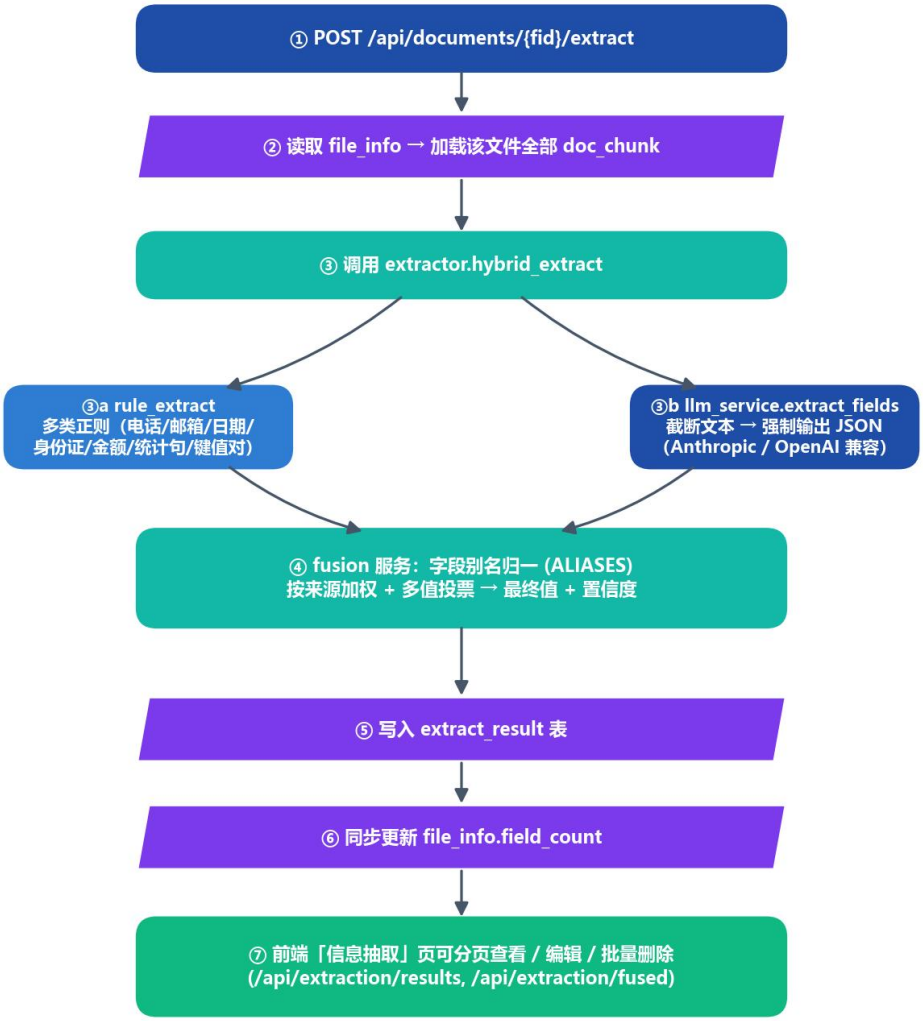


图 7-3 信息抽取（hybrid_extract）流程

7.4 模板上传与字段识别流程

用户上传模板 + 填写要求 → POST /api/templates/upload → template_filler.identify_fields 按 .xlsx/.xls/.docx 分发 → 区分 table_fill（表头横向）与 field_fill（字段名+值竖向）两种模式 → 旧版 .xls 走 xlrd 扫描表头 → 把识别结果写入 template_info（field_count、template_mode）和 template_mapping。

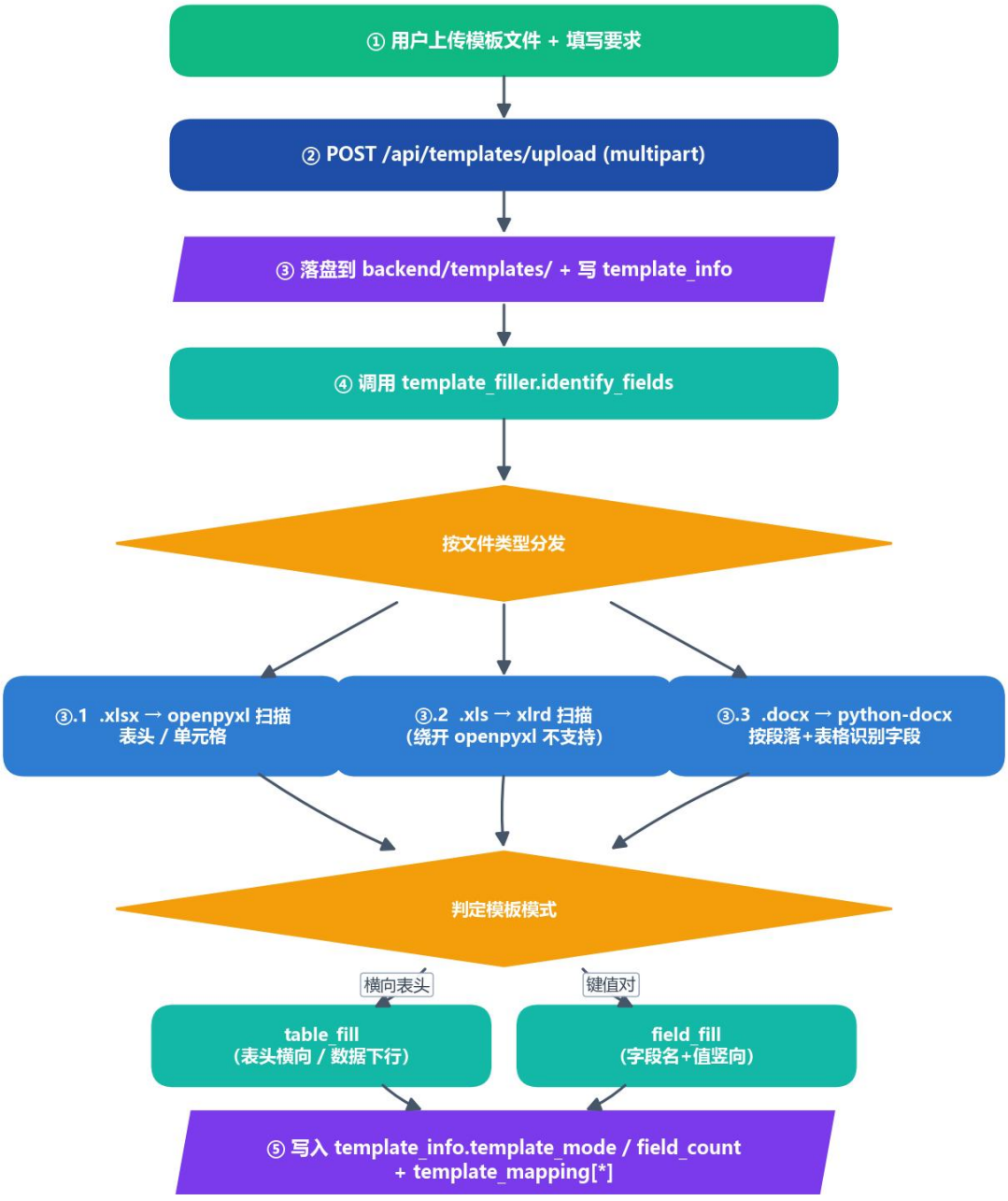


图 7-4 模板上传与字段识别流程

7.5 模板自动填表（smart_fill）流程

POST /api/templates/{tid}/fill（可选 source_file_ids）→ 取目标抽取数据集 → smart_fill 按列名做模糊匹配 + 跳过英文表头/空行/省份分组行 + OCR 类脏字符清洗 → 在保留模板合并单元格、表头样式与数字格式的前提下写入数据 → 自动汇总「合计、平均」类列 → 如有图片字段调用 image_filler（DISPIMG 或浮动图）→ 输出文件路径写入 task_info.output_file。



图 7-5 模板自动填表（smart_fill）流程

7.6 智能对话 Agent 多轮工具调用流程

POST /api/chat/ask → vector_store.search(user_id=…) 召回相关切片 + 拼接已抽取字段摘要写入 system → 进入 tool_use 循环（最多 12 轮）：LLM 返回 tool_use → execute_tool 在 doc_editor / smart_fill / vector_store 等服务内执行（search_documents / extract_info / list_files / edit_document / analyze_document / format_document / create_spreadsheet / format_spreadsheet / add_chart / validate_spreadsheet / fill_template_smart / get_extract_results）→ tool_result 回灌进对话 → 循环直到 LLM 不再请求工具，输出最终 Markdown 答复。

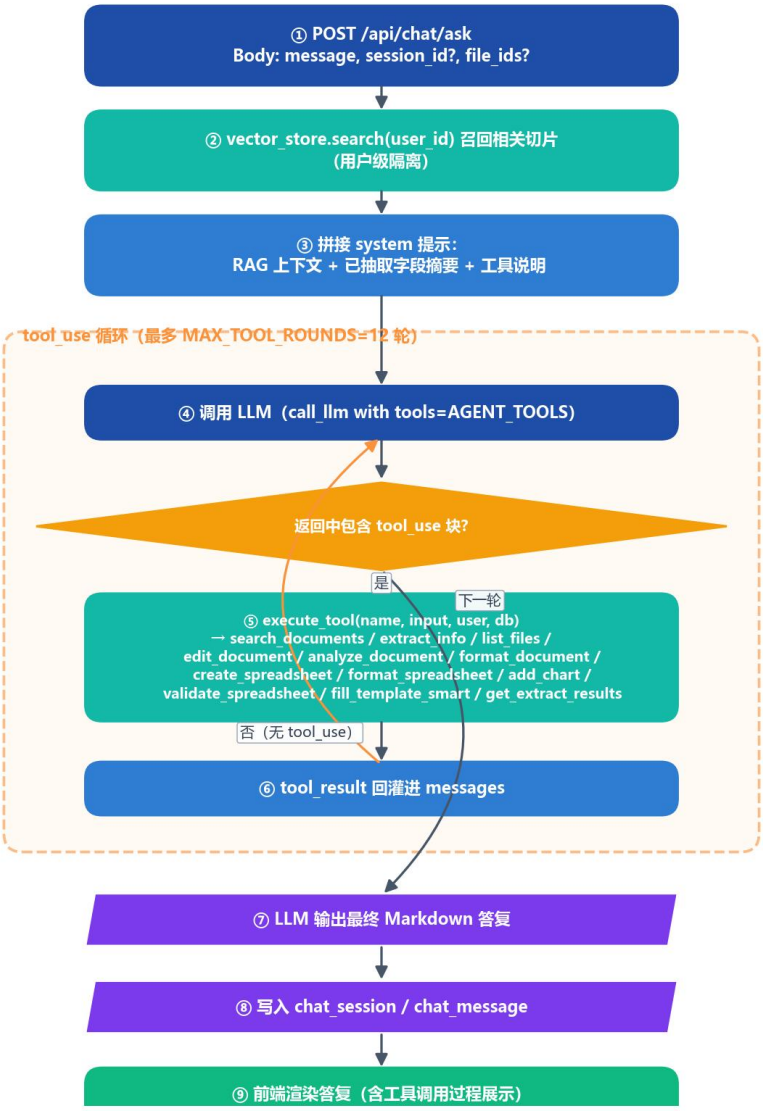


图 7-6 智能对话 Agent 多轮工具调用流程

7.7OpenAPI 调用流程（n8n / Coze 集成）

外部平台在请求头加 X-API-Key（与 WEBHOOK_KEY 或 SECRET_KEY 之一匹配）➞ /api/open/* 处理（上传 ➞ 解析 ➞ LLM 抽取 ➞ 语义搜索 ➞ 轻量问答 ➞ 文件列表）➞ 返回 JSON。GET /api/open/ 提供索引，便于 n8n 节点的 schema 自动发现。

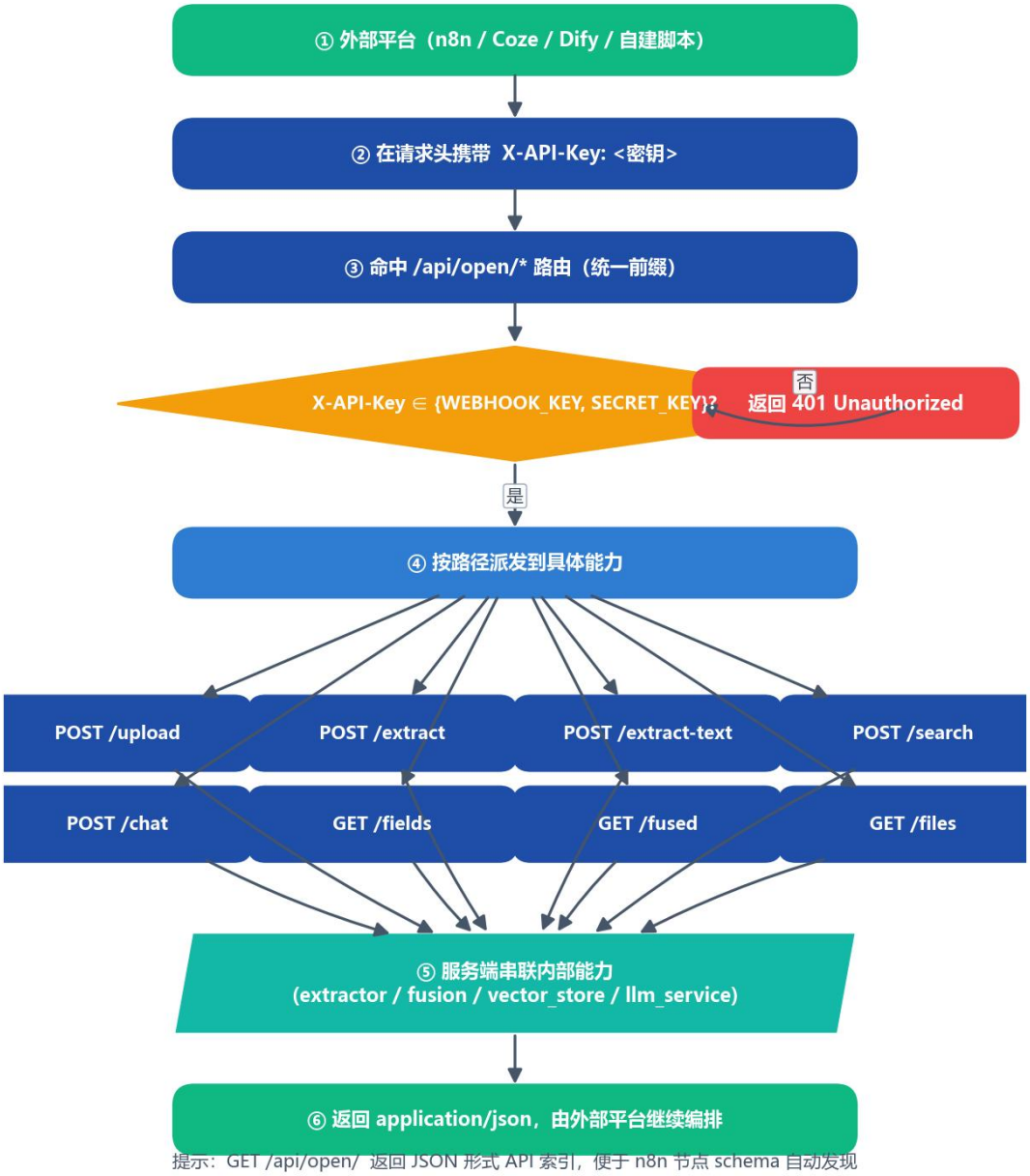


图 7-7 OpenAPI 调用流程（n8n / Coze 集成）

7.8MCP 工具调用流程（JSON-RPC + SSE）

MCP 客户端发起 initialize ➞ /api/mcp/jsonrpc 返回 protocolVersion

2024-11-05、serverInfo → tools/list 返回工具清单 → tools/call 由 mcp_service 派发到 13 个具体工具 → 流式场景使用 /api/mcp/sse 推送事件、/api/mcp/message 上行消息（Bearer 鉴权）。

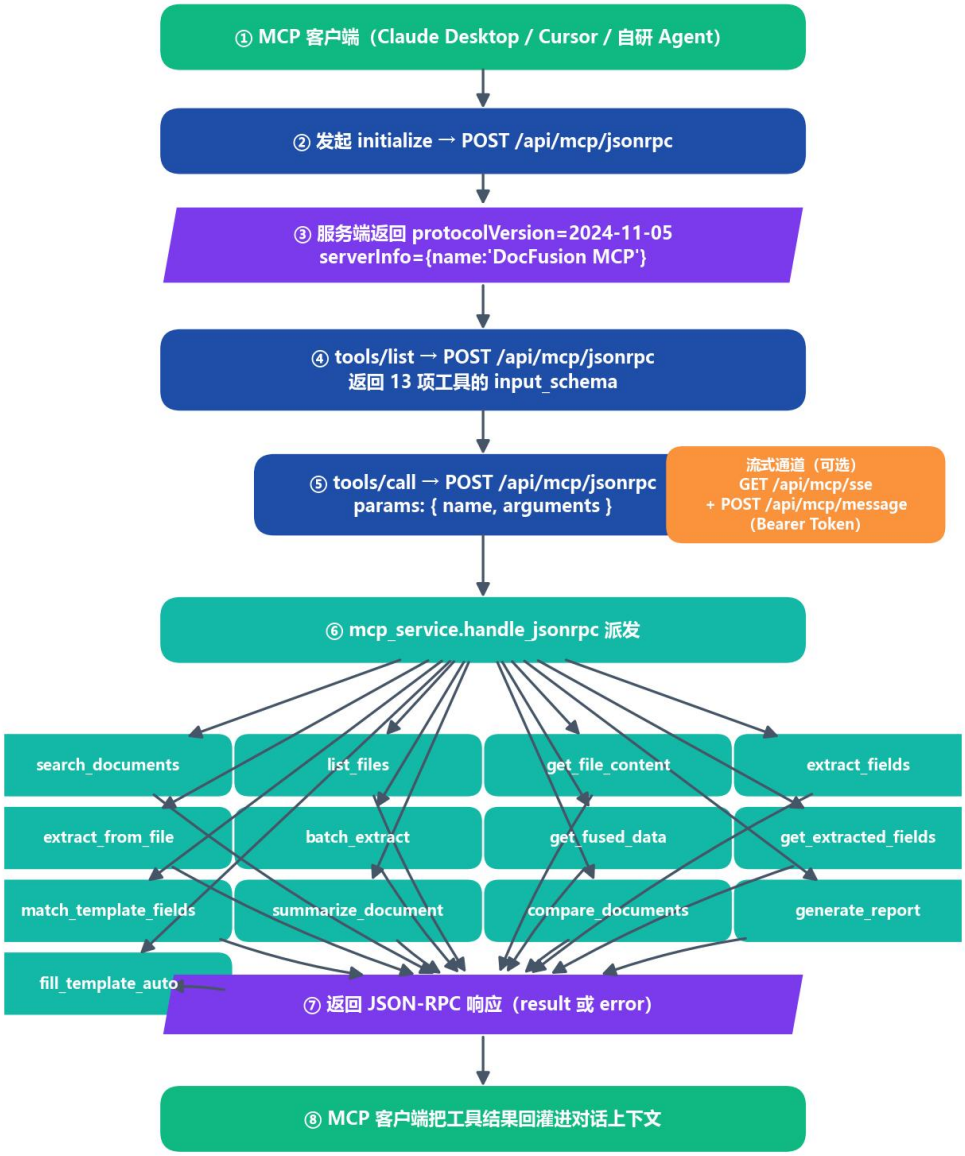


图 7-8 MCP 工具调用流程（JSON-RPC + SSE）

八、系统建模(UML 补充)

8.1 类图 / 领域模型

领域模型是后端 SQLAlchemy ORM 模型的概念视图,展示 12 个核心实体及其属性、关键方法、关联(1—*)、聚合(部分-整体)与依赖(..>)关系。聚合用 *-- 标识,该端为「整体」,删除时级联清理「部分」。

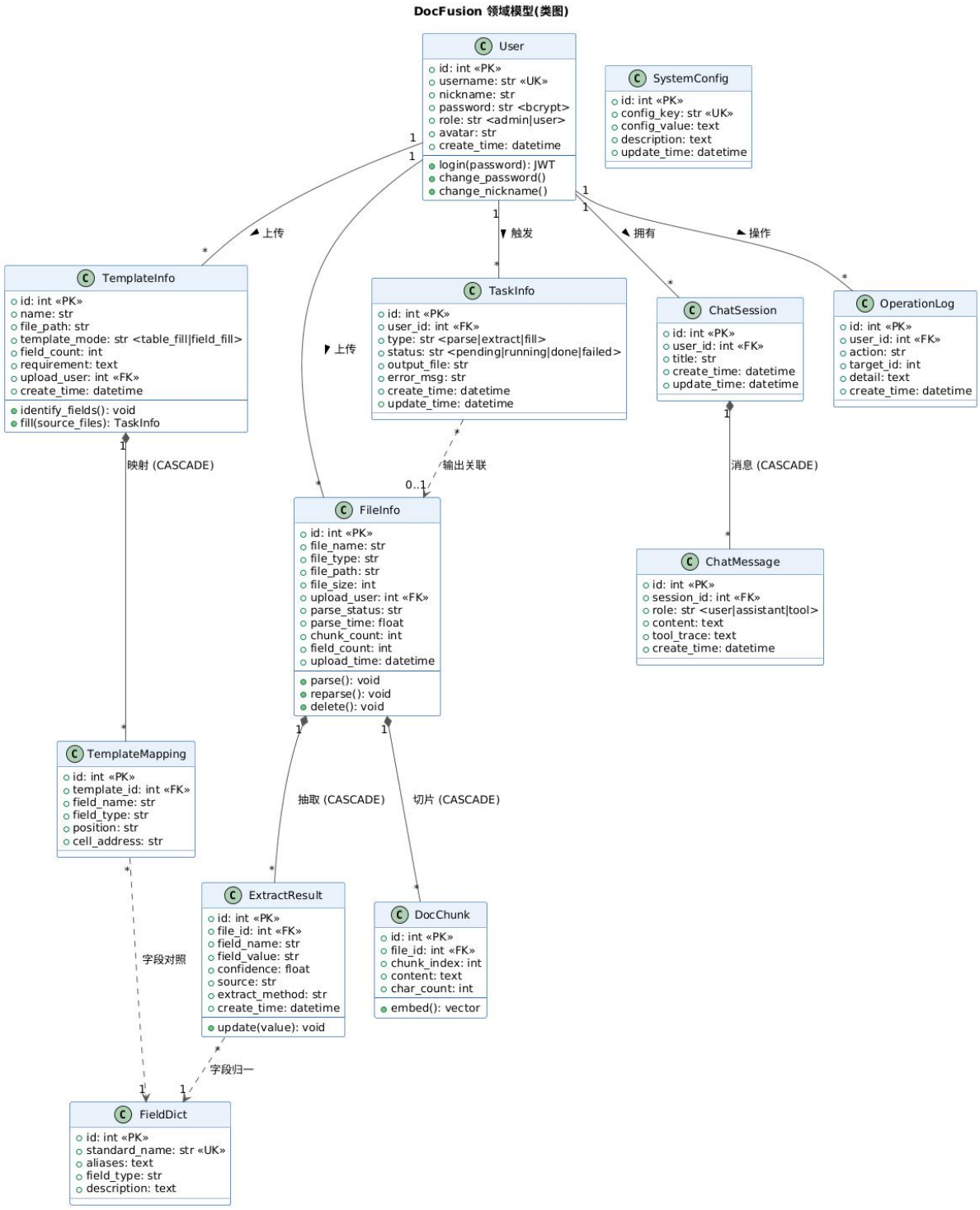


图 8-1 DocFusion 领域模型(类图)

类图要点:

- 强聚合(部分随整体删除):FileInfo→DocChunk、FileInfo→ExtractResult、TemplateInfo→TemplateMapping、ChatSession→ChatMessage。
- 弱关联(独立生命周期,但有归属):User 拥有 FileInfo / TemplateInfo / TaskInfo / ChatSession / OperationLog。
- 字典依赖:ExtractResult、TemplateMapping ..> FieldDict(字段归一)。
- 输出关联:TaskInfo ..> FileInfo(0..1,某些任务产出新的 file_info,如填表输出)。

8.2 时序图(Sequence Diagrams)

围绕 6 个最关键的业务路径,补充 UML 时序图,精确刻画参与者(Actor)、组件、外部系统之间的调用顺序、并发分支与异常处理。

8.2.1 SD-01 用户登录与 JWT 鉴权

对应用例:UC-01。展示登录请求 → bcrypt 校验 → JWT 签发 → 后续请求自动注入 Bearer Token → 401 触发跳转登录页的全链路。

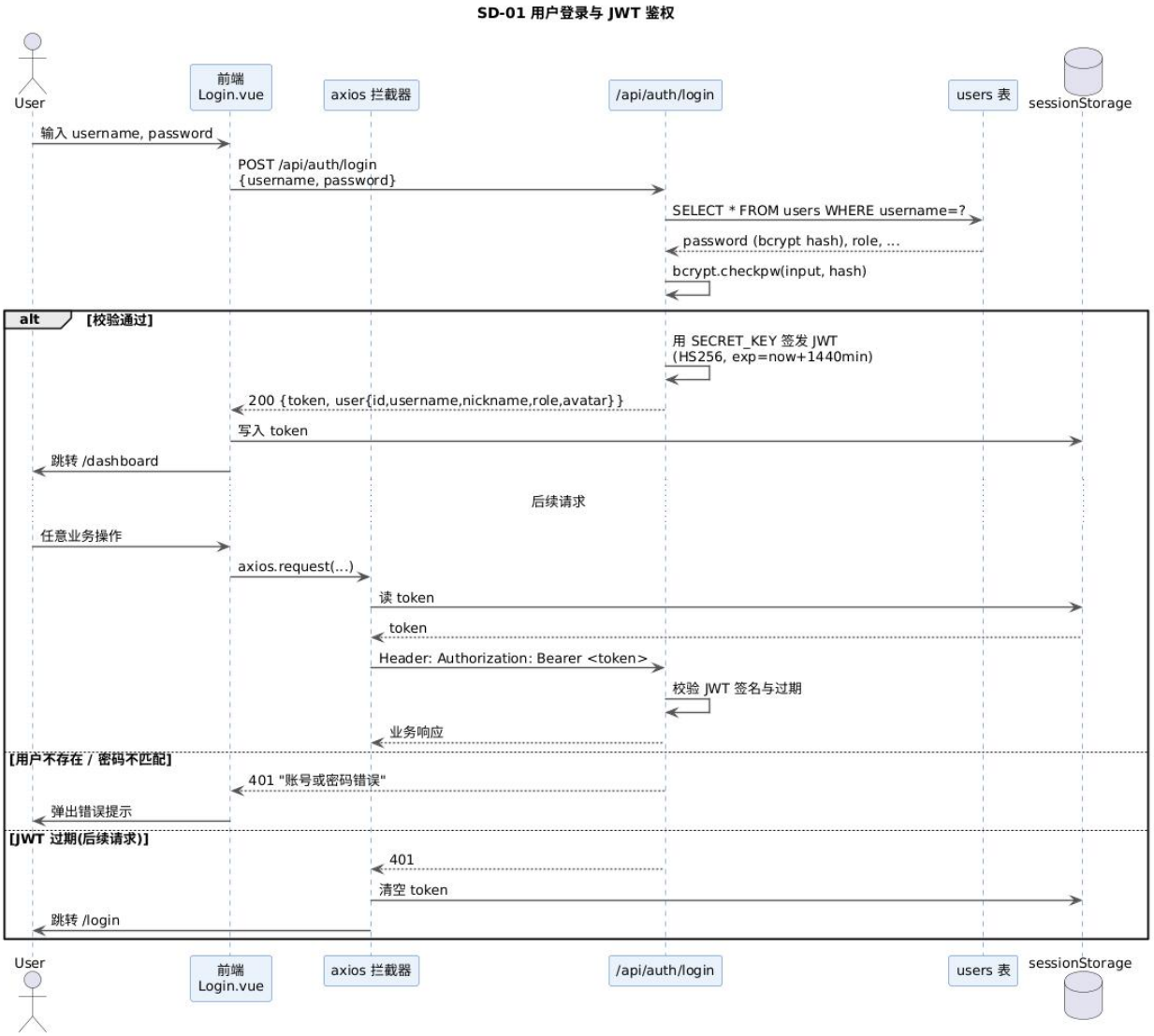


图 8-2 SD-01 用户登录与 JWT 鉴权时序图

8.2.2 SD-02 文档上传与异步解析

对应用例:UC-02。展示上传 → 落盘 → 写 file_info(pending) → BackgroundTasks 异步触发 → document_parser 按 file_type 分发(含 .doc → .docx 旁路) → 切片入库 + 向量化 → 状态置 done → 前端轮询。

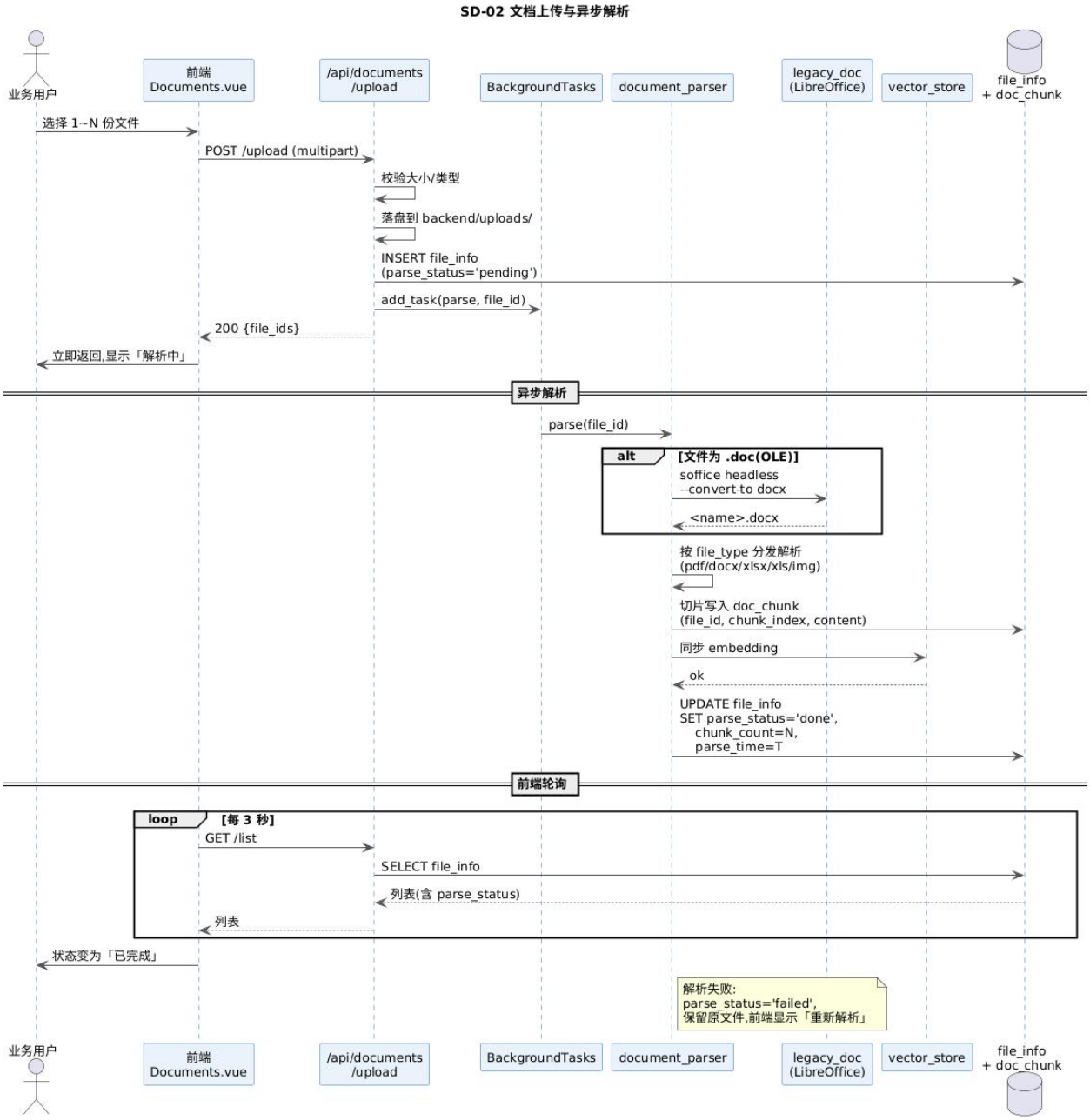


图 8-3 SD-02 文档上传与异步解析时序图

8.2.3 SD-03 信息抽取(hybrid_extract)

对应用例:UC-03。展示 task 创建 → 切片读取 → rule_extract 粗抽 → llm_service 强制 JSON 输出细抽 → fusion 按字段别名归一 + 来源加权 + 多值投票 → 写入 extract_result 与 task_info。

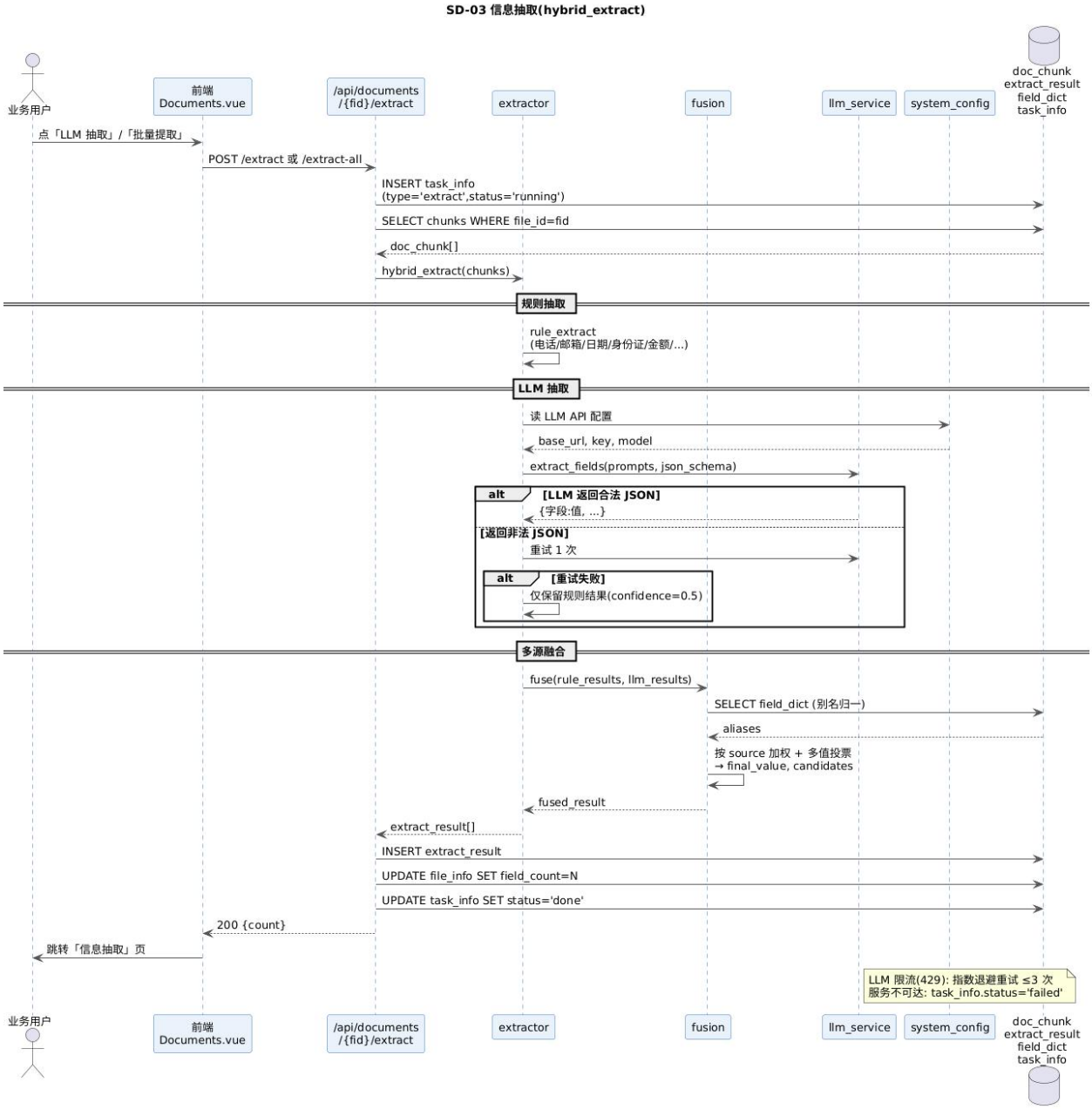


图 8-4 SD-03 信息抽取时序图

8.2.4 SD-04 模板智能填表 smart_fill

对应用例:UC-06。展示数据源聚合 → LLM 解析筛选要求(可选) → smart_fill 复制模板 → 模糊列匹配 → 数据写入(保留样式)→ 自动汇总 → image_filler 写入图片(DISPIMG/浮动图)→ 输出落盘 → 用户下载。

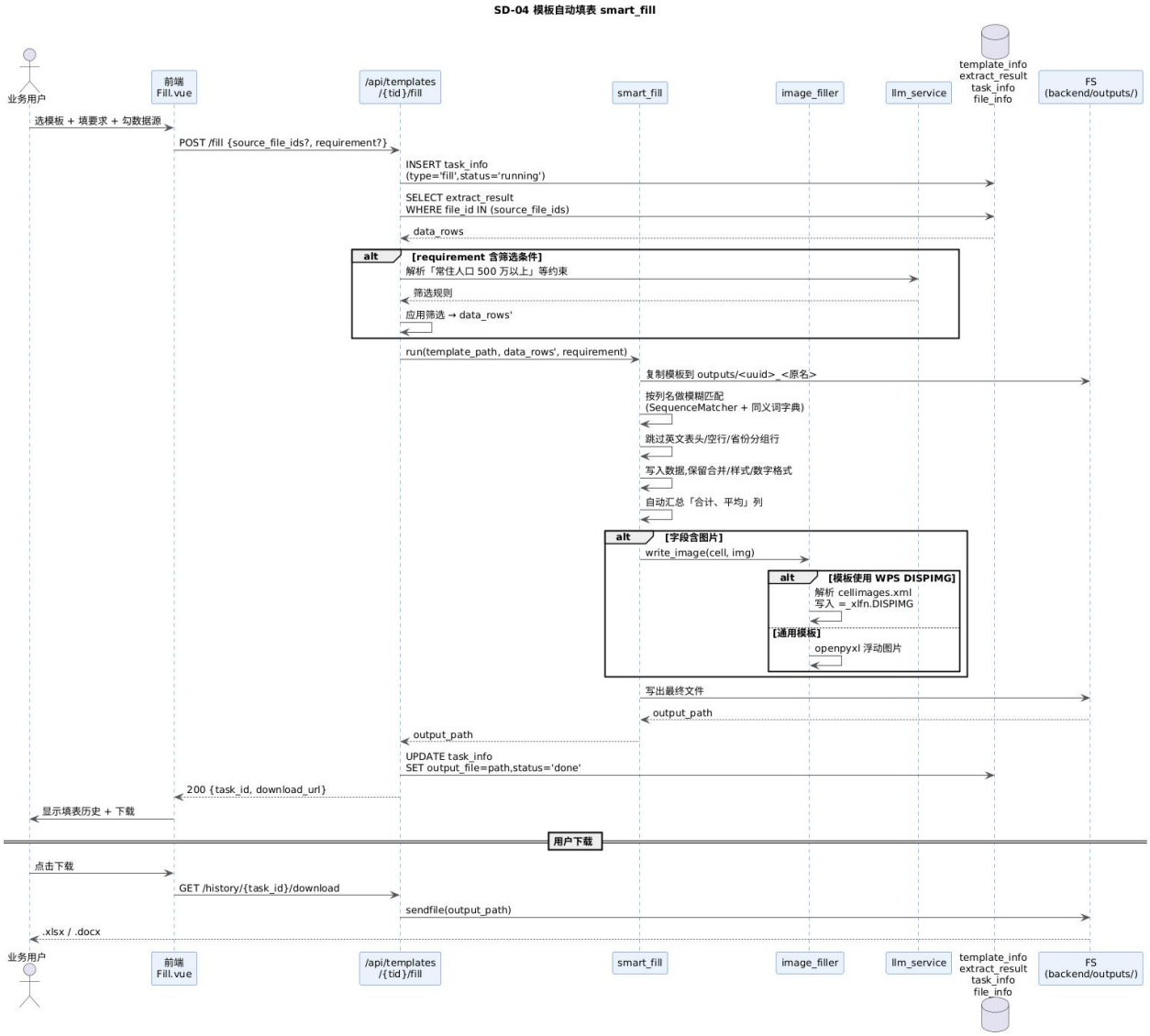


图 8-5 SD-04 模板智能填表 smart_fill 时序图

8.2.5 SD-05 智能对话 Agent 多轮工具调用

对应用例:UC-07。展示 RAG 召回 → system 提示拼装 → 多轮 tool_use 循环(最多 12 轮)→ tool_result 回灌 → final_text 输出 → chat_message 写入。包含工具异常分支与上限保护。

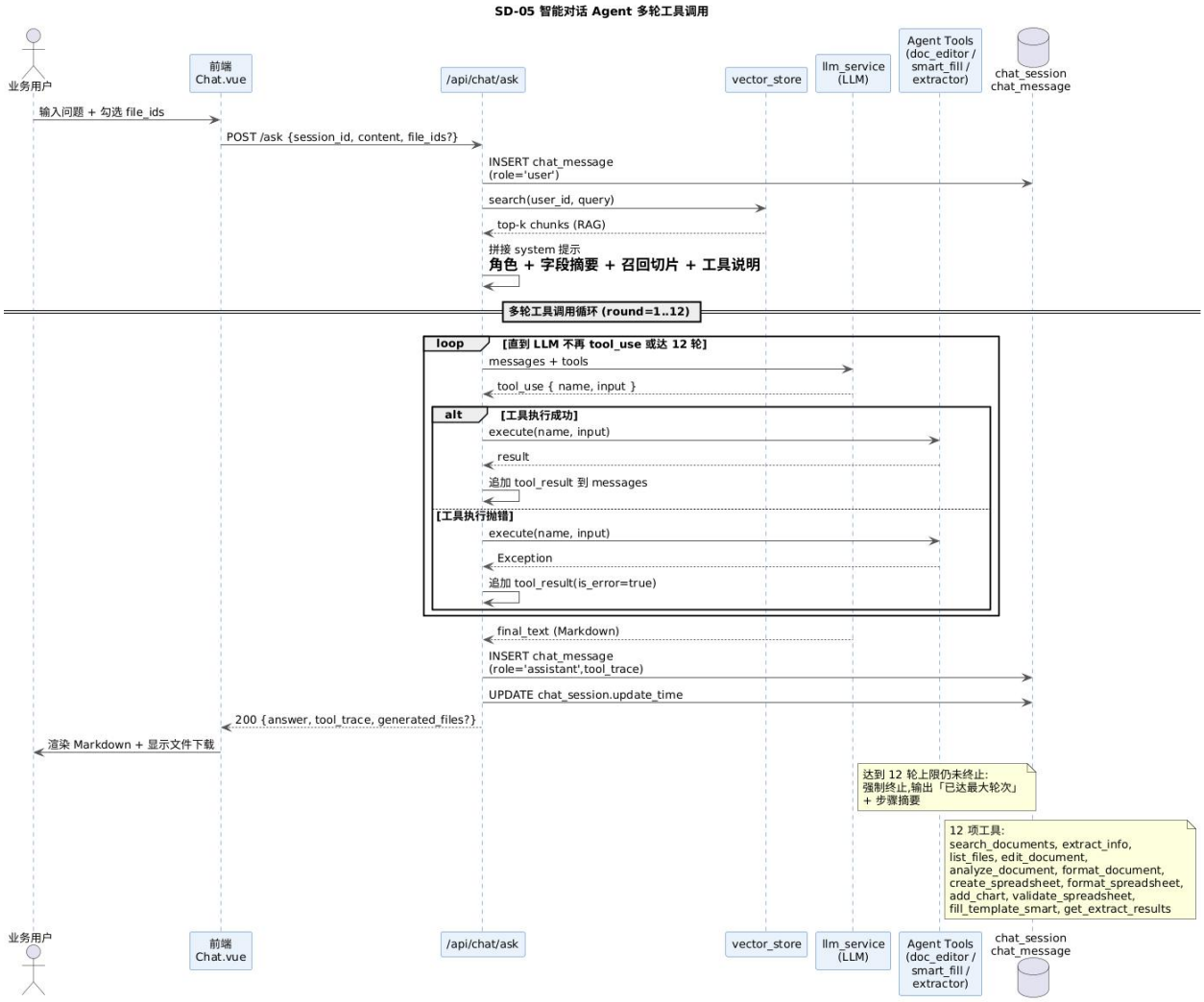


图 8-6 SD-05 智能对话 Agent 多轮工具调用时序图

8.2.6 SD-06 MCP 工具调用(JSON-RPC + SSE)

对应用例:UC-12。展示 initialize 握手(协议版本 2024-11-05)→ tools/list → tools/call → 可选 SSE 长连接(Bearer 鉴权)推送进度事件。包含三类错误的标准 JSON-RPC 返回。

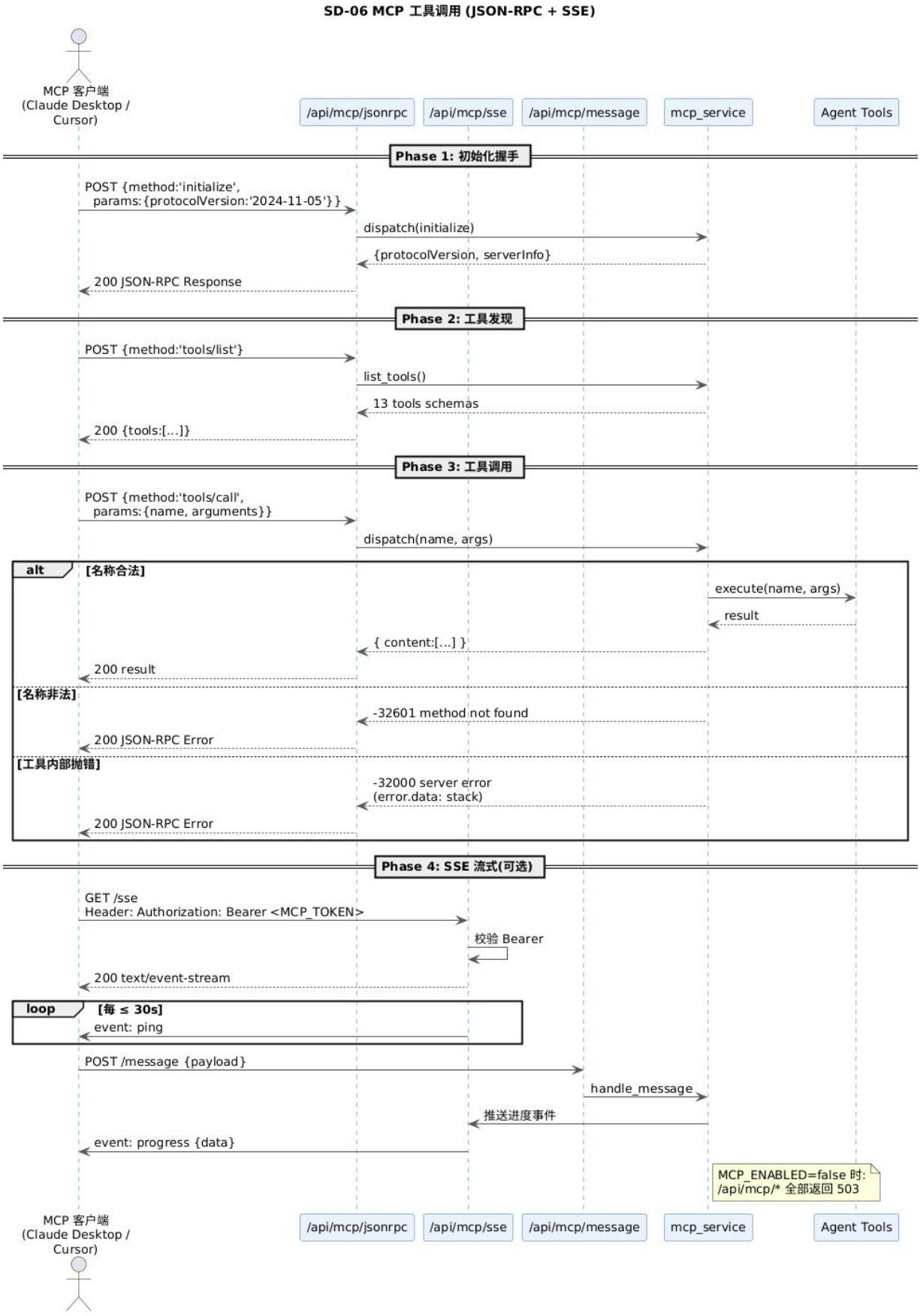


图 8-7 SD-06 MCP 工具调用时序图

8.3 状态图(State Diagrams)

对系统中三类典型状态机做正式状态图,明确状态间的迁移条件、动作与终止条件。

8.3.1 ST-01 file_info.parse_status 状态机

文件解析状态:pending → parsing → done / failed。failed 与 done 均可经「重新解析」回到 parsing,用户删除文档时进入终态。

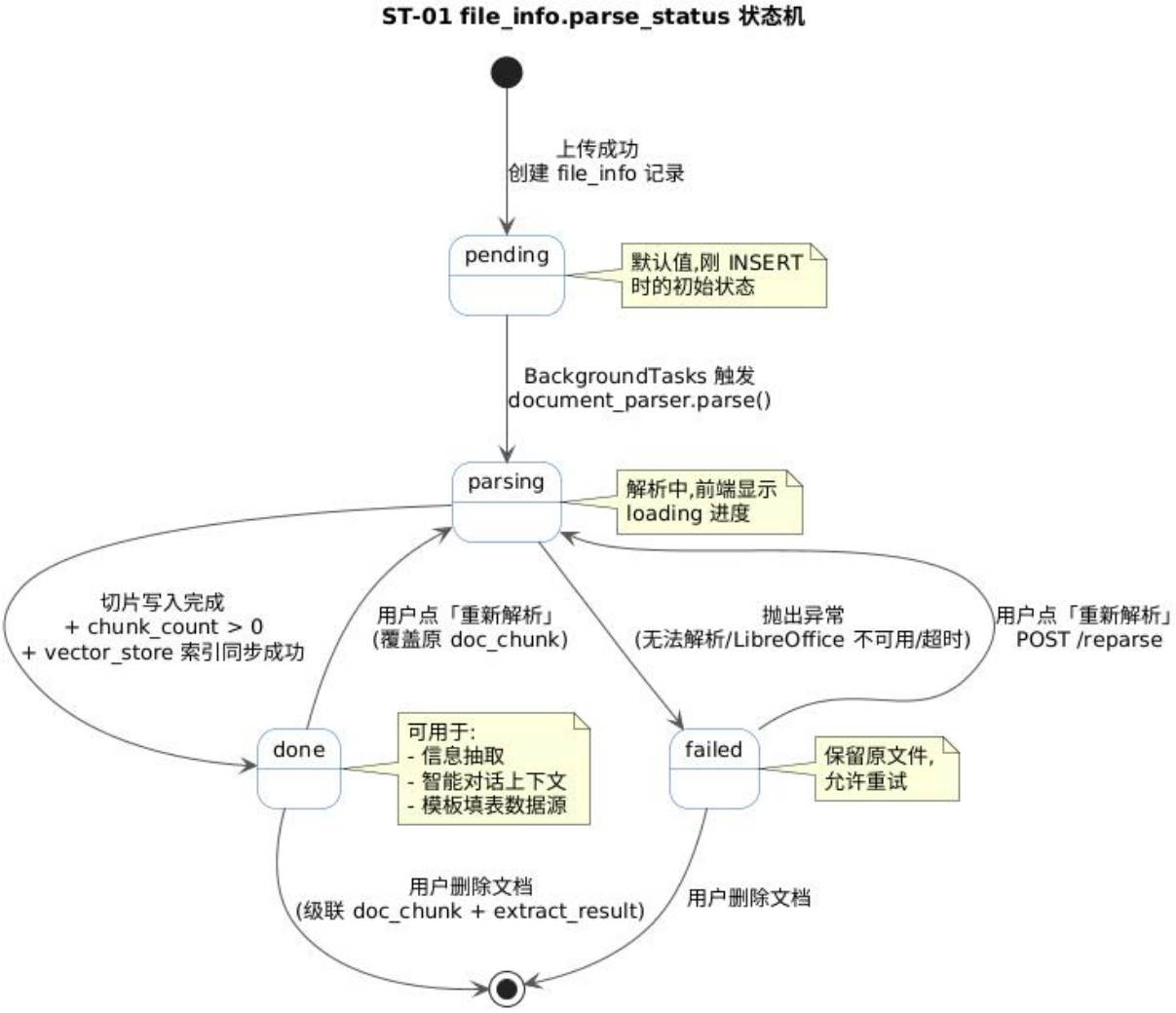


图 8-8 ST-01 file_info.parse_status 状态机

8.3.2 ST-02 task_info.status 状态机

通用任务状态:pending → running → done / failed / partial_success。partial_success 用于批量任务(如 extract-all)中部分子任务成功的场景。

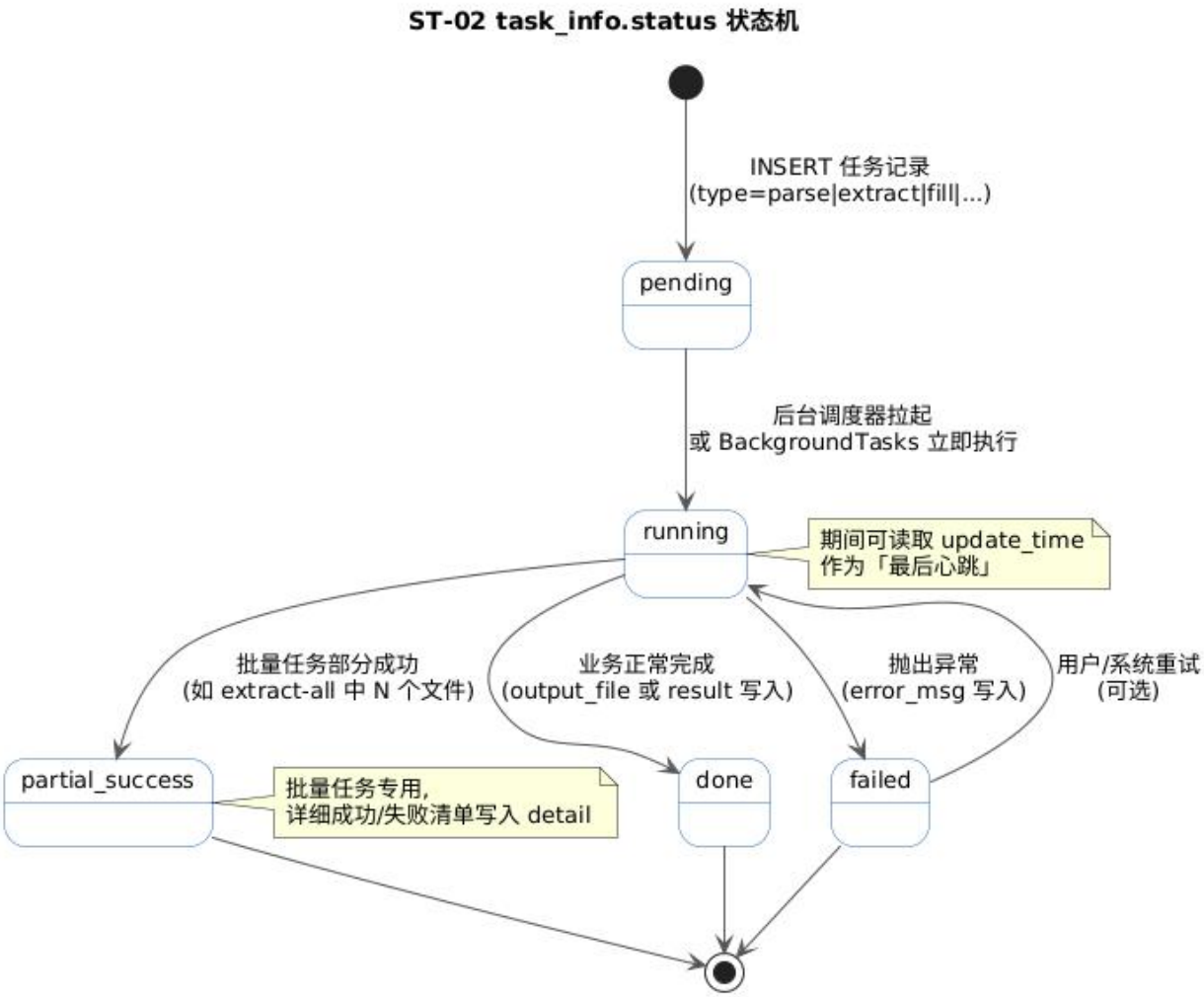


图 8-9 ST-02 task_info.status 状态机

8.3.3 ST-03 chat_session 生命周期

会话生命周期:created → active(含 idle/processing 子状态)→ archived/deleted。archived 为后续迭代规划的归档功能,当前 v1 直接物理删除。

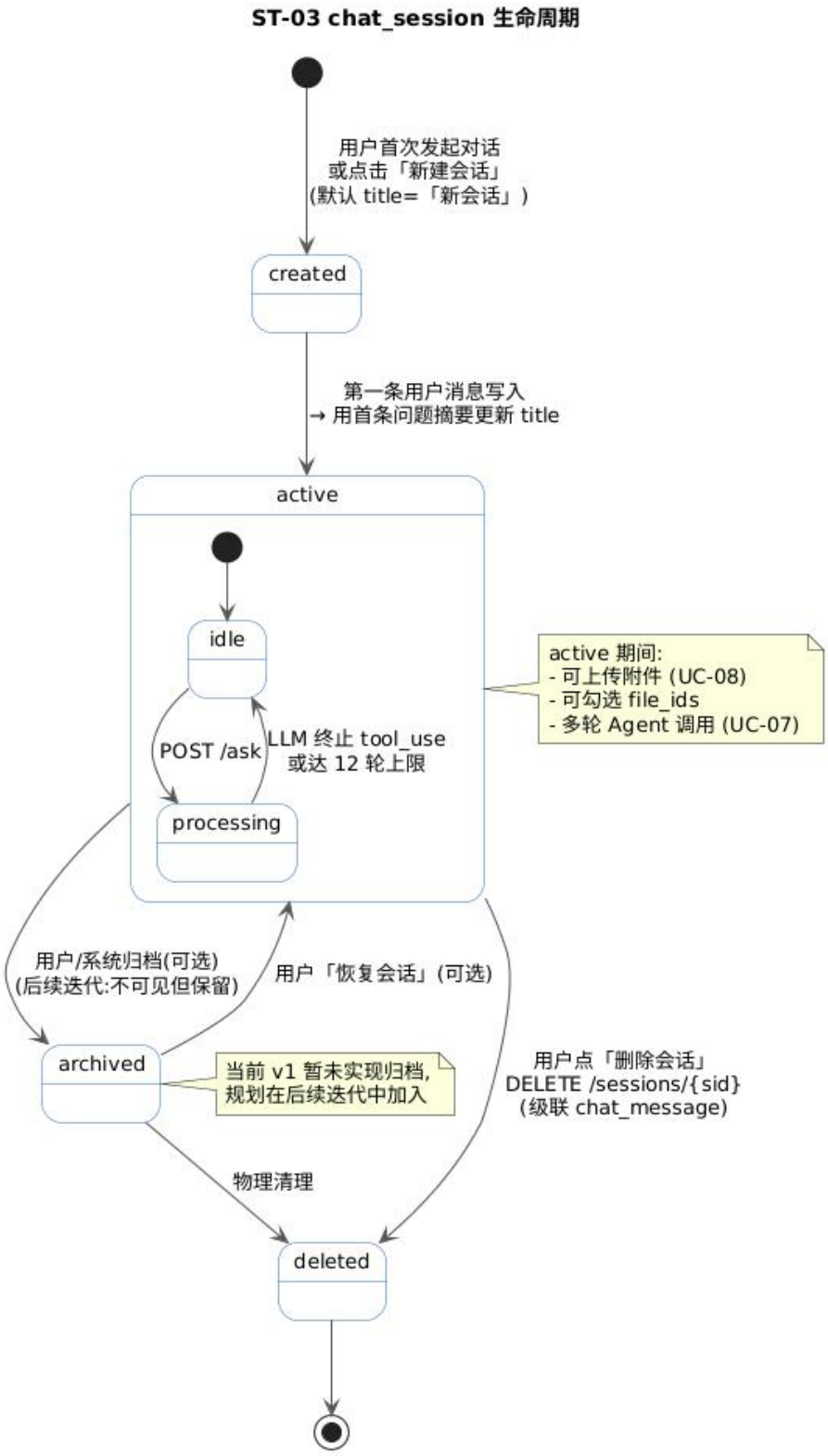


图 8-10 ST-03 chat_session 生命周期状态机

8.4 数据流图(DFD)

用 DFD 标准符号(外部实体、加工 P、数据存储 D、数据流)展示系统输入输出与

内部数据加工过程。

8.4.1 DFD-0 上下文图

第 0 层(上下文图):把 DocFusion 视为单一加工,展示 4 类外部实体(USR/ADM/MCP/API)与 3 类外部资源(LLM/嵌入模型/文件系统)与系统的所有数据流入流出。

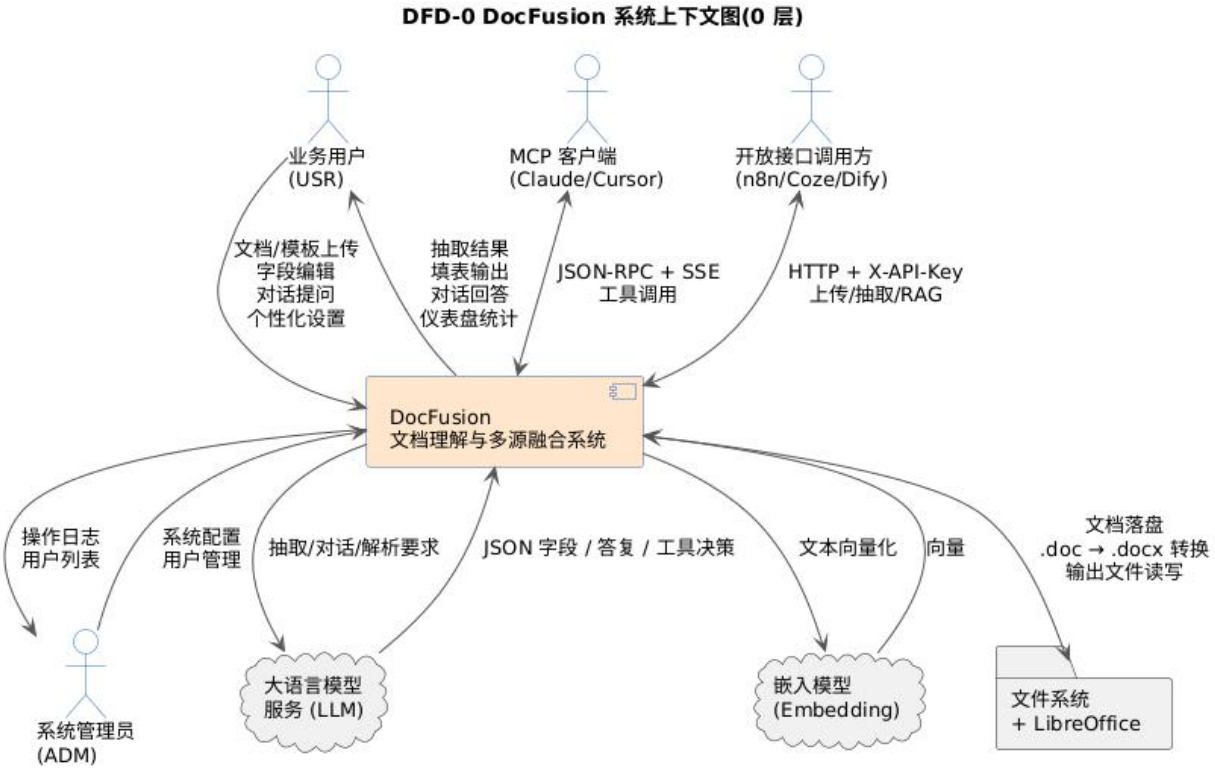


图 8-11 DFD-0 系统上下文图

8.4.2 DFD-1 1 层分解

第 1 层:把系统加工分解为 7 个子加工(认证、文档管理与解析、信息抽取、模板填表、智能对话、系统管理、对外接口),展示子加工与 8 类数据存储之间的读写关系。

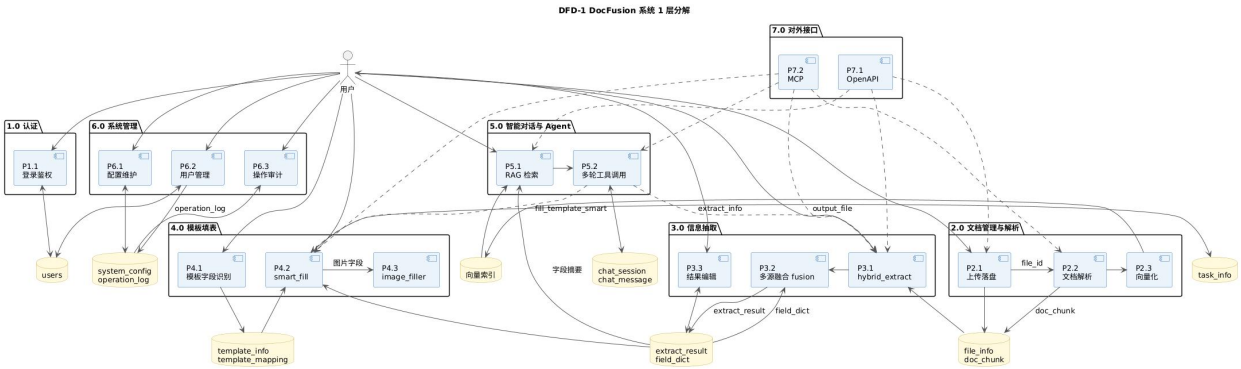


图 8-12 DFD-1 系统 1 层分解图

8.5 组件图 / 包依赖

展示 DocFusion 的物理组件结构与依赖方向,以包(Package)为容器组织前端、路由、服务、模型与外部依赖,以箭头展示「依赖于」关系,便于评估替换/演进时的影响范围。

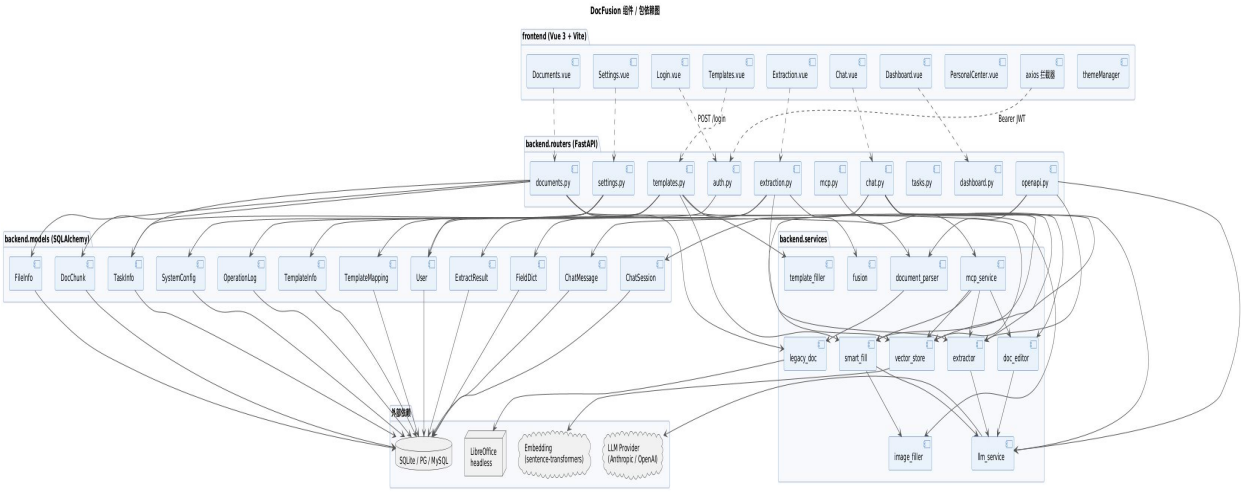


图 8-13 组件 / 包依赖图

组件图要点:

- 严格分层:frontend → routers → services → models → DB,跨层访问被禁止。
- services 内部允许互相调用(如 doc_editor、smart_fill 内部都依赖 llm_service)。
- mcp.py 与 openapi.py 复用同一套 services,仅鉴权与暴露范围不同,体现「同核异面」设计。
- 外部依赖以 cloud / node 区分:LLM 与嵌入模型为云服务;LibreOffice 为本地进程。

九、数据库设计说明

本系统默认使用 SQLite（backend/app.db），通过 SQLAlchemy ORM 维护，可平滑迁移至 PostgreSQL / MySQL。共定义 12 张业务表，全部于应用启动时由 Base.metadata.create_all 自动建表;针对历史库另有 ALTER 兼容(如 users.avatar、users.nickname、task_info.output_file 等列的自动补齐）。

9.1 概念模型与实体关系（ER）

users 1—* file_info、template_info、task_info、chat_session、operation_log;

file_info 1—* doc_chunk、extract_result（均 ON DELETE CASCADE）;

template_info 1—* template_mapping（ON DELETE CASCADE）；

chat_session 1—* chat_message（ON DELETE CASCADE）；

field_dict、system_config 为字典 / 配置表，无外键。

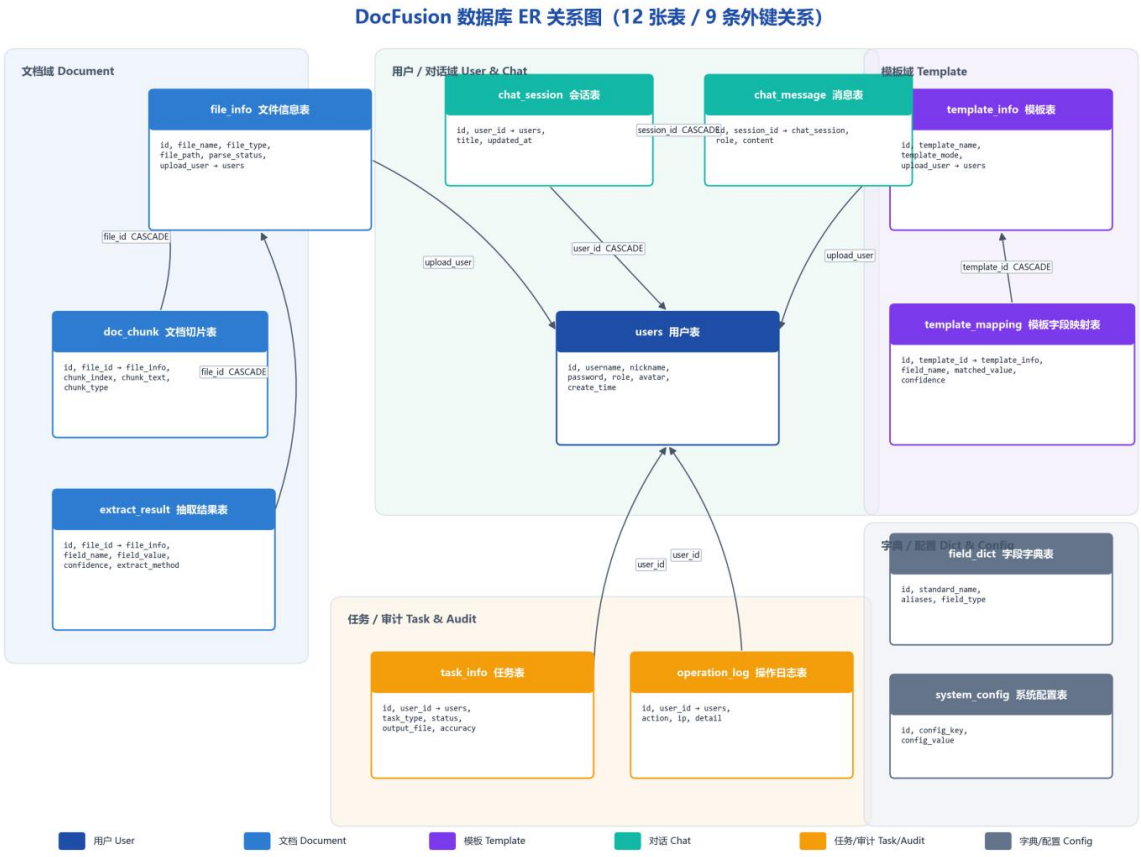


图 9-1 DocFusion 数据库 ER 关系图（12 张表 / 9 条外键关系）

9.2 数据表清单（物理模型）

表名	中文名	用途
users	用户表	存储平台用户基本信息、登录凭据、角色与头像。
file_info	文件信息表	记录所有用户上传文档的元信息与解析状态。
doc_chunk	文档切片表	保存文档解析后的文本切片，供向量检索与 RAG 使用。
extract_result	信息抽取结果表	记录从文件中抽取的结构化字段（规则 + LLM 混合抽取）。

template_info	模板表	用户上传的填表模板（xlsx / xls / docx）。
template_mapping	模板字段映射表	记录模板每个字段与抽取结果的匹配（含来源与置信度）。
task_info	任务表	异步/长流程任务记录（解析、抽取、批量提取、模板填写等）。
operation_log	操作日志表	记录关键业务操作，供管理员审计。
field_dict	字段字典表	标准字段名与别名映射，用于多源融合 fusion 服务。
system_config	系统配置表	全局可配置项（大模型 API、嵌入模型、MCP 开关等）。
chat_session	对话会话表	智能对话模块的会话上下文。
chat_message	对话消息表	会话内的逐条消息（user / assistant / tool）。

表 9-1 数据表清单（物理模型）

9.3 数据表字段定义

9.3.1 用户表 users

存储平台用户基本信息、登录凭据、角色与头像。

表 9-2 用户表 users

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
username	String(50)	登录账号，唯一，不可修改	UNIQUE NOT NULL
nickname	String(50)	昵称，可修改，默认与账号同名	默认空串
password	String(255)	bcrypt 哈希后的密码	NOT NULL
role	String(20)	角色：admin / user（内测期统一为 admin）	默认 user
avatar	String(500)	头像文件名（存于 backend/uploads/avatars/）	默认空串

create_time	DateTime	账号创建时间	默认 now()

9.3.2 文件信息表 file_info

记录所有用户上传文档的元信息与解析状态。

表 9-3 文件信息表 file_info

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
file_name	String(255)	原始文件名	NOT NULL
file_type	String(20)	文件类型：pdf / docx / xlsx / xls / jpg / png 等	NOT NULL
file_path	String(500)	文件相对/绝对路径	NOT NULL
file_size	Integer	文件大小（字节）	默认 0
upload_user	Integer	上传用户 ID	FK → users.id
parse_status	String(20)	解析状态:pending / parsing / done / failed	默认 pending
parse_time	Float	解析耗时（秒）	默认 0.0
chunk_count	Integer	解析后切片数	默认 0
field_count	Integer	已抽取字段数	默认 0
upload_time	DateTime	上传时间	默认 now()

9.3.3 文档切片表 doc_chunk

保存文档解析后的文本切片，供向量检索与 RAG 使用。

表 9-4 文档切片表 doc_chunk

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
file_id	Integer	所属文件 ID	FK → file_info.id ON DELETE CASCADE
chunk_index	Integer	切片序号	默认 0
chunk_text	Text	切片文本内容	—

chunk_type	String(50)	切片类型：paragraph / table / heading / image_caption 等	—
source_position	String(255)	来源定位（页码/段落号/表号）	—

9.3.4 信息抽取结果表 extract_result

记录从文件中抽取的结构化字段（规则 + LLM 混合抽取）。

表 9-5 信息抽取结果表 extract_result

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
file_id	Integer	来源文件 ID	FK → file_info.id ON DELETE CASCADE
field_name	String(255)	字段名（已经过 fusion 别名归一）	—
field_value	Text	字段值	—
confidence	Float	抽取置信度（0－1）	默认 0.0
extract_method	String(50)	抽取方法：rule / llm / fused	—
source_chunk_id	Integer	来源切片 ID（可空）	可空
source_position	String(255)	来源定位（可空）	可空
create_time	DateTime	记录创建时间	默认 now()

9.3.5 模板表 template_info

用户上传的填表模板（xlsx / xls / docx）。

表 9-6 模板表 template_info

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
template_name	String(255)	模板名称（原始文件名）	—
template_type	String(20)	模板类型：xlsx /	—

		xls / docx	
template_path	String(500)	模板文件路径	—
field_count	Integer	识别出的字段数	默认 0
template_mode	String(20)	模板模式：table_fill（横向表头）/ field_fill（纵向键值对）	默认 field_fill
user_requirement	Text	用户填写要求（自然语言描述）	默认空串
upload_user	Integer	上传用户 ID	FK → users.id（可空）
upload_time	DateTime	上传时间	默认 now()

9.3.6 模板字段映射表 template_mapping

记录模板每个字段与抽取结果的匹配（含来源与置信度）。

表 9-7 模板字段映射表 template_mapping

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
template_id	Integer	所属模板 ID	FK → template_info.id ON DELETE CASCADE
field_name	String(255)	模板中的字段名	—
matched_value	Text	匹配到的值	—
confidence	Float	匹配置信度	默认 0.0
source_file	String(255)	来源文件名	—
source_position	String(255)	来源文件中的定位	可空
need_confirm	Integer	是否需要人工确认（0/1）	默认 0

9.3.7 任务表 task_info

异步/长流程任务记录（解析、抽取、批量提取、模板填写等）。

表 9-8 任务表 task_info

字段名	类型	说明	约束/默认

id	Integer	主键，自增	PK
task_type	String(50)	任务类型：parse / extract / extract_all / fill_template 等	—
task_name	String(255)	任务名称（一般为相关文件名）	—
status	String(20)	状态：pending / running / done / failed	默认 pending
user_id	Integer	发起用户 ID	FK → users.id（可空）
related_id	Integer	关联实体 ID（如 file_id / template_id）	可空
result_summary	Text	结果摘要	可空
output_file	String(500)	产出文件路径（如填表导出文件）	可空
accuracy	Float	准确率（如适用）	可空
elapsed	Float	耗时（秒）	可空
start_time	DateTime	开始时间	默认 now()
end_time	DateTime	结束时间	可空

9.3.8 操作日志表 operation_log

记录关键业务操作，供管理员审计。

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
user_id	Integer	操作用户 ID（可空）	FK → users.id
username	String(50)	操作用户名（冗余存储便于检索）	—
action	String(100)	操作类型，如 login / upload / delete_doc /	—

		fill_template 等	
detail	Text	操作详情	可空
ip	String(50)	客户端 IP	可空
create_time	DateTime	操作时间	默认 now()

表 9-9 操作日志表 operation_log

9.3.9 字段字典表 field_dict

标准字段名与别名映射，用于多源融合 fusion 服务。

表 9-10 字段字典表 field_dict

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
standard_name	String(100)	标准字段名	NOT NULL
aliases	Text	别名列表（如 JSON 数组或逗号分隔）	—
field_type	String(50)	字段类型：text / number / date / address / phone 等	—
description	Text	字段说明	可空

9.3.10 系统配置表 system_config

全局可配置项（大模型 API、嵌入模型、MCP 开关等）。

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
config_key	String(100)	配置键（llm_api_base / llm_api_key / llm_model / llm_api_format / embedding_model / mcp_enabled）	UNIQUE NOT NULL
config_value	Text	配置值	—

表 9-11 系统配置表 system_config

9.3.11 对话会话表 chat_session

智能对话模块的会话上下文。

表 9-12 对话会话表 chat_session

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
user_id	Integer	所属用户	FK → users.id ON DELETE CASCADE
title	String(200)	会话标题（默认取首条问题摘要）	默认空串
create_time	DateTime	创建时间	默认 now()
updated_at	DateTime	最后更新时间（每次新消息触发）	默认 now()，onupdate

9.3.12 对话消息表 chat_message

会话内的逐条消息（user / assistant / tool）。

表 9-13 对话消息表 chat_message

字段名	类型	说明	约束/默认
id	Integer	主键，自增	PK
session_id	Integer	所属会话	FK → chat_session.id ON DELETE CASCADE
role	String(20)	消息角色：user / assistant / tool	NOT NULL
content	Text	消息内容（含工具调用 JSON 或 Markdown 答复）	—
create_time	DateTime	时间戳	默认 now()

9.4 索引设计

除主键(PK)、唯一约束(UNIQUE)与外键(FK)默认索引外,根据真实查询热路径补充以下业务索引,确保高频列表/分页/统计查询的稳定性能。

表名	索引名	列	类型
file_info	idx_file_user_time	(upload_user,	复合

		upload_time DESC)	
file_info	idx_file_status	(parse_status)	单列
doc_chunk	idx_chunk_file_idx	(file_id, chunk_index)	复合
extract_result	idx_extract_file_field	(file_id, field_name)	复合
extract_result	idx_extract_field	(field_name)	单列
template_info	idx_tpl_user_time	(upload_user, create_time DESC)	复合
template_mapping	idx_tplmap_tpl	(template_id)	单列
task_info	idx_task_user_status_time	(user_id, status, create_time DESC)	复合
task_info	idx_task_type_status	(type, status)	复合
chat_session	idx_session_user_time	(user_id, update_time DESC)	复合
chat_message	idx_msg_session_time	(session_id, create_time)	复合
operation_log	idx_log_user_time	(user_id, create_time DESC)	复合
operation_log	idx_log_action_time	(action, create_time DESC)	复合
field_dict	idx_dict_std	(standard_name)	单列（已 UNIQUE）
system_config	idx_cfg_key	(config_key)	单列（已 UNIQUE）

9.5 枚举值与级联策略

(一)枚举值集中清单:本项目大多数状态/类型字段使用字符串枚举,集中维护如下,

与代码 constants.py 保持一致。

表名	字段名	枚举值
users	role	admin、user
file_info	file_type	pdf、docx、xlsx、xls、doc、jpg、jpeg、png、bmp、tiff
file_info	parse_status	pending、parsing、done、failed
extract_result	source	rule、llm、fusion、manual
extract_result	extract_method	regex、llm、hybrid、manual
template_info	template_mode	table_fill、field_fill、hybrid
task_info	type	parse、extract、extract_all、fill、upload、other
task_info	status	pending、running、done、failed、partial_success、canceled
chat_message	role	user、assistant、tool、system
operation_log	action	login、upload、extract、fill、delete、update_user、reset_password、update_config、…(可扩展)
system_config	config_key	LLM_BASE_URL、LLM_API_KEY、LLM_MODEL、LLM_FORMAT、EMBEDDING_MODEL、MCP_ENABLED、MAX_UPLOAD_SIZE、MAX_TOOL_ROUNDS 等

(二)级联策略矩阵:

父表 → 子表	操作	策略	说明
users → file_info	DELETE	RESTRICT	删用户前需先清理其文件，避免误删

users → template_info	DELETE	RESTRICT	同上
users → task_info	DELETE	SET NULL	保留任务历史用于审计
users → chat_session	DELETE	CASCADE	用户删除时同步清理会话与消息
users → operation_log	DELETE	SET NULL	日志保留，user_id 置空
file_info → doc_chunk	DELETE	CASCADE	文件删除时切片必须删除
file_info → extract_result	DELETE	CASCADE	同上
template_info → template_mapping	DELETE	CASCADE	模板删除时映射必须删除
chat_session → chat_message	DELETE	CASCADE	会话删除时消息必须删除

注:当前实现以 SQLAlchemy 关系级 cascade 为主;迁移至 PostgreSQL 后建议补充 DDL 级别 ON DELETE 约束。

9.6 容量与性能设计

(一)数据量估算(以 500 用户、单用户日均 20 份文档、单文档平均 30 切片为基线,1 年累计):

表名	单条估算大小	1 年增量估算	备注
users	1 KB	500 行	内测期不大幅增长
file_info	1 KB	365 万行	主要增长来自上传
doc_chunk	4 KB（含 content）	1 亿行	占比最大，首批迁移至 PG 时优先
extract_result	0.5 KB	1.8 亿行	单文档平均 50 字段 × 365 万
template_info	1 KB	5 万行	模板复用率高
template_mapping	0.3 KB	50 万行	单模板平均 10 字段
task_info	0.5 KB	1000 万行	含解析 / 抽取 /

			填表任务
operation_log	0.5 KB	1000 万行	建议 6 个月归档一次
chat_session	0.5 KB	50 万行	
chat_message	2 KB	5000 万行	会话内 100 条均值

(二)性能设计要点:

1) 切片表(doc_chunk)与抽取表(extract_result)是查询热点,均加复合索引(file_id 前置)。

2) 长文本字段(content / field_value / tool_trace)在 SQLite 阶段使用 TEXT;迁移至 PG 后使用 TEXT(自动 toast)或 JSONB。

3) 仪表盘统计用「物化视图 + 定时刷新」(后续迭代,当前每次现算)。

4) 操作日志按月分区(PG 原生分区)以便归档与清理。

5) 单查询返回行数硬上限 10000(分页强制),避免接口拖慢。

9.7 大字段存储与向量索引协同

(一)大字段:

doc_chunk.content (TEXT,平均 1500 字符,峰值 8000):用作 RAG 检索的原文回填,需高频随机访问。

extract_result.field_value (TEXT,长度方差大,极少超过 2KB):需结合 confidence 排序。

chat_message.content / tool_trace (TEXT,可能数 KB):仅会话内顺序访问,无需索引。

(二)向量索引协同方案(当前实现):

1) embedding 由 sentence-transformers 在线生成,结果保存为 NumPy 矩阵(.npy),按用户分文件存放 backend/vector_store/<user_id>/。

2) 切片元信息(file_id, chunk_id)与向量行号一一对应,通过侧文件 mapping.json 维护。

3) 检索时:np.matmul(query_vec, all_vecs.T) 计算余弦相似度,argpartition 取 top-k。

4) sentence-transformers 不可用时降级:jieba 分词 + 词重叠启发式打分。

(三)演进方向:

1) 数据量超过 100 万切片后,迁移至 pgvector(PostgreSQL 扩展)或专用向量库

(Chroma / Milvus / Qdrant)。

2) 引入 IVF/HNSW 索引以支持 P95 < 50ms 的检索。

3) 增量更新:文件解析完成时单独 append 向量,而非全量重算。

9.8 数据安全与脱敏

(一)敏感字段清单与现状:

字段	敏感等级	当前存储	改进建议
users.password	高	bcrypt 哈希(已加密)	满足要求，定期轮换 bcrypt rounds
system_config (LLM_API_KEY 等)	高	明文	后续以 Fernet/AES-GCM 对称加密，密钥来自环境变量
system_config(WEBHOOK_KEY / SECRET_KEY)	高	明文	同上
chat_message.content	中	明文	用户内容，需在导出 / 审计时按需脱敏 (身份证 / 手机号正则替换)
extract_result.field_value	中	明文	同上，可按 field_name 标签决定是否脱敏
operation_log.detail	中	明文	仅管理员可见，按需打码
file_info.file_path	低	明文相对路径	接口侧不外露绝对路径

(二)传输与访问安全:

1) 全站 HTTPS(生产);后端 CORS 限定到前端域名。

2) JWT HS256 签名,SECRET_KEY 通过环境变量注入,严禁入版本库。

3) 用户数据强隔离:vector_store 按 user_id 分文件;file_info / extract_result / chat_session 等查询强制注入 user_id 条件。

4) X-API-Key 与 MCP_TOKEN 通过环境变量配置,支持多 Key 轮换(后续迭代)。

(三)合规与审计:

1) 所有写操作(用户管理、配置变更、删除)均写 operation_log。

2) 个人信息处理符合《个人信息保护法》最小必要原则;新增数据出境场景需法律评估。

9.9 备份与恢复策略

(一)备份对象:

1) 主数据库:SQLite 文件 backend/app.db;迁移后为 PG/MySQL 实例。

2) 上传目录:backend/uploads/、backend/templates/、backend/outputs/。

3) 向量索引:backend/vector_store/。

4) 系统配置:backend/.env(密钥)与 system_config 表。

(二)备份策略(建议生产配置):

数据	频率	方式	保留
数据库	每日 02:00	全量 dump (sqlite3 .backup/pg_dump)	30 天
数据库	每周日 03:00	全量 + 压缩归档	12 周
数据库	每月 1 日 04:00	全量 + 离线介质	12 个月
uploads/	每日 02:30	rsync 增量	30 天
vector_store/	与数据库同步	rsync 增量	30 天 (可重新生成,优先级低)

(三)恢复流程(RTO ≤ 30min,RPO ≤ 24h):

1) 停服:systemctl stop docfusion-backend。

2) 数据库恢复:cp app.db.bak.YYYYMMDD app.db (或 pg_restore)。

3) 文件目录恢复:rsync -av backup/uploads/ backend/uploads/。

4) 启动服务:systemctl start docfusion-backend。

5) 校验:GET /api/health & 抽样 GET /api/documents/list & 解析任务自检。

(四)恢复演练:每季度模拟一次完整恢复,验证备份可用性与 RTO。

9.10 数据库版本管理与迁移

(一)当前实现:

1) 启动时由 Base.metadata.create_all 自动建表(只增不改)。

2) 历史库新增列(如 users.avatar、users.nickname、task_info.output_file)

通过应用启动时的 ALTER TABLE 兼容补齐。

3) 数据修复以一次性脚本(scripts/migrate_*.py)形式提供。

(二)推荐演进:引入 Alembic 版本化迁移。

1) 安装 alembic 并 alembic init migrations。

2) 用 autogenerate 生成版本脚本:alembic revision --autogenerate -m 'msg'。

3) 手工审核生成的 op.add_column / op.drop_column 等指令,确保数据安全。

4) 部署时运行 alembic upgrade head 替代 create_all。

5) 版本号写入 alembic_version 表,代码与 DB schema 双向追溯。

(三)SQLite → PostgreSQL / MySQL 迁移点:

差异点	SQLite 行为	PostgreSQL	MySQL	处理建议
自增主键	INTEGER PRIMARY KEY 自动 ROWID	SERIAL/IDENTITY	AUTO_INCREMENT	SQLAlchemy 的 Integer + autoincrement 即可
DateTime	TEXT 存储	TIMESTAMP	DATETIME	注意时区：统一以 UTC 存储，展示时按用户时区转换
Float	REAL	DOUBLE PRECISION	DOUBLE	无差异，但金额字段建议改 DECIMAL
UNIQUE(NULL)	多个 NULL 视为不同（可重复）	同 SQLite	同 SQLite	业务唯一字段不应允许 NULL
级联约束	默认未启用外键	完整支持	完整支持（InnoDB）	已通过 SQLAlchemy ORM cascade 处理，迁移后开启 DDL 级 FK
FULL TEXT	不支持（需 FTS5）	tsvector + GIN	FULLTEXT 索引	RAG 仍优先走向量；关键词检索用对应方言的 FTS
正则函数	受限	regexp_match / SIMILAR TO	REGEXP_LIKE	抽取规则统一在应用层实现，

				降低 DB 依赖

(四)迁移流程:

1) 用 alembic 在新库一次性创建 schema。

2) 用 sqlite3 .dump 导出 INSERT,sed/python 脚本调整方言后批量灌入新库。

3) 应用切换 DATABASE_URL 环境变量,验证 health + 主流程烟测。

4) 灰度并行运行 24h,确认无问题后下线 SQLite。

9.11 审计列与软删除规范

(一)当前已有审计列:

- users.create_time、file_info.upload_time、template_info.create_time、task_info.create_time / update_time、chat_session.create_time / update_time、chat_message.create_time、operation_log.create_time、system_config.update_time。

(二)后续迭代扩充:对所有业务表统一审计列

created_at(DateTime,默认 now,不可改)

updated_at(DateTime,默认 now,onupdate=now)

created_by(Integer,FK→users.id,记录创建者)

updated_by(Integer,FK→users.id,记录最后更新者)

deleted_at(DateTime,可空,软删除标记;非 NULL 即视为已删)

deleted_by(Integer,FK→users.id,记录删除者)

(三)软删除策略:

1) DELETE 操作改为 UPDATE deleted_at = now(),保留数据 90 天后由定时任务物理清理。

2) 所有 SELECT 加 WHERE deleted_at IS NULL(可由 SQLAlchemy 全局查询过滤器实现)。

3) 用户/管理员可见的删除恢复入口仅在「回收站」(后续迭代)中提供。

4) 关联强约束的子表(如 doc_chunk)仍走级联物理删,避免孤儿数据。

(四)审计粒度:

- 所有写操作(C/U/D)对应 operation_log 一条记录,字段含 user_id / action / target_table / target_id / detail(JSON)。

- 高敏感操作(配置变更、用户角色变更、删除)额外记录变更前后 diff(JSON Patch)。

9.12 数据字典与字段归一

(一)目的:为「多源融合 fusion」服务提供标准字段名与别名映射,确保「联系电话/电话/手机号/contact」等同义异形字段被归一为同一标准名。

(二)field_dict 表使用约定:

standard_name | 标准名(系统内全局唯一,如 phone)

aliases | 别名集合(JSON 数组或竖线分隔字符串)

field_type | 字段类型 (string|int|float|date|email|phone|id_card|amount|...)

description | 业务描述

(三)归一流程:

1) 抽取阶段:extractor 输出原始字段名(可能不规范)。

2) fusion 服务调用 normalize(field_name) → 查 field_dict.aliases → 返回 standard_name。

3) 多份文档同名字段汇聚后按 source 加权 + 多值投票,取 final_value。

4) 写入 extract_result 的 field_name 始终为 standard_name,保证下游模板填表与对话引用一致。

(四)初始字典示例(节选,实际由 scripts/init_field_dict.py 灌入):

standard_name='phone', aliases=['联系电话','电话','手机号','contact','tel'], type='phone'

standard_name='id_card', aliases=['身份证','身份证号','证件号'], type='id_card'

standard_name='amount', aliases=['金额','合计金额','总额','total'], type='amount'

standard_name='date', aliases=['日期','签订日期','签发日期','date'], type='date'

(五)管理界面:后续迭代提供 /api/settings/field-dict 增删改查,支持业务方自助维护。

9.13 ER 关系完整说明

汇总 12 张表的所有外键关系(共 9 条),便于快速核对。

外键	引用	级联	说明
file_info.upload_user →	users.id	RESTRICT	文件归属

users.id			
doc_chunk.file_id → file_info.id	file_info.id	CASCADE	切片归属
extract_result.file_id → file_info.id	file_info.id	CASCADE	抽取结果归属
template_info.upload_user → users.id	users.id	RESTRICT	模板归属
template_mapping.template_id → template_info.id	template_info.id	CASCADE	模板字段归属
task_info.user_id → users.id	users.id	SET NULL	任务归属
chat_session.user_id → users.id	users.id	CASCADE	会话归属
chat_message.session_id → chat_session.id	chat_session.id	CASCADE	消息归属
operation_log.user_id → users.id	users.id	SET NULL	日志归属

注:field_dict、system_config 为字典/配置表,无外键。

十、API 接口规格说明

本系统后端共暴露 61 个 RESTful 接口，分布在 10 个模块。除明确标注「公开」「X-API-Key」「Bearer Token」者外，均要求请求头 Authorization: Bearer <JWT>。所有接口默认返回 application/json，下载类接口返回二进制流。

10.1 模块概览

表 10-1 模块概览

模块	前缀	中文名	端点数
auth	/api/auth	认证与用户	7
documents	/api/documents	文档管理	9
extraction	/api/extraction	信息抽取结果管理	6
templates	/api/templates	模板与填表	10
chat	/api/chat	智能对话与 Agent	7
dashboard	/api/dashboard	仪表盘统计	1

settings	/api/settings	系统设置	6
tasks	/api/tasks	任务中心	2
openapi	/api/open	对外开放接口（X-API-Key）	9
mcp	/api/mcp	MCP 协议端点	4

10.2 认证与用户（/api/auth）

用户登录、注册、信息维护、头像与密码管理。基于 JWT(HS256)，登录后客户端在 sessionStorage 中持有 access_token。

1.接口名称：register

该接口用于「用户注册」。注册时自动以账号作为默认昵称；返回的 token 可立即用于受保护接口。

表 10-2 接口 register 表

字段	描述
请求方式	POST
URL	/api/auth/register
请求头	Content-Type: application/json
请求参数	Body: username: str （必填）账号，3－50 字 password: str （必填）密码

响应格式	{
	access_token: str,
	token_type: 'bearer',
	user_id: int
	}

错误处理	400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	公开
备注	注册时自动以账号作为默认昵称;返回的 token 可立即用于受保护接口。

2.接口名称：login

该接口用于「用户登录」。采用 OAuth2 表单格式（application/x-www-form-urlencoded），便于 Swagger 自动联调。

表 10-3 接口 login 表

字段	描述
请求方式	POST
URL	/api/auth/login
请求头	Content-Type: application/x-www-form-urlencoded
请求参数	Form (OAuth2PasswordRequestForm): username: str （必填） password: str （必填）

响应格式	{
	access_token: str,
	token_type: 'bearer'
	}

错误处理	400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	公开
备注	采用 OAuth2 表单格式(application/x-www-form-urlencoded)，便于 Swagger 自动联调。

3.接口名称：me

该接口用于「获取当前用户」。前端在初始化阶段调用，用于刷新顶栏头像/昵称与 sessionStorage.

表 10-4 接口 me 表

字段	描述
请求方式	GET
URL	/api/auth/me
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	{

	id: int, username: str, nickname: str, role: str, avatar: str }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	前端在初始化阶段调用，用于刷新顶栏头像/昵称与 sessionStorage.

4.接口名称：upload_avatar

该接口用于「上传头像」。落盘到 backend/uploads/avatars/，文件名以用户 ID 命名；上传成功后前端 dispatch 'avatar-updated' 事件。

表 10-5 接口 upload_avatar 表

字段	描述
请求方式	POST
URL	/api/auth/avatar
请求头	Authorization: Bearer <JWT> Content-Type: multipart/form-data
请求参数	Form-data: file: UploadFile （必填）图片文件（jpg/png 等）
响应格式	{ avatar: '<filename>' }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	落盘到 backend/uploads/avatars/，文件名以用户 ID 命名；上传成功后前端 dispatch 'avatar-updated' 事件。

5.接口名称：get_avatar

该接口用于「读取头像」。供 <img src> 直接引用，不要求鉴权以便缓存。

表 10-6 接口 get_avatar 表

字段	描述
请求方式	GET
URL	/api/auth/avatars/{filename}
请求头	（无）
请求参数	Path: filename: str　头像文件名
响应格式	image/* 二进制流
错误处理	500 Internal Server Error: 服务端异常（详见日志）
访问权限	公开
备注	供 <img src> 直接引用，不要求鉴权以便缓存。

6.接口名称：change_password

该接口用于「修改密码」。服务端先用 bcrypt.checkpw 校验旧密码，再写入新密码哈希。

表 10-7 接口 change_password 表

字段	描述
请求方式	POST
URL	/api/auth/change-password
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body: old_password: str　(必填) new_password: str　(必填)
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）

访问权限	需登录
备注	服务端先用 bcrypt.checkpw 校验旧密码，再写入新密码哈希。

7.接口名称：change_nickname

该接口用于「修改昵称」。成功后前端 dispatch 'nickname-updated' 事件刷新顶栏。

表 10-8 接口 change_nickname 表

字段	描述
请求方式	POST
URL	/api/auth/change-nickname
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body: nickname: str （必填）新昵称
响应格式	{ nickname: str }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	成功后前端 dispatch 'nickname-updated' 事件刷新顶栏。

10.3 文档管理（/api/documents）

文档上传、列表、详情、删除、批量删除、重新解析、单文件 LLM 抽取、批量 LLM 抽取、原文件下载。

1.接口名称：upload

该接口用于「上传文档」。落盘到 backend/uploads/，写 file_info 后通过 BackgroundTasks 异步触发解析。

表 10-9 接口 upload 表

字段	描述
请求方式	POST
URL	/api/documents/upload

请求头	Authorization: Bearer <JWT> Content-Type: multipart/form-data
请求参数	Form-data: files: List[UploadFile] （必填）支持多文件并发上传
响应格式	{ uploaded: [{ id: int, file_name: str, file_type: str, parse_status: 'pending' }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	落盘到 backend/uploads/，写 file_info 后通过 BackgroundTasks 异步触发解析。

2.接口名称：list_docs

该接口用于「文档列表」。默认按 upload_time 倒序；支持按 file_type 精确过滤。

表 10-10 接口 list_docs 表

字段	描述
请求方式	GET
URL	/api/documents/list
请求头	Authorization: Bearer <JWT>
请求参数	Query: page: int = 1 page_size: int = 20 file_type: str （可选）按类型过滤
响应格式	{ items: [...], total: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	默认按 upload_time 倒序；支持按 file_type 精确过滤。

3.接口名称：detail

该接口用于「文档详情」。返回 file_info 主体 + 解析切片摘要 + 已抽取字段摘要。

表 10-11 接口 detail 表

字段	描述
请求方式	GET
URL	/api/documents/{fid}
请求头	Authorization: Bearer <JWT>
请求参数	Path: fid: int 文档 ID
响应格式	{ file_info: {...}, chunks_summary: {...}, fields_summary: {...} }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	返回 file_info 主体 + 解析切片摘要 + 已抽取字段摘要。

4.接口名称：delete

该接口用于「删除文档」。级联删除 doc_chunk、extract_result（数据库 ON DELETE CASCADE）；同时清理本地文件与向量索引片段。

表 10-12 接口 delete 表

字段	描述
请求方式	DELETE
URL	/api/documents/{fid}
请求头	Authorization: Bearer <JWT>
请求参数	Path: fid: int 文档 ID
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录（仅本人/管理员）

备注	级联删除 doc_chunk、extract_result（数据库 ON DELETE CASCADE）；同时清理本地文件与向量索引片段。

5.接口名称：batch_delete

该接口用于「批量删除文档」。服务端会过滤掉非当前用户拥有的 ID，仅删除有权操作的记录。

表 10-13 接口 batch_delete 表

字段	描述
请求方式	POST
URL	/api/documents/batch-delete
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body: ids: List[int] （必填）待删除的文件 ID 列表
响应格式	{ deleted: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	服务端会过滤掉非当前用户拥有的 ID，仅删除有权操作的记录。

6.接口名称：reparse

该接口用于「重新解析文档」。BackgroundTasks 异步执行；状态可在 /api/tasks/list 跟踪。

表 10-14 接口 reparse 表

字段	描述
请求方式	POST
URL	/api/documents/{fid}/reparse
请求头	Authorization: Bearer <JWT>
请求参数	Path: fid: int

响应格式	{ task_id: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	BackgroundTasks 异步执行；状态可在 /api/tasks/list 跟踪。

7.接口名称：extract

该接口用于「单文件 LLM 抽取」。走 extractor.hybrid_extract（规则 +LLM）；结果写入 extract_result 表。

表 10-15 接口 extract 表

字段	描述
请求方式	POST
URL	/api/documents/{fid}/extract
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Path: fid: int Body（可选）: fields: List[str] （可选）指定要抽取的字段；为空则自动抽取关键信息
响应格式	{ extracted: [{ field_name, field_value, confidence, extract_method }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	走 extractor.hybrid_extract（规则 + LLM）；结果写入 extract_result 表。

8.接口名称：extract_all

该接口用于「批量抽取已解析」。对当前用户所有 parse_status='done' 的文档

批量提交抽取任务。

表 10-16 接口 extract_all 表

字段	描述
请求方式	POST
URL	/api/documents/{fid}/extract-all
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	{ tasks: [{ task_id, file_id }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	对当前用户所有 parse_status='done' 的文档批量提交抽取任务。

9.接口名称：download

该接口用于「下载原始文件」。用于在前端「下载原文档」按钮，或导出审阅。

表 10-17 接口 download 表

字段	描述
请求方式	GET
URL	/api/documents/{fid}/download
请求头	Authorization: Bearer <JWT>
请求参数	Path: fid: int
响应格式	二进制文件流（Content-Disposition 携带原始文件名）
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于在前端「下载原文档」按钮，或导出审阅。

10.4 信息抽取结果管理（/api/extraction）

对 extract_result 表的查询、编辑、批量删除，以及多源融合视图。

1.接口名称：get_results

该接口用于「抽取结果分页查询」。支持按 file_id 与 field_name 模糊过滤。

表 10-18 接口 get_results 表

字段	描述
请求方式	GET
URL	/api/extraction/results
请求头	Authorization: Bearer <JWT>
请求参数	Query: file_id: int （可选） field_name: str （可选） page: int = 1 page_size: int = 20
响应格式	{ items: [{ id, file_id, field_name, field_value, confidence, extract_method, source_position }], total: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	支持按 file_id 与 field_name 模糊过滤。

2.接口名称：get_fused

该接口用于「多源融合数据」。调用 fusion 服务：字段别名归一 + 来源加权 + 多值投票，输出最终值与候选清单。

表 10-19 接口 get_fused 表

字段	描述
请求方式	GET
URL	/api/extraction/fused
请求头	Authorization: Bearer <JWT>
请求参数	（无）

响应格式	{fields: [{field_name, value, candidates: [...], confidence, sources: [...] }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	调用 fusion 服务：字段别名归一 + 来源加权 + 多值投票，输出最终值与候选清单。

3.接口名称：get_files_with_fields

该接口用于「已抽取的文件列表」。用于前端「按文件查看抽取」选择器。

表 10-20 接口 get_files_with_fields 表

字段	描述
请求方式	GET
URL	/api/extraction/files
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	[{ file_id, file_name, file_type, field_count }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于前端「按文件查看抽取」选择器。

4.接口名称：get_file_fields

该接口用于「按文件查看字段」。返回单个文件的全部抽取字段。

表 10-21 接口 get_file_fields 表

字段	描述
请求方式	GET
URL	/api/extraction/files/{fid}/fields
请求头	Authorization: Bearer <JWT>
请求参数	Path: fid: int

响应格式	[{ id, field_name, field_value, confidence, extract_method, source_position }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	返回单个文件的全部抽取字段。

5.接口名称：update

该接口用于「编辑字段值」。用于人工修正自动抽取结果；建议在 source 字段标注 'manual'。

表 10-22 接口 update 表

字段	描述
请求方式	PUT
URL	/api/extraction/results/{rid}
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Path: rid: int Body: field_value: str （必填）新字段值
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于人工修正自动抽取结果；建议在 source 字段标注 'manual'。

6.接口名称：batch_delete_results

该接口用于「批量删除字段」。服务端按用户拥有的文件 ID 校验，避免越权删除。

表 10-23 接口 batch_delete_results 表

字段	描述

请求方式	POST
URL	/api/extraction/results/batch-delete
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body: ids: List[int] （必填）
响应格式	{ deleted: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	服务端按用户拥有的文件 ID 校验，避免越权删除。

10.5 模板与填表（/api/templates）

模板上传、字段识别、智能填写、历史记录、按任务下载导出文件。

1.接口名称：upload

该接口用于「上传模板」。上传后立即调用 template_filler.identify_fields 识别字段；旧版 .xls 走 xlrd。

字段	描述
请求方式	POST
URL	/api/templates/upload
请求头	Authorization: Bearer <JWT> Content-Type: multipart/form-data
请求参数	Form-data: file: UploadFile （必填）模板文件（xlsx / xls / docx） user_requirement: str （可选）填写要求文本 requirement_file: UploadFile （可选）要求附件
响应格式	{ id: int, template_name: str, template_mode: 'table_fill'|'field_fill', fields: [...] }

错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	上传后立即调用 template_filler.identify_fields 识别字段；旧版 .xls 走 xlrd。

表 10-24 接口 upload 表

2.接口名称：list_tpl

该接口用于「模板列表」。按 upload_time 倒序展示。

表 10-25 接口 list_tpl 表

字段	描述
请求方式	GET
URL	/api/templates/list
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	[{ id, template_name, template_type, template_mode, field_count, upload_time }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	按 upload_time 倒序展示。

3.接口名称：delete_tpl

该接口用于「删除模板」。级联删除 template_mapping（CASCADE）。

表 10-26 接口 delete_tpl 表

字段	描述
请求方式	DELETE
URL	/api/templates/{tid}
请求头	Authorization: Bearer <JWT>
请求参数	Path:

	tid: int
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	级联删除 template_mapping（CASCADE）。

4.接口名称：update_requirement

该接口用于「更新填写要求」。再次填表前可重写「用户要求」，影响 LLM 与 smart_fill 的策略。

表 10-27 接口 update_requirement 表

字段	描述
请求方式	PUT
URL	/api/templates/{tid}/requirement
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Path: tid: int Body: user_requirement: str （必填）
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	再次填表前可重写「用户要求」,影响 LLM 与 smart_fill 的策略。

5.接口名称：mappings

该接口用于「查看字段映射」。用于前端确认/校对模板字段。

表 10-28 接口 mappings 表

字段	描述

请求方式	GET
URL	/api/templates/{tid}/mappings
请求头	Authorization: Bearer <JWT>
请求参数	Path: tid: int
响应格式	[{ field_name, matched_value, confidence, source_file, source_position, need_confirm }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于前端确认/校对模板字段。

6.接口名称：fill

该接口用于「执行模板填写」。调用 smart_fill: 保留合并单元格/样式，自动汇总「合计、平均」列；图片字段走 image_filler。

表 10-29 接口 fill 表

字段	描述
请求方式	POST
URL	/api/templates/{tid}/fill
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Path: tid: int Body (可选): source_file_ids:List[int] （可选）指定数据源；为空则自动匹配
响应格式	{ task_id: int, output_file: str }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录

备注	调用 smart_fill：保留合并单元格/样式，自动汇总「合计、平均」列；图片字段走 image_filler。

7.接口名称：fill_history

该接口用于「填写历史分页」。用于前端「填表历史」页签。

表 10-30 接口 fill_history 表

字段	描述
请求方式	GET
URL	/api/templates/history
请求头	Authorization: Bearer <JWT>
请求参数	Query: page: int = 1 page_size: int = 20 keyword: str （可选）标题/文件名关键字
响应格式	{ items: [{ task_id, template_name, status, output_file, start_time, elapsed }], total: int }
错误处理	401 Unauthorized：未提供 token / token 无效或过期 500 Internal Server Error：服务端异常（详见日志）
访问权限	需登录
备注	用于前端「填表历史」页签。

8.接口名称：delete_history

该接口用于「删除一条历史」。同时删除关联的 output_file。

表 10-31 接口 delete_history 表

字段	描述
请求方式	DELETE
URL	/api/templates/history/{task_id}
请求头	Authorization: Bearer <JWT>
请求参数	Path: task_id: int
响应格式	{ ok: true }

错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	同时删除关联的 output_file。

9.接口名称：download_fill_by_task

该接口用于「按任务下载导出文件」。URL 中不含中文文件名，避免编码与库中字段不一致导致 403。

表 10-32 接口 download_fill_by_task 表

字段	描述
请求方式	GET
URL	/api/templates/history/{task_id}/download
请求头	Authorization: Bearer <JWT>
请求参数	Path: task_id: int
响应格式	二进制文件流（Content-Disposition 携带导出文件名）
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	URL 中不含中文文件名，避免编码与库中字段不一致导致 403。

10.接口名称：download

该接口用于「按文件名下载」。兼容老接口；存在编码问题时建议改用 history/{task_id}/download。

表 10-33 接口 download 表

字段	描述
请求方式	GET
URL	/api/templates/download/{filename}
请求头	Authorization: Bearer <JWT>
请求参数	Path: filename: str

响应格式	二进制文件流
错误处理	401 Unauthorized：未提供 token / token 无效或过期 500 Internal Server Error：服务端异常（详见日志）
访问权限	需登录
备注	兼容老接口；存在编码问题时建议改用 history/{task_id}/download。

10.6 智能对话与 Agent（/api/chat）

会话管理、消息流、附件上传与下载、Agent 多轮工具调用入口。

1.接口名称：list_sessions

该接口用于「会话列表」。按 updated_at 倒序；title 默认取首条问题摘要。

表 10-34 接口 list_sessions 表

字段	描述
请求方式	GET
URL	/api/chat/sessions
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	[{ id, title, updated_at }]
错误处理	401 Unauthorized：未提供 token / token 无效或过期 500 Internal Server Error：服务端异常（详见日志）
访问权限	需登录
备注	按 updated_at 倒序；title 默认取首条问题摘要。

2.接口名称：get_session_messages

该接口用于「会话历史消息」。用于切换会话时回填历史。

表 10-35 接口 get_session_messages 表

字段	描述
请求方式	GET
URL	/api/chat/sessions/{session_id}/messages
请求头	Authorization: Bearer <JWT>

请求参数	Path: session_id: int
响应格式	[{ role: 'user'|'assistant'|'tool', content: str, create_time: datetime }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于切换会话时回填历史。

3.接口名称：delete_session

该接口用于「删除会话」。级联删除 chat_message（CASCADE）。

表 10-36 接口 delete_session 表

字段	描述
请求方式	DELETE
URL	/api/chat/sessions/{session_id}
请求头	Authorization: Bearer <JWT>
请求参数	Path: session_id: int
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	级联删除 chat_message（CASCADE）。

4.接口名称：ask

该接口用于「提问 / 触发 Agent」。进入 tool_use 循环，最多 12 轮（MAX_TOOL_ROUNDS）；图片附件触发多模态消息块。

表 10-37 接口 ask 表

字段	描述
请求方式	POST
URL	/api/chat/ask

请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body: message: str （必填） history: List[Msg] （可选） session_id: int （可选） file_id: int （可选）单附件 ID file_ids: List[int] （可选）多附件 ID 集合
响应格式	{ answer: str, session_id: int, tool_calls: [{ name, input, output }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	进入 tool_use 循环，最多 12 轮（MAX_TOOL_ROUNDS）；图片附件触发多模态消息块。

5.接口名称：chat_upload

该接口用于「会话内上传附件」。文档触发后台解析；图片直接 done。完成后才进入「可勾选」附件列表。

表 10-38 接口 chat_upload 表

字段	描述
请求方式	POST
URL	/api/chat/upload
请求头	Authorization: Bearer <JWT> Content-Type: multipart/form-data
请求参数	Form-data: file: UploadFile （必填） Query: session_id: int （可选）
响应格式	{ file_id: int, file_type: str, status: 'pending'|'done' }

错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	文档触发后台解析；图片直接 done。完成后才进入「可勾选」附件列表。

6.接口名称：chat_file_list

该接口用于「会话可选附件列表」。返回当前用户所有可操作文档：已上传解析完成的 + 模板填写导出的文件。

表 10-39 接口 chat_file_list 表

字段	描述
请求方式	GET
URL	/api/chat/files
请求头	Authorization: Bearer <JWT>
请求参数	Query: session_id: int （可选）
响应格式	[{ id, file_name, file_type, source: 'upload'|'export' }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	返回当前用户所有可操作文档:已上传解析完成的 + 模板填写导出的文件。

7.接口名称：download_file

该接口用于「下载会话内文件」。兼容上传文档与填写导出文档（同一入口）。

表 10-40 接口 download_file 表

字段	描述
请求方式	GET
URL	/api/chat/files/{fid}/download
请求头	Authorization: Bearer <JWT>

请求参数	Path: fid: str 支持 'doc-<id>' / 'task-<id>' 复合 ID
响应格式	二进制文件流
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	兼容上传文档与填写导出文档（同一入口）。

10.7 仪表盘统计（/api/dashboard）

工作台数据，含文档/抽取/模板/任务计数与近期趋势。

1.接口名称：stats

该接口用于「工作台统计」。前端 ECharts 直接消费，定时 60s 刷新。

表 10-41 接口 stats 表

字段	描述
请求方式	GET
URL	/api/dashboard/stats
请求头	Authorization: Bearer <JWT>
请求参数	（无）

响应格式	{
	doc_count: int,
	extract_count: int,
	template_count: int,
	task_count: int,
	recent_uploads: [...],
	recent_tasks: [...],
	type_distribution: {...}
	}

错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）

访问权限	需登录
备注	前端 ECharts 直接消费，定时 60s 刷新。

10.8 系统设置（/api/settings）

管理员维护：大模型 / 嵌入模型 / MCP 开关、用户管理、操作日志查询。

1.接口名称：get_config

该接口用于「获取系统配置」。敏感字段（如 llm_api_key）在前端展示时通常做脱敏。

表 10-42 接口 get_config 表

字段	描述
请求方式	GET
URL	/api/settings/config
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	{ llm_api_base, llm_api_key, llm_model, llm_api_format, embedding_model, mcp_enabled }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	敏感字段（如 llm_api_key）在前端展示时通常做脱敏。

2.接口名称：set_config

该接口用于「更新系统配置」。更新后写入 system_config 表并热加载到 config 模块，下一次调用即刻生效。

表 10-43 接口 set_config 表

字段	描述
请求方式	PUT
URL	/api/settings/config
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body（字段均可选）:

	llm_api_base: str llm_api_key: str llm_model: str llm_api_format: 'anthropic'|'openai' embedding_model: str mcp_enabled: bool
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 403 Forbidden: 当前用户非管理员 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	管理员
备注	更新后写入 system_config 表并热加载到 config 模块，下一次调用即刻生效。

3.接口名称：list_users

该接口用于「用户列表」。用于「用户管理」页签。

表 10-44 接口 list_users 表

字段	描述
请求方式	GET
URL	/api/settings/users
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	[{ id, username, nickname, role, create_time }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 403 Forbidden: 当前用户非管理员 500 Internal Server Error: 服务端异常（详见日志）
访问权限	管理员
备注	用于「用户管理」页签。

4.接口名称：update_user

该接口用于「修改用户」。用于改角色与重置密码。

表 10-45 接口 update_user 表

字段	描述
请求方式	PUT
URL	/api/settings/users/{uid}
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Path: uid: int Body: role: str （可选）'admin'|'user' password: str （可选）重置后的明文（服务端自动 bcrypt 哈希）
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 403 Forbidden: 当前用户非管理员 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	管理员
备注	用于改角色与重置密码。

5.接口名称：delete_user

该接口用于「删除用户」。内置 admin 账号受保护、不可删除。

表 10-46 接口 delete_user 表

字段	描述
请求方式	DELETE
URL	/api/settings/users/{uid}
请求头	Authorization: Bearer <JWT>
请求参数	Path:

	uid: int
响应格式	{ ok: true }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 403 Forbidden: 当前用户非管理员 500 Internal Server Error: 服务端异常（详见日志）
访问权限	管理员
备注	内置 admin 账号受保护、不可删除。

6.接口名称：get_logs

该接口用于「操作日志分页」。非管理员通常只能看到本人相关条目（以当前实现为准）。

表 10-47 接口 get_logs 表

字段	描述
请求方式	GET
URL	/api/settings/logs
请求头	Authorization: Bearer <JWT>
请求参数	Query: page: int = 1 page_size: int = 20
响应格式	{ items: [{ id, user_id, username, action, detail, ip, create_time }], total: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	非管理员通常只能看到本人相关条目（以当前实现为准）。

10.9 任务中心（/api/tasks）

查询异步 / 长流程任务及统计。

1.接口名称：list_tasks

该接口用于「任务列表」。支持按 task_type、status 过滤；用于「任务中心」页签。

表 10-48 接口 list_tasks 表

字段	描述
请求方式	GET
URL	/api/tasks/list
请求头	Authorization: Bearer <JWT>
请求参数	Query: page: int = 1 page_size: int = 20 task_type: str （可选） status: str （可选）
响应格式	{ items: [{ id, task_type, task_name, status, related_id, output_file, start_time, end_time, elapsed }], total: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	支持按 task_type、status 过滤；用于「任务中心」页签。

2.接口名称：task_stats

该接口用于「任务统计」。用于工作台与任务中心的状态卡片。

表 10-49 接口 task_stats 表

字段	描述
请求方式	GET
URL	/api/tasks/stats
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	{ pending: int, running: int, done: int, failed: int }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	用于工作台与任务中心的状态卡片。

10.10 对外开放接口（X-API-Key）（/api/open）

面向 n8n / Coze / Dify 等低代码平台的开放 HTTP 接口集合。统一使用请求头 X-API-Key 鉴权（兼容 WEBHOOK_KEY 与 SECRET_KEY）。

1.接口名称：api_info

该接口用于「API 索引」。便于 n8n 节点的 schema 自动发现；也方便集成方做接入文档生成。

表 10-50 接口 api_info 表

字段	描述
请求方式	GET
URL	/api/open/
请求头	X-API-Key: <密钥>
请求参数	（无）
响应格式	{ version: '1.0', endpoints: [{ path, method, description }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	便于 n8n 节点的 schema 自动发现；也方便集成方做接入文档生成。

2.接口名称：upload

该接口用于「上传并解析」。落盘后立即触发解析任务；可结合 /api/open/extract 做下一步抽取。

表 10-51 接口 upload 表

字段	描述
请求方式	POST
URL	/api/open/upload
请求头	X-API-Key: <密钥> Content-Type: multipart/form-data
请求参数	Form-data:

	file: UploadFile （必填）
响应格式	{ file_id: int, file_name: str, parse_status: 'pending' }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	落盘后立即触发解析任务；可结合 /api/open/extract 做下一步抽取。

3.接口名称：extract

该接口用于「对已上传文件抽取」。走 LLM 深度抽取，要求 file 处于 parse_status='done' 状态。

表 10-52 接口 extract 表

字段	描述
请求方式	POST
URL	/api/open/extract
请求头	X-API-Key: <密钥> Content-Type: application/json
请求参数	Body: file_id: int （必填） fields: List[str] （可选）
响应格式	{ fields: { 字段名: 字段值, ... } }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	走 LLM 深度抽取，要求 file 处于 parse_status='done' 状态。

4.接口名称：extract_text

该接口用于「对纯文本抽取」。无需先上传文件，直接对一段文本做规则 +LLM 混合抽取。

表 10-53 接口 extract_text 表

字段	描述
请求方式	POST
URL	/api/open/extract-text
请求头	X-API-Key: <密钥> Content-Type: application/json
请求参数	Body: text: str （必填） fields: List[str] （可选）
响应格式	{ fields: { 字段名: 字段值, ... } }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	无需先上传文件，直接对一段文本做规则 + LLM 混合抽取。

5.接口名称：get_fields

该接口用于「查询已抽取字段」。可不带 file_id 返回当前 API Key 视野下的全部字段。

表 10-54 接口 get_fields 表

字段	描述
请求方式	GET
URL	/api/open/fields
请求头	X-API-Key: <密钥>
请求参数	Query: file_id: int （可选）

响应格式	[{ file_id, field_name, field_value, confidence, extract_method }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	可不带 file_id 返回当前 API Key 视野下的全部字段。

6.接口名称：get_fused

该接口用于「查询融合数据」。等价于内部 /api/extraction/fused 的对外版本。

表 10-55 接口 get_fused 表

字段	描述
请求方式	GET
URL	/api/open/fused
请求头	X-API-Key: <密钥>
请求参数	（无）
响应格式	{ fields: [{ field_name, value, candidates, confidence }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	等价于内部 /api/extraction/fused 的对外版本。

7.接口名称：semantic_search

该接口用于「语义搜索」。向量检索；模型不可用时降级为 jieba 词重叠打分。

表 10-56 接口 semantic_search 表

字段	描述
请求方式	POST
URL	/api/open/search
请求头	X-API-Key: <密钥> Content-Type: application/json

请求参数	Body: query: str （必填） top_k: int （可选，默认 5）
响应格式	[{ file_id, file_name, chunk_text, score }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	向量检索；模型不可用时降级为 jieba 词重叠打分。

8.接口名称：chat

该接口用于「轻量问答 (RAG)」。无 Agent 工具调用，仅做 RAG 检索 + 单轮 LLM 回答；适合快速问答场景。

表 10-57 接口 chat 表

字段	描述
请求方式	POST
URL	/api/open/chat
请求头	X-API-Key: <密钥> Content-Type: application/json
请求参数	Body: message: str （必填）
响应格式	{ answer: str, references: [{ file_name, chunk_text }] }
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	无 Agent 工具调用，仅做 RAG 检索 + 单轮 LLM 回答；适合快速问答场景。

9.接口名称：list_files

该接口用于「文件列表」。用于在工作流节点中枚举可操作的文件。

表 10-58 接口 list_files 表

字段	描述
请求方式	GET
URL	/api/open/files
请求头	X-API-Key: <密钥>
请求参数	（无）
响应格式	[{ file_id, file_name, file_type, parse_status, upload_time }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 401 Unauthorized: X-API-Key 缺失或不匹配 500 Internal Server Error: 服务端异常（详见日志）
访问权限	X-API-Key
备注	用于在工作流节点中枚举可操作的文件。

10.11 MCP 协议端点（/api/mcp）

面向 Claude Desktop / Cursor 等 MCP 客户端的外置工具集。protocolVersion 2024-11-05, serverInfo.name = "DocFusion MCP"。

1.接口名称：jsonrpc

该接口用于「JSON-RPC 主通道」。支持 initialize / tools/list / tools/call 三个核心方法; tools/call 派发到 mcp_service 中的 13 个工具。

表 10-59 接口 jsonrpc 表

字段	描述
请求方式	POST
URL	/api/mcp/jsonrpc
请求头	Authorization: Bearer <JWT> Content-Type: application/json
请求参数	Body (JSON-RPC 2.0): { jsonrpc: '2.0', id, method:

	'initialize'|'tools/list'|'tools/call', params: {...} }
响应格式	JSON-RPC 响应（含 result 或 error）
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	支持 initialize / tools/list / tools/call 三个核心方法; tools/call 派发到 mcp_service 中的 13 个工具。

2.接口名称：sse

该接口用于「SSE 心跳通道」。用于 MCP 客户端建立长连接;首条事件返回 message 通道地址。

表 10-60 接口 sse 表

字段	描述
请求方式	GET
URL	/api/mcp/sse
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	text/event-stream（持续推送 endpoint 提示与心跳）
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	公开（建立后由消息通道认证）
备注	用于 MCP 客户端建立长连接; 首条事件返回 message 通道地址。

3.接口名称：message

该接口用于「SSE 消息通道」。搭配 /api/mcp/sse 实现 MCP 的 streaming 形态;服务端按 Bearer 解析 uid。

表 10-61 接口 message 表

字段	描述
请求方式	POST
URL	/api/mcp/message
请求头	Authorization: Bearer <JWT>

	Content-Type: application/json
请求参数	Header: Authorization: Bearer <JWT> Body: JSON-RPC 2.0 报文
响应格式	JSON-RPC 响应
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 400 Bad Request: 请求体格式错误或必填字段缺失 500 Internal Server Error: 服务端异常（详见日志）
访问权限	Bearer Token
备注	搭配 /api/mcp/sse 实现 MCP 的 streaming 形态；服务端按 Bearer 解析 uid。

4.接口名称：tools

该接口用于「工具清单」。供工具发现：search_documents、list_files、get_file_content、extract_fields、extract_from_file、batch_extract、get_fused_data、get_extracted_fields、match_template_fields、summarize_document、compare_documents、generate_report、fill_template_auto。

表 10-62 接口 tools 表

字段	描述
请求方式	GET
URL	/api/mcp/tools
请求头	Authorization: Bearer <JWT>
请求参数	（无）
响应格式	[{ name: str, description: str, input_schema: object }]
错误处理	401 Unauthorized: 未提供 token / token 无效或过期 500 Internal Server Error: 服务端异常（详见日志）
访问权限	需登录
备注	供工具发现：search_documents、list_files、get_file_content、extract_fields、extract_from_file、batch_extract、get_fused_data、get_extracted_fields、match_template_fields、summarize_document、

	compare_documents、generate_report、fill_template_auto。

十一、关键技术

11.1 后端技术

1. 核心框架：Python 3 + FastAPI，提供 RESTful API，配合 Uvicorn 部署运行。

2. 数据访问：SQLAlchemy ORM，统一会话管理与事务边界；默认 SQLite，可替换为其他关系型数据库。

3. 认证机制：JWT（HS256）签发与校验，接口层通过依赖注入获取当前用户，保障资源访问安全。

4. 文档与 NLP：结合文档解析、分词（如 jieba）与抽取管道，支撑中文场景下的信息处理。

5. 文档智能编辑引擎：基于 python-docx / openpyxl 自研 doc_editor 服务，封装 Word 段落 replace/delete/append/insert_after、跨 run 文本替换、按标题/正文样式过滤的 format_docx；Excel 侧支持 set_cell/clear_cell、create_xlsx、format_xlsx（字体/边框/底纹/数字格式/列宽行高）、add_chart_xlsx（饼/折/柱/条）、validate_spreadsheet（whole/decimal/list/textLength/custom 等数据有效性）。

6. 混合抽取管道：extractor.hybrid_extract 同时执行规则抽取（电话、邮箱、日期、身份证、金额、地址、机构、URL、键值对、统计句、增速等多类正则）与 LLM JSON 抽取，再做字段融合，提升中文长文本的召回与稳健性。

7. 向量检索与降级：vector_store 优先采用 SentenceTransformers 嵌入 + 余弦相似度(NumPy 矩阵乘)实现语义检索；当模型不可用时自动降级为 jieba 分词 + 词重叠启发式打分，并按 user_id 做检索隔离。

8. 多源字段融合：fusion 服务通过 ALIASES 表归一字段名，按来源类型（表格 / 段落等）赋权，对同一字段的多候选执行投票打分，输出最终值、候选列表与置信度，支撑后续模板自动填表。

9. 旧版 Word 兼容：legacy_doc 通过 LibreOffice headless（支持 vendor 内置版本与 LIBREOFFICE_PROGRAM 环境变量）将 .doc(OLE) 转 .docx；Windows 环境可降级使用 Word COM（pywin32）。

11.2 前端技术

1. 单页应用：Vue.js 3 + Vue Router，组件化开发与路由守卫（基于会话令牌）控制未登录访问。

2. UI 与可视化：Element Plus 与 ECharts，统一交互与图表展示体验。

3. 工程化：Vite 开发与构建；开发态通过反向代理将 /api 转发至后端，降低跨域配置成本。

4. 本地开发时可在构建工具侧绕过系统代理直连本机 API（如设置 NO_PROXY 包含 localhost），避免代理劫持导致联调失败；生产环境仍建议使用反向代理与 HTTPS。

5. 富文本展示：Markdown 渲染（marked）等能力用于说明类内容展示。

6. 主题与换肤系统：通过 document.documentElement.style.setProperty 注入 CSS 变量（--primary-color、--bg-gradient-*、--card-blur、--palette-1~4 等）并同步覆盖 Element Plus 变量（--el-color-primary 等）；多预设主题、自定义主题与图表调色板均通过 localStorage（docfusion_custom_themes / docfusion_active_theme_id 等键）持久化。

7. 玻璃拟态视觉体系：global.css 提供多径向渐变背景、.glass-card 毛玻璃与 hover 光带动画，并对 Element Plus 关键组件做样式衔接，统一 DocFusion 视觉语言。

8. 登录页 Canvas 动态背景：基于 HTML5 Canvas 2D + requestAnimationFrame，绘制三角/圆/方形线框粒子，鼠标靠近吸引、点击产生径向冲量；ResizeObserver + devicePixelRatio 适配高清屏；色相围绕品牌蓝紫（baseHue=230）。

9. 自定义鼠标跟随：position:fixed 圆形描边层，监听 mousemove 平滑跟随，悬停可点击元素时放大并半透明填充；通过 ThemeSelector 内开关与 cursor-toggle 自定义事件联动；@media(pointer:coarse) 下自动隐藏，避免触控设备误触。

10. 响应式主壳：Layout 在 width<768 时切换至 el-drawer 抽屉式导航，桌面端保留固定侧栏；顶栏头像/昵称通过 avatar-updated / nickname-updated 自定义事件实时刷新，<router-view> 外层应用进出场过渡。

11.3 数据库与存储技术

1. 关系数据：业务实体（用户、模板、任务、系统配置等）结构化存储。

2. 文件存储：上传文件、导出结果等以服务器本地目录为主（uploads / exports 等），可按项目需要迁移至对象存储。

3. 向量与检索：向量块等数据以文件或专用存储维护，用于检索增强与对话场景。

11.4 系统集成技术

1. RESTful API：统一路径前缀与语义化资源划分（如 /api/documents、/api/templates 等）。

2. 跨域：后端 CORS 中间件支持浏览器端联调；生产环境可按域白名单收敛。

3. 大模型服务：通过可配置 Base URL 与密钥对接兼容 OpenAI 格式的推理服务，便于**切换供应商**。

4. 文档格式：可选集成 LibreOffice（项目提供安装脚本与 vendor 目录约定）以提升 Office 类文档转换能力。

5. **向量检索与智能体**：采用 sentence-transformers 等嵌入模型维护向量库，支

撑对话上下文检索与开放 API 中的语义搜索；后端集成文档编辑引擎，对话层可通过多轮工具调用完成 Word 段落编辑与版式调整、Excel 建表与图表、数据有效性等操作。

6. OpenAPI 开放接口：路由前缀 /api/open，使用 Header「X-API-Key」鉴权（兼容内置 WEBHOOK_KEY 与 SECRET_KEY），提供上传并解析、对已上传文件执行 LLM 深度抽取、纯文本抽取、字段查询、融合数据获取、语义搜索、轻量问答、文件列表等端点；GET /api/open/ 输出 JSON 形式 API 索引，便于 n8n / Coze / Dify 等低代码平台直接编排接入。

7. **MCP 协议端点**：基于 JSON-RPC 2.0（/api/mcp/jsonrpc）+ SSE（/api/mcp/sse）+ 消息通道（/api/mcp/message）实现 protocolVersion 2024-11-05 的 MCP 服务端，serverInfo.name="DocFusion MCP"；工具集合（/api/mcp/tools）涵盖 search_documents、list_files、get_file_content、extract_fields、extract_from_file、batch_extract、get_fused_data、get_extracted_fields、match_template_fields、summarize_document、compare_documents、generate_report、fill_template_auto 等。

8. **Agent 多轮工具调用**：智能对话以 Anthropic 风格的 tool_use / tool_result 协议驱动，最多 12 轮（MAX_TOOL_ROUNDS=12）调用 doc_editor.AGENT_TOOLS 中的工具，可在**一次会话内完成「搜索→抽取→编辑→排版→制图→校验」的端到端操作**。

9. 图片填充策略：image_filler 同时支持 WPS DISPIMG（解析 cellimages.xml、复制媒体与关系并写入 =_xlfn.DISPIMG 公式）与通用 openpyxl 浮动图片插入，覆盖国产/国际办公套件主流场景。

11.5 DevOps 与部署

1. 依赖管理：Python requirements.txt 固定后端依赖版本；前端使用 npm 管理依赖。

2. 部署形态：前后端可分离部署——静态资源由 Nginx 等托管，API 服务由进程管理器或容器编排运行；可根据并发与可用性要求引入负载均衡与数据库主从。

3. 配置与密钥：敏感信息通过环境变量或 .env（勿入库）注入，与代码仓库解耦。

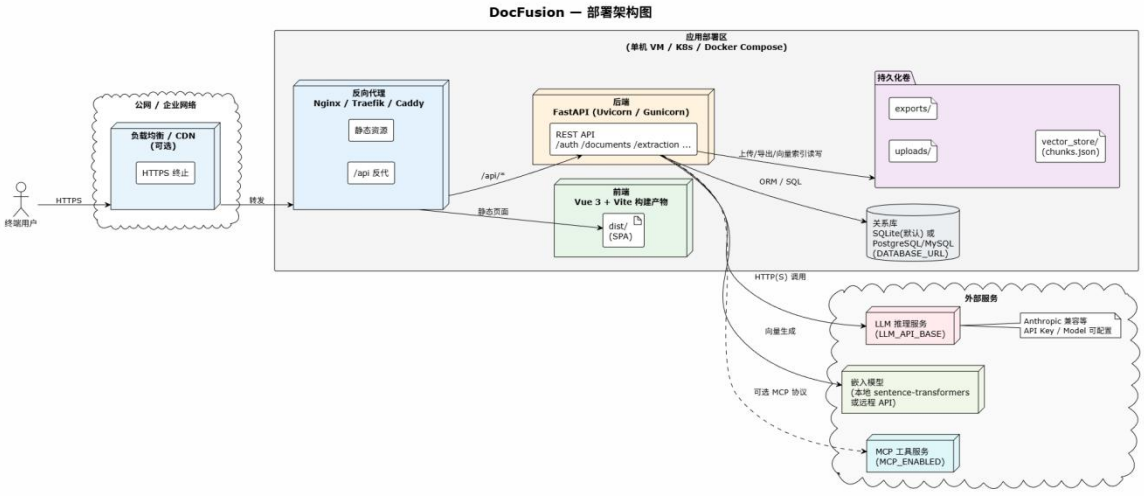


图 11-1 部署架构图

十二、具体功能实现与当前使用情况

12.1 功能实现情况

12.1.1 认证与用户模块

注册登录、JWT、会话令牌在前端会话存储中的使用；用户基本信息与头像等扩展字段。支持修改昵称、密码与上传头像；管理员可对用户列表进行角色维护、重置密码或删除用户（内置超级管理员受保护）。

12.1.2 文档管理模块

文档上传、列表与健康检查等接口联动；与解析管道衔接。支持批量删除、重新解析；支持对单文档触发 LLM 提取，以及对已解析文档的批量提取（extract-all）。

12.1.3 信息抽取模块

抽取任务与结果浏览，支撑从长文到字段的转化。支持对抽取结果进行编辑、批量删除记录；提供多源融合视图及按文件查看字段。

12.1.4 模板与填表模块

模板列表、历史、需求描述与填表生成流程。支持维护「用户要求」以约束填表；模板区分字段填表 / 表格填表等模式；填表结果记入任务，支持按填表任务 ID 下载生成文件，避免中文文件名在 URL 中的兼容问题。

1) 智能填表引擎：smart_fill 服务在保留模板合并单元格、表头样式、数字格式的前提下，对数据源与模板按列名做模糊匹配映射，自动跳过英文表头、空行、省份分组行等「非数据」行；内置 OCR 类脏字符清洗、相同字段去重；对显式/隐式的「合计、平均」列进行自动汇总与均值计算。

2) 多格式模板识别：template_filler 提供 identify_fields 入口，对 .xlsx / .xls / .docx 三类模板分别识别 table_fill（表头横向，数据下行）与 field_fill（字段名+值竖向）两种模式；旧版 .xls 走 xlrd 扫描表头以避免 openpyxl 不支持二进制流。

3) 图片填充：image_filler 支持 WPS DISPIMG 与通用浮动图片两套写入路径。

4) 旧版兼容：模板侧 .doc 文档统一经 legacy_doc 转换为 .docx 后再进入填表/编辑流水线。

12.1.5 智能对话模块

基于后端对话路由与前端会话界面，实现问答与辅助核验。 **支持多会话列表、历史消息与删除会话；会话内可上传文档或图片，可选择多个已解析文件作为上下文；对话采用向量检索与多轮 Agent 工具调用，可驱动 Word/Excel 文档的智能编辑、排版**

与数据分析类操作。

1) Agent 工具清单：search_documents（语义检索）、extract_info（按字段/全字段抽取）、list_files（列出用户文件）、edit_document（Word/Excel 内容编辑）、analyze_document（解析 Word 标题/正文层级）、format_document（按 heading/body 精细排版）、create_spreadsheet（新建 Excel）、format_spreadsheet（字体/边框/底纹/数字格式/列宽行高）、add_chart（饼/折/柱/条形图）、validate_spreadsheet（数据有效性 / 下拉菜单 / 自定义公式）、fill_template_smart（调用 smart_fill 完成模板自动填写）、get_extract_results（读取已抽取字段）。

2) 多模态消息：当用户在会话内勾选图片类附件时，build_image_content_blocks 会把图片以 base64 嵌入到 Anthropic 多模态消息块，配合 LLM 的视觉能力做截图/表格图理解。

3) RAG 拼装：每轮对话先以 vector_store.search(user_id=...) 取出相关文本块，再与已抽取字段摘要一并写入 system 提示，保证用户数据隔离的同时让模型获得结构化与非结构化双重上下文。

12.1.6 仪表盘与设置模块

运营统计展示；系统参数（模型与 MCP 等）维护。系统配置包含嵌入模型、MCP 开关等；管理员可查看操作日志（非管理员通常仅可见本人相关记录，以当前实现为准）。

个性化中心：右下角悬浮入口提供主题预设切换、自定义主题编辑器（基础色/四色背景渐变/卡片毛玻璃强度/图表调色板）以及自定义鼠标跟随开关，所有偏好通过 localStorage 持久化，跨会话生效。

12.1.7 任务与扩展模块

异步或长流程任务记录、输出文件字段等。任务中心提供任务列表与统计；填表类任务关联输出文件路径，与前端下载及开放 API 场景衔接。

开放接口生态：/api/open/*9 个端点配合 X-API-Key 鉴权，可被 n8n/Coze/Dify 等低代码工作流直接调用；/api/mcp/*4 个端点实现 JSON-RPC+SSE 双通道，供 ClaudeDesktop、Cursor 等 MCP 客户端把 DocFusion 作为外置工具集挂载。

12.1.8 界面与个性化模块

围绕「易看、易用、可定制」目标，DocFusion 内置一套独立的视觉与交互体系：

1)主题中心：8 套预设主题+自定义主题编辑器，支持基础色/背景渐变/卡片毛玻璃/图表调色板独立配置；

2)玻璃拟态：全局.glass-card 与多径向渐变背景，配合 Element Plus 样式无缝衔接；

3)动态登录页：Canvas 2D 几何粒子背景、鼠标交互与点击爆炸特效，整体围绕品牌蓝紫色相；

4)自定义鼠标：可在主题中心一键开关，触控设备自动隐藏；

5)响应式：< 768px 自动切换 el-drawer 抽屉导航，移动端可用（后续可考虑上线移动端）。

12.2 当前使用情况与界面截图

整张页面的背景用 Canvas 2D 绘制了一层动态粒子，蓝紫色的几何粒子会跟着鼠标移动、被鼠标吸引形成光晕，点击空白处还会触发一次「爆炸」扩散特效。中央是一个毛玻璃风格（backdrop-filter）的登录卡片，用户输入账号、密码，点「登录」后前端会带着 username / password 请求 /api/auth/login，后端校验通过会返回 access_token（JWT），前端把它写进 sessionStorage 并跳转到工作台。

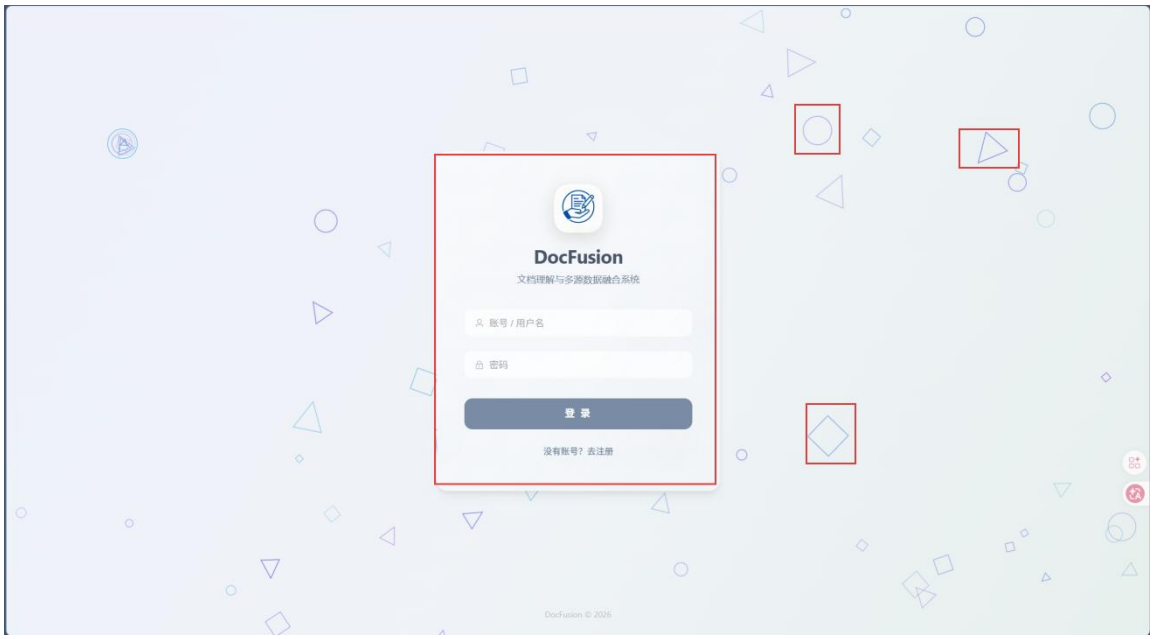


图 12-1 登录页 Canvas 动态背景（粒子吸引 / 点击爆炸效果）

这是登录之后打开的工作台上半屏。顶栏最左是系统 Logo 与侧栏折叠按钮，最右挂着当前用户的昵称和头像，头像支持随时替换（组件内部监听 avatar-updated 自定义事件，换头像后顶栏会立刻跟着刷新）。往下是四张核心统计卡片——文档总数、抽取次数、模板数量、任务数量，数字实时从 /api/dashboard/stats 接口拉回来的。



图 12-2 工作台上部分

工作台继续往下滚就是这张图的趋势分析区。左侧是一张 ECharts 折线图，展示最近 30 天的文档上传数、抽取次数等走势；右侧是近期任务列表，抽取、填表等任务的状态（pending / running / done / failed）一目了然。这里所有数据都是由 /api/dashboard/stats 一次性聚合返回的——我们特意把指标在后端汇总一次，前端只负责展示，这样首屏不用反复轮询多个接口，加载速度更快，用户看到的也是同一时刻的一致视图。



图 12-3 工作台下部分

右下角有一个悬浮的「调色盘」按钮，点开就是这张图里的个性化中心。上半部分是 8 套预设主题（深海蓝、樱花粉、森林绿、暗夜黑等），点一下立刻切换，整站的

卡片底色、按钮色、图表色都会同步变化。下半部分是自定义主题编辑器，用户可以单独调整基础色、四色背景渐变、卡片毛玻璃强度，甚至能改 ECharts 折线图/饼图的 10 色调色板。所有偏好都会写进 localStorage 持久化，下次登录或者切换设备登录同一账号时能自动还原，不用再配一遍。



图 12-4 个性化中心——主题预设与自定义编辑器（含背景渐变 / 毛玻璃 / 调色板）

这是文档管理的主界面。顶部是筛选条，可以按文件类型（docx / xlsx / pdf / 图片等）下拉过滤，也可以直接用文件名做模糊搜索。「上传文件」支持一次多选多份文件，选完后前端会依次调 /api/documents/upload，上传响应通常在 300 ms 以内——此时文件已落盘，但解析任务（切片 + 向量化）被放到后台 APScheduler 队列异步执行。这意味着用户点完上传就能继续操作别的页面，无需同步等待解析完成

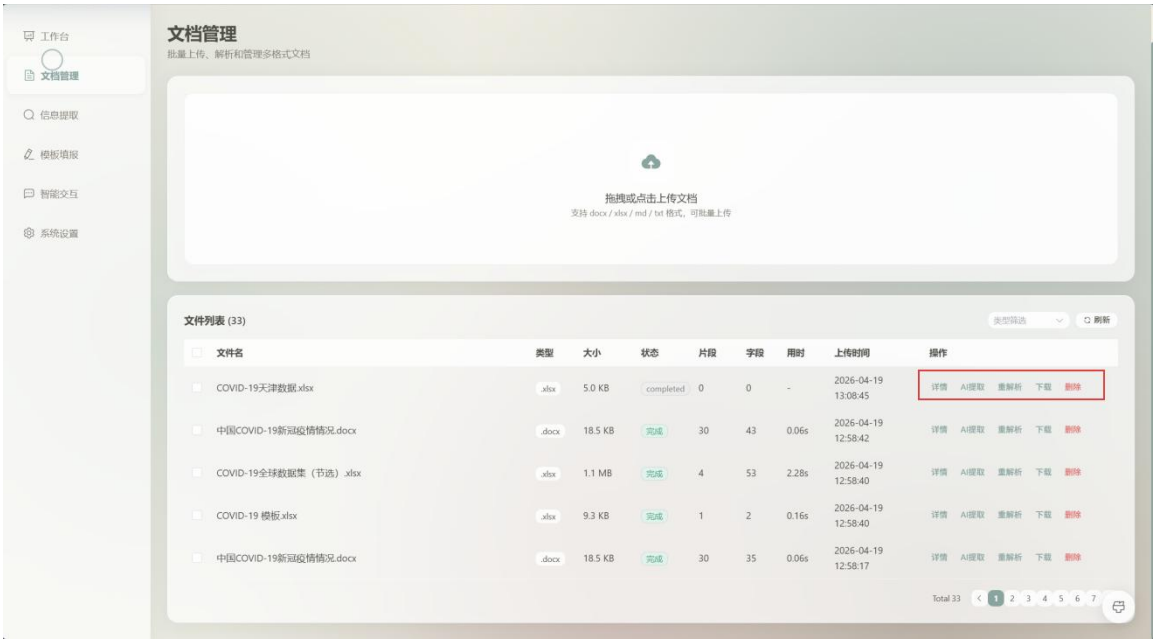


图 12-5 文档管理主界面

这是文档管理里的文件列表主表格。每一行最左边都有一个复选框，一旦勾选，表格上方就会浮出一条批量操作栏，点「批量删除」走 /api/documents/batch-delete，后端会在一次事务里同时清理 file_info、doc_chunk 两张表以及磁盘上的物理文件，避免出现「数据库里删了但盘上还留着」的孤儿文件。行右侧则是针对单文件的三个常用操作：「重新解析」会把 parse_status 置回 pending 重跑一次解析管道，「LLM 抽取」会调 /api/documents/{fid}/extract-all 对整份文件做一次 hybrid 抽取，「下载」直接把原始二进制原样返回。这种设计把「常用的单行操作」留在行级、「一次性操作多条」放在顶部，让两类操作不会混在一起。



图 12-6 文档管理-文件列表（可批量删除）

点表格里的文件名就会进入这张文件详情页，右侧抽屉会从侧边滑出来。上半部分是元数据（文件大小、解析状态、切片数、向量数、上传人、上传时间），下半部分是切片预览——每一条切片都带有 chunk_id、所属页码、原文片段。这个页面最实际的作用是让用户「自证解析有没有成功」：如果切片数为 0，就说明解析失败，需要点「重新解析」再跑一次；如果切片里出现乱码，基本可以判断原文是扫描件或图片 PDF，需要先走 OCR 再进抽取管道。

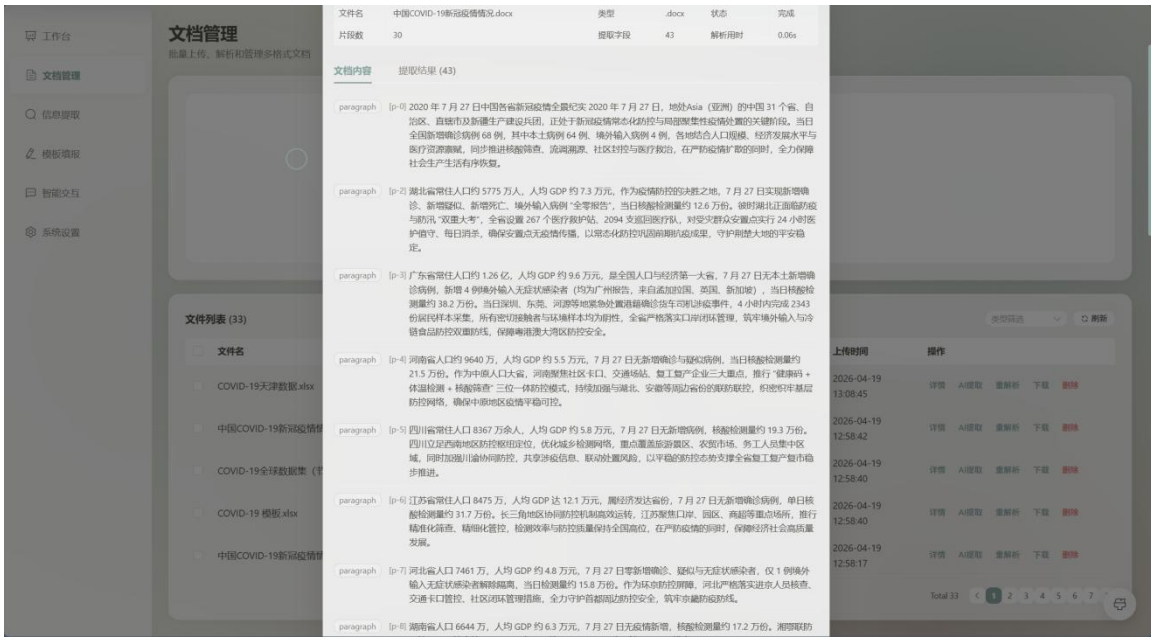


图 12-7 文档管理-文件详情

信息抽取模块默认打开的是「按文件查看」视图。左侧是一列已抽取过的文件清单，点选任意一份文件后，右侧会以分组卡片的形式把从这份文件里抽出来的字段展开——比如从一份合同里抽到的「甲方 / 乙方 / 金额 / 签订日期 / 有效期」等字段会按类别分成几组。接口走的是 /api/extraction/files/{fid}/fields，分组在后端按 extract_result.category 做 group by，让用户一眼就看得出「这是哪份文件、里面有哪些字段」，不需要自己在表格里来回翻。

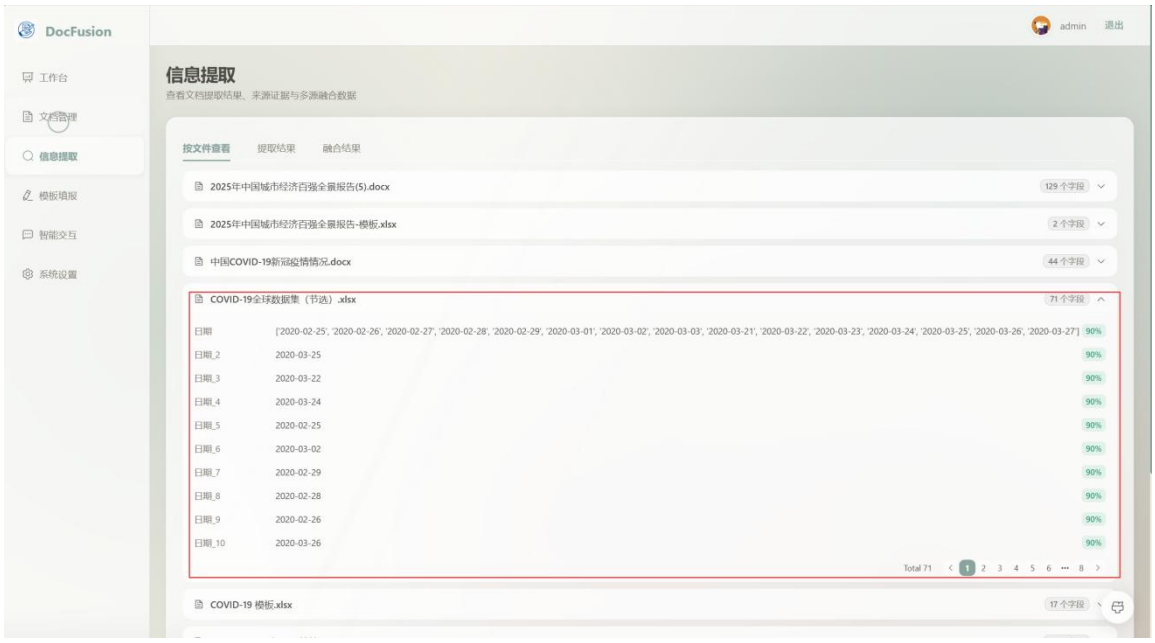


图 12-8 信息抽取-按文件查看

这是抽取结果的平铺视图。列上依次是字段名、字段值、置信度（0.00 - 1.00 浮点）、来源文件、来源页码。行末「编辑」按钮点下去变成就地编辑态，改完回车走 PUT

/api/extraction/results/{rid}——后端更新 value 并把 confidence 置为 1.0（人工修正视为完全可信）。LLM 抽偏的人名、金额这类字段，用户无需重跑整个抽取流程，几秒就能完成就地兜底。

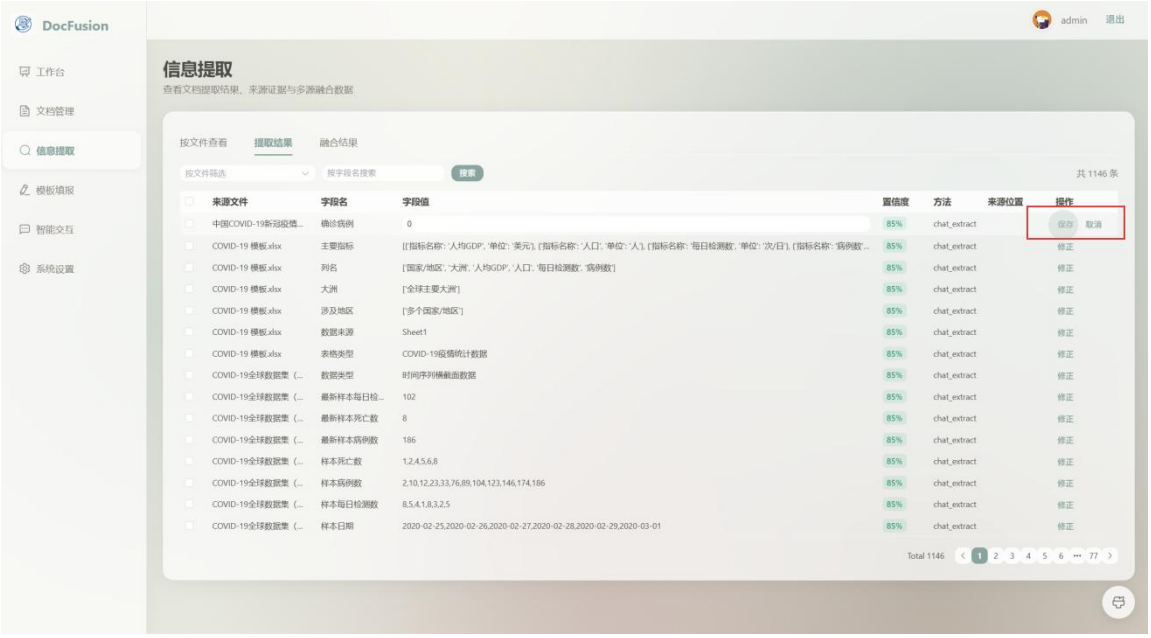


图 12-9 信息抽取-提取结果

这张图展示的是抽取结果的批量删除场景。用户勾选多条记录之后，顶部会浮出「批量删除」按钮，点下去先弹一个二次确认框（防止误删导致抽取结果丢失），确认后走 /api/extraction/results/batch-delete，后端会在一次事务里把选中的 extract_result 行全部删掉。这个动作在「重跑抽取」之前特别常见——先把之前的旧结果（可能是模型调参之前的）清掉，再跑一次新的抽取，避免新旧数据混在一起看不清。

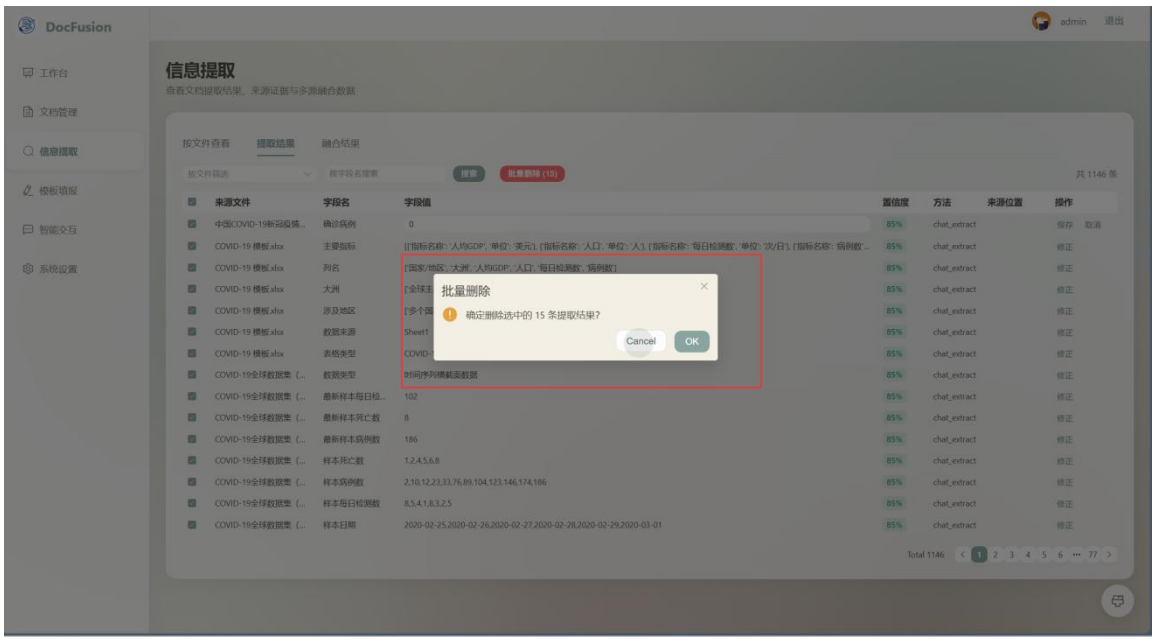


图 12-10 信息抽取-提取结果-批量删除

切到「多源融合视图」后，界面从"一份文件一份结果"切换成"按字段聚合"。同一字段（例如"公司名称"）会把 N 份文件里抽到的值全部拉到一起：相同值自动合并去重，冲突值标红并列出所有来源，用户可直接勾选一个作为"定稿值"。接口 /api/extraction/fused 的后端先做字段名归一（去:、空格、全半角差异），再按归一后的键聚合，适合合同 / 发票 / 身份证等多源数据交叉核验。

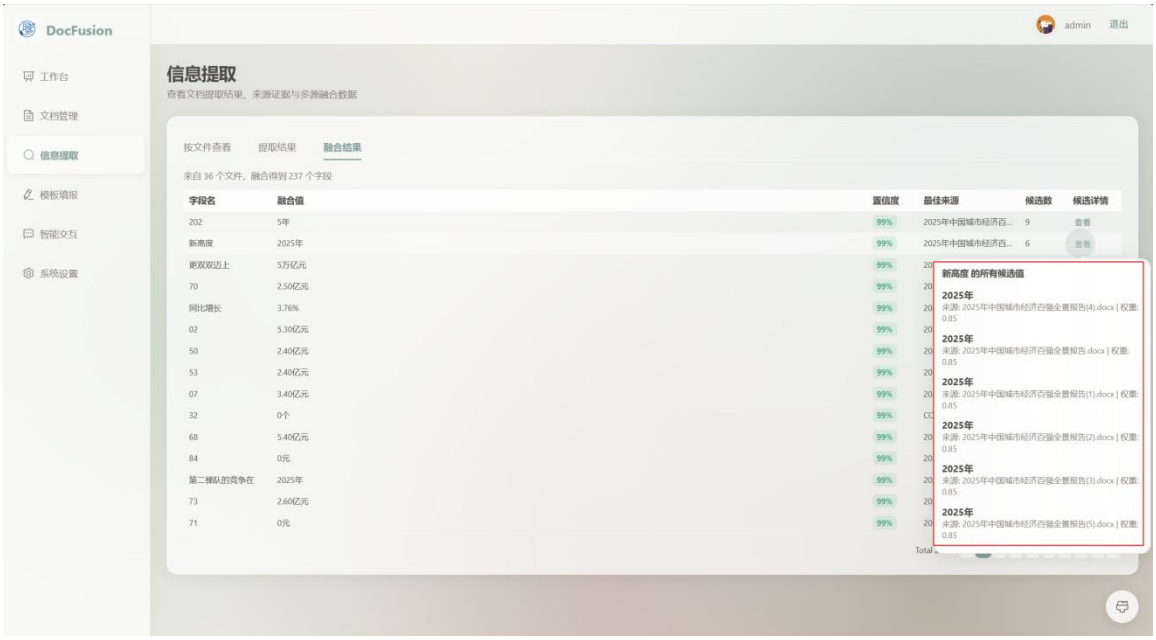


图 12-11 信息抽取-融合结果

模板填表是 DocFusion 的主打能力之一，这是它的入口界面。页面分三块：左「选择模板」从已上传模板里挑；中「用户要求」用自然语言写筛选 / 排版约束；右「选择数据源」勾选已抽取过的文件。三项齐备后点「开始填表」走 /api/templates/{tid}/fill——后端立即返回 task_id（通常 200 ms 内），真正的填表计算放到 APScheduler 队列异步跑，用户可在任务中心轮询进度。

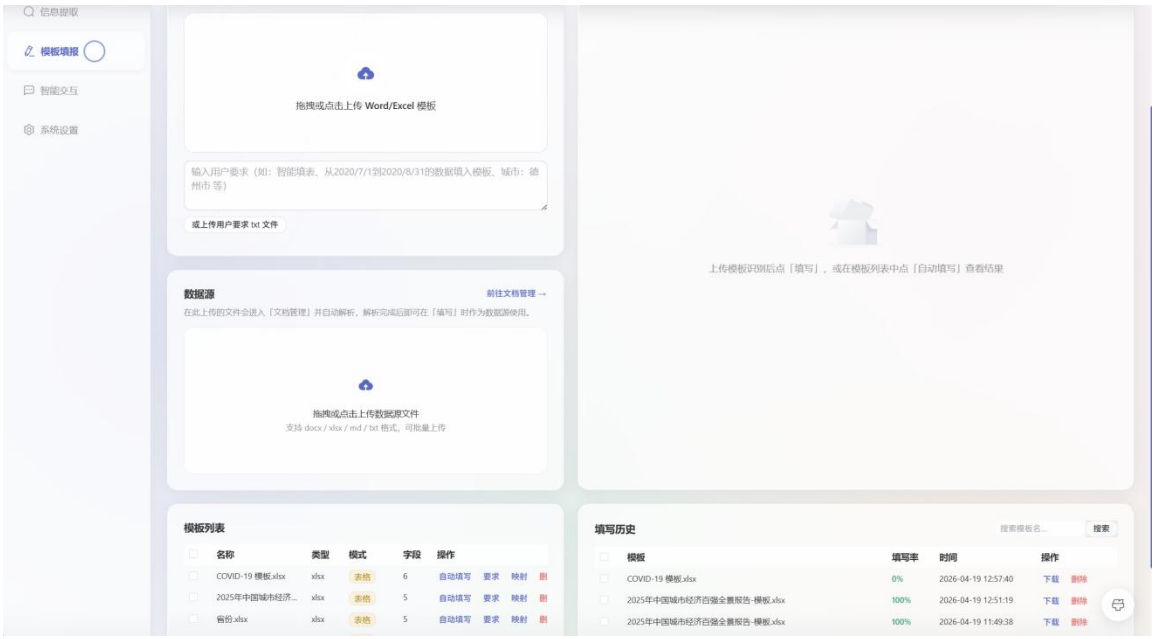


图 12-12 模板填表界面

这是填表完成后的结果页。左侧是实时预览的表格 / 文档，合并单元格、字号、颜色与原模板逐像素对齐；右侧是每个字段的置信度条。

置信度计算公式（smart_fill 引擎综合三项加权）：

字段名相似度（权重 0.5）：用 rapidfuzz.fuzz.ratio 对"用户要求字段 / 数据源列头"做模糊匹配，得分 0－100，归一到 0－1。

数据类型匹配度（权重 0.3）：目标列期望数字，实际值能否被 Decimal 解析；期望日期，能否被 dateparser.parse 成功；全部命中得 1，部分命中按比例打折。

数据完整性（权重 0.2）：值非空、不在 OCR 脏字符黑名单里、不属于"省份分组行 / 英文表头 / 空行"等噪声区间，全部命中得 1。

三项加权求和归一到[0, 1]，前端按阈值染色：

≥ 0.85 绿色（高可信，可直接交付）

0.6 – 0.85 黄色（建议复核）

< 0.6 红色（强烈建议人工修改）

用户顺着红色条往下看，就能快速锁定哪几列最易出错，不用从头复核到尾。

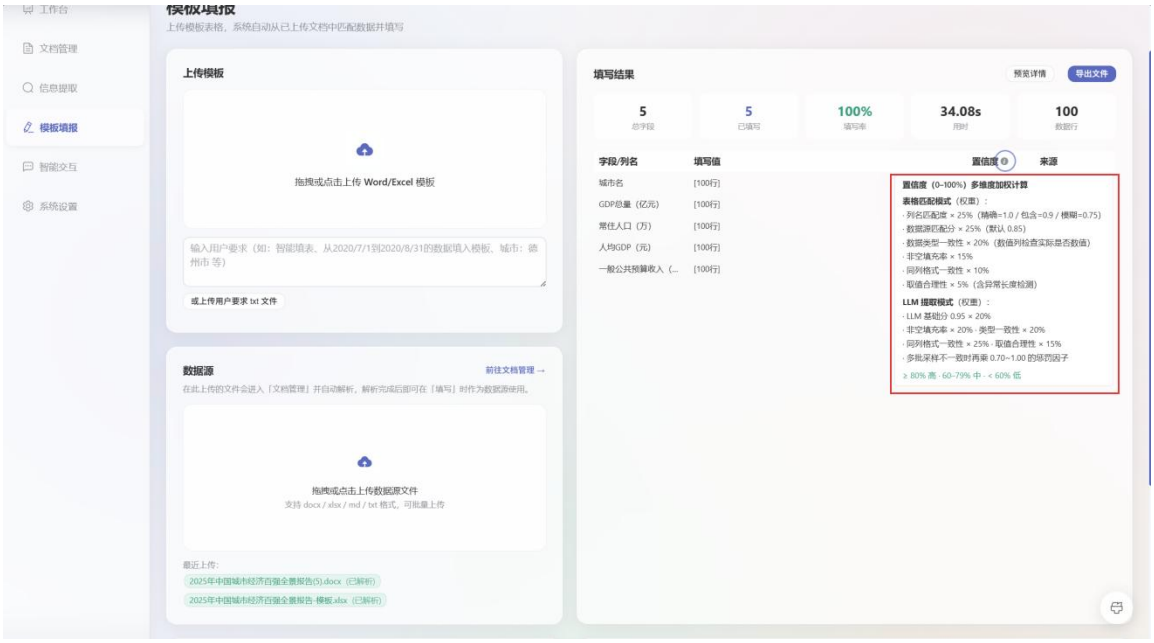


图 12-13 模板填表-填表结果（可查看置信度）

填表结果不满意时无需重选模板。点「编辑用户要求」在弹出文本框里改约束（比如由「取前 100 条」改成「取常住人口 500 万以上的城市」）再点一次填表即可。修改后的约束会和原模板重走 smart_fill，二次填表耗时通常比首次少 30% 以上——因为模板字段识别的结果被缓存，无需再扫一次表头。同一份模板就能在不同场景反复复用，真正做到「一次上传、多次复用」。

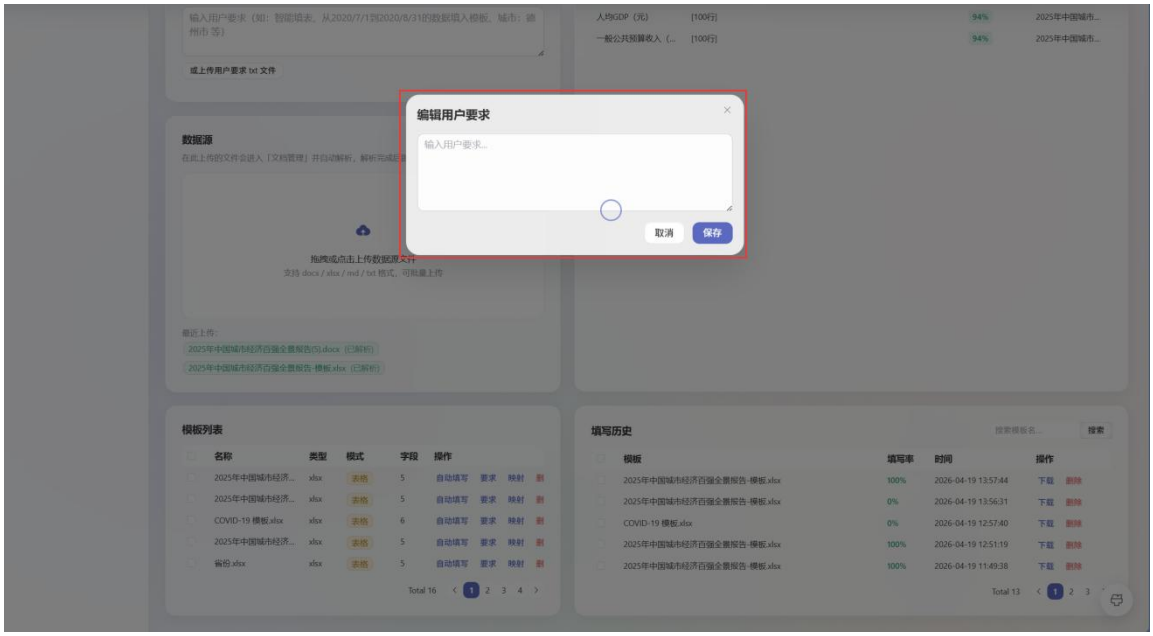


图 12-14 模板填表-编辑用户要求

这张图是填完之后的最终预览页。Excel 模板会被渲染成可滚动的表格，Word 模板会渲染成分页的文档视图，预览所用的字体 / 缩进 / 颜色和原文件保持一致，方便用户在不下载文件的情况下直接判断「能不能交付」。点右上角「下载」会调

/api/templates/history/{task_id}/download 返回填好的文件——注意这里走的是 task_id 而不是中文文件名，避免了 URL 编码对中文的兼容问题。

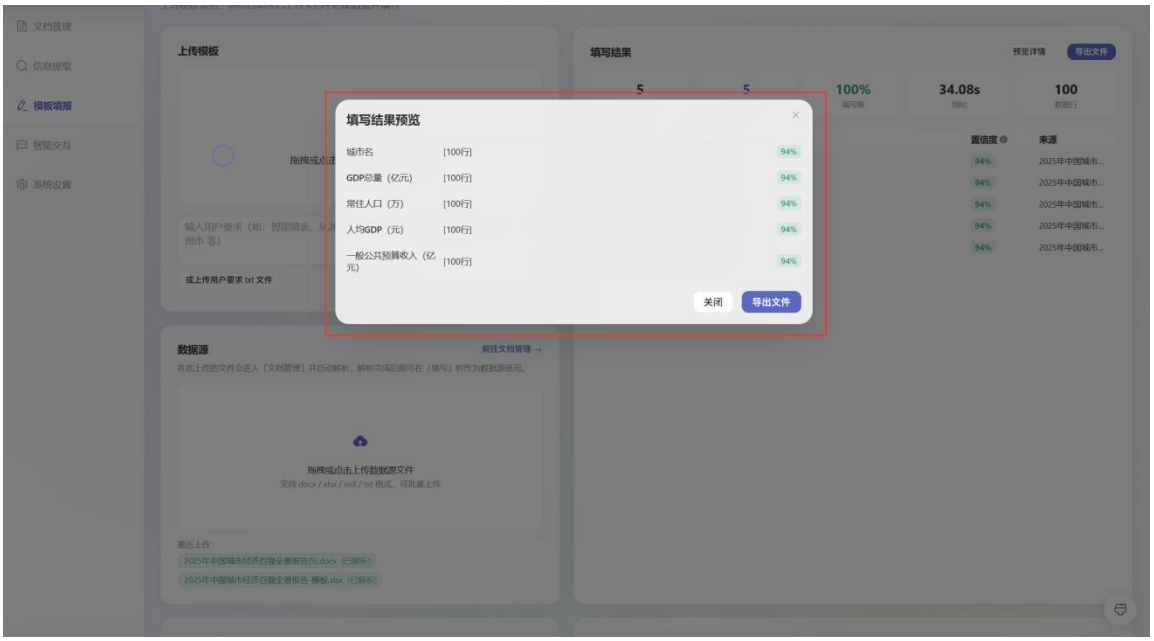


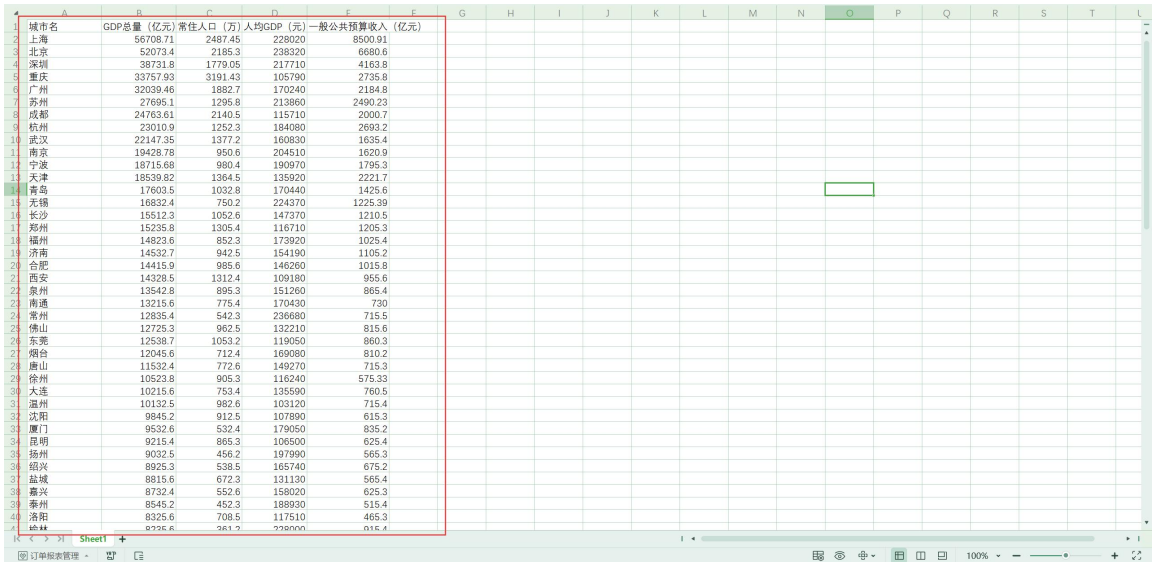
图 12-15 模板填表-填表结果预览

这是模板管理中心。上半部分是「我的模板」——每一张模板卡片都标明了 template_mode （table_fill 表示横向表头 + 数据下行，field_fill 表示字段名 + 值竖排两列）、字段数量、上传时间，用户一眼就能分辨出模板类型；下半部分是「填表历史」，按任务时间倒序排列，每条历史都带有「重新下载」和「再填一次」的快捷操作，不需要再走一遍完整的选模板 → 选数据 → 填表流程，对于要反复出同一类报表的场景效率提升很明显。



图 12-16 模板填表-模板列表，历史记录

这张图是用「中国城市百强榜」数据集做的一个真实场景演示（对于官方测试要求填写率 100%）。用户可在「用户要求」文本框里用自然语言输入多条约束——「只取常住人口 500 万以上的城市」「按省份分组并加合计行」「GDP 保留两位小数」，这些要求会原文送进 LLM 和 smart_fill 作为筛选与排版指令。和传统 BI 工具「拖字段到模板」的交互相比，这种「说人话」的方式对普通业务人员几乎零门槛，不用懂函数也不用懂 SQL。



城市名	GDP总量（亿元）	常住人口（万）	人均GDP（元）	一般公共预算收入（亿元）
上海	56708.71	2487.45	228020	8500.91
北京	52073.4	2185.3	238320	6680.6
深圳	38731.8	1779.05	217710	4163.8
重庆	33757.93	3191.43	105790	2735.8
广州	32039.46	1882.7	170240	2184.8
苏州	27695.1	1295.8	213860	2490.23
成都	24763.61	2140.5	115710	2000.7
杭州	23010.9	1252.3	184080	2693.2
武汉	22147.35	1377.2	160830	1635.4
南京	19428.78	950.6	204510	1620.9
宁波	18715.68	980.4	190970	1795.3
天津	18539.82	1364.5	135920	2221.7
青岛	17603.5	1032.8	170440	1425.6
无锡	16832.4	750.2	224370	1225.39
长沙	15512.3	1052.6	147370	1210.5
郑州	15235.8	1305.4	116710	1205.3
福州	14823.6	852.3	173920	1025.4
济南	14532.7	942.5	154190	1105.2
合肥	14415.9	985.6	146260	1015.8
西安	14328.5	1312.4	109180	955.6
泉州	13542.8	895.3	151260	865.4
南通	13215.6	775.4	170430	730
常州	12835.4	542.3	236680	715.5
佛山	12725.3	962.5	132210	815.6
东莞	12538.7	1053.2	119050	860.3
烟台	12045.6	712.4	169080	810.2
唐山	11532.4	772.6	149270	715.3
徐州	10523.8	905.3	116240	575.33
大连	10215.6	753.4	135590	760.5
温州	10132.5	982.6	103120	715.4
沈阳	9845.2	912.5	107890	615.3
厦门	9532.6	532.4	179050	835.2
昆明	9215.4	865.3	106500	625.4
扬州	9032.5	456.2	197990	565.3
绍兴	8925.3	538.5	165740	675.2
盐城	8815.6	672.3	131130	565.4
嘉兴	8732.4	552.6	158020	625.3
泰州	8545.2	452.3	188930	515.4
洛阳	8325.6	708.5	117510	465.3
榆林	8235.6	361.2	228000	915.4
宜昌	8052.3	402.5	200050	425.3
惠州	7935.4	608.2	130470	515.3
珠海	7832.5	248.3	315440	565.2
临沂	7725.4	1105.6	69870	415.3
保定	7632.5	925.3	82480	395.4
潍坊	7545.3	942.6	80050	465.2
鄂尔多斯	7425.3	220.5	336750	825.6
济宁	7352.4	835.2	88030	415.3
淄博	7235.6	472.3	153200	395.2
呼和浩特	7125.3	318.5	223710	365.4
太原	7032.5	542.3	129680	395.3
南阳	6935.2	965.4	71840	315.6
赣州	6825.4	905.3	75390	335.2
绵阳	6732.6	492.5	136700	315.3
滁州	6645.3	405.2	164000	315.4
邯郸	6532.4	952.3	68590	315.2
遵义	6425.5	662.4	97000	295.3
江门	6332.6	485.3	130490	295.2
襄阳	6113.85	525.3	116390	285.4
常德	6025.4	522.3	115360	275.3
宜宾	5932.5	462.4	128300	295.4
廊坊	5925.6	552.4	107270	310.5
株洲	5832.4	392.5	148590	265.3
新乡	5725.3	622.4	91980	255.2
周口	5632.5	905.3	62210	215.3
芜湖	5532.4	372.5	148520	295.3
岳阳	5425.3	502.4	108000	255.2
聊城	5332.4	592.3	90030	235.2
德阳	5225.3	352.4	148270	235.3
茂名	5132.5	625.3	82080	215.2
连云港	5032.4	462.3	108850	235.2
汕头	4925.3	552.4	89160	215.2
邢台	4832.5	712.3	67840	195.2
衡阳	4725.4	662.3	71350	205.2
曲靖	4632.5	582.4	79540	195.2
荆州	4532.4	522.3	86780	185.2
肇庆	4425.3	412.4	107300	195.2
信阳	4332.5	622.3	69620	175.2
商丘	4225.4	772.3	54710	165.2
驻马店	4132.5	702.3	58840	155.2
桂林	4025.3	492.4	81750	165.2
安顺	3932.4	252.3	155860	155.2
上饶	3852.5	652.3	59060	175.2
安庆	3756.2	415.2	90470	165.3
莆田	3752.3	320.5	117080	185.6
菏泽	3725.4	875.3	42560	145.2
郴州	3702.5	472.3	78390	145.2
宁德	3685.4	312.4	117970	165.3
九江	3652.3	452.3	80750	155.2
咸阳	3625.4	422.3	85850	145.2
宜春	3585.3	502.3	71380	145.2
德州	3562.4	562.3	63350	135.2
南平	3542.5	272.3	130090	135.2
清远	3525.3	402.3	87630	135.2
益阳	3502.4	382.3	91610	125.2
黄冈	3485.3	582.3	59850	125.2
荆门	3462.4	262.3	132000	125.2
六安	3445.3	442.3	77900	115.2
邵阳	3432.5	652.3	52620	115.2
怀化	3425.6	462.3	74100	105.3

图 12-17 百强测试集完整填写结果（仅输入智能填表）

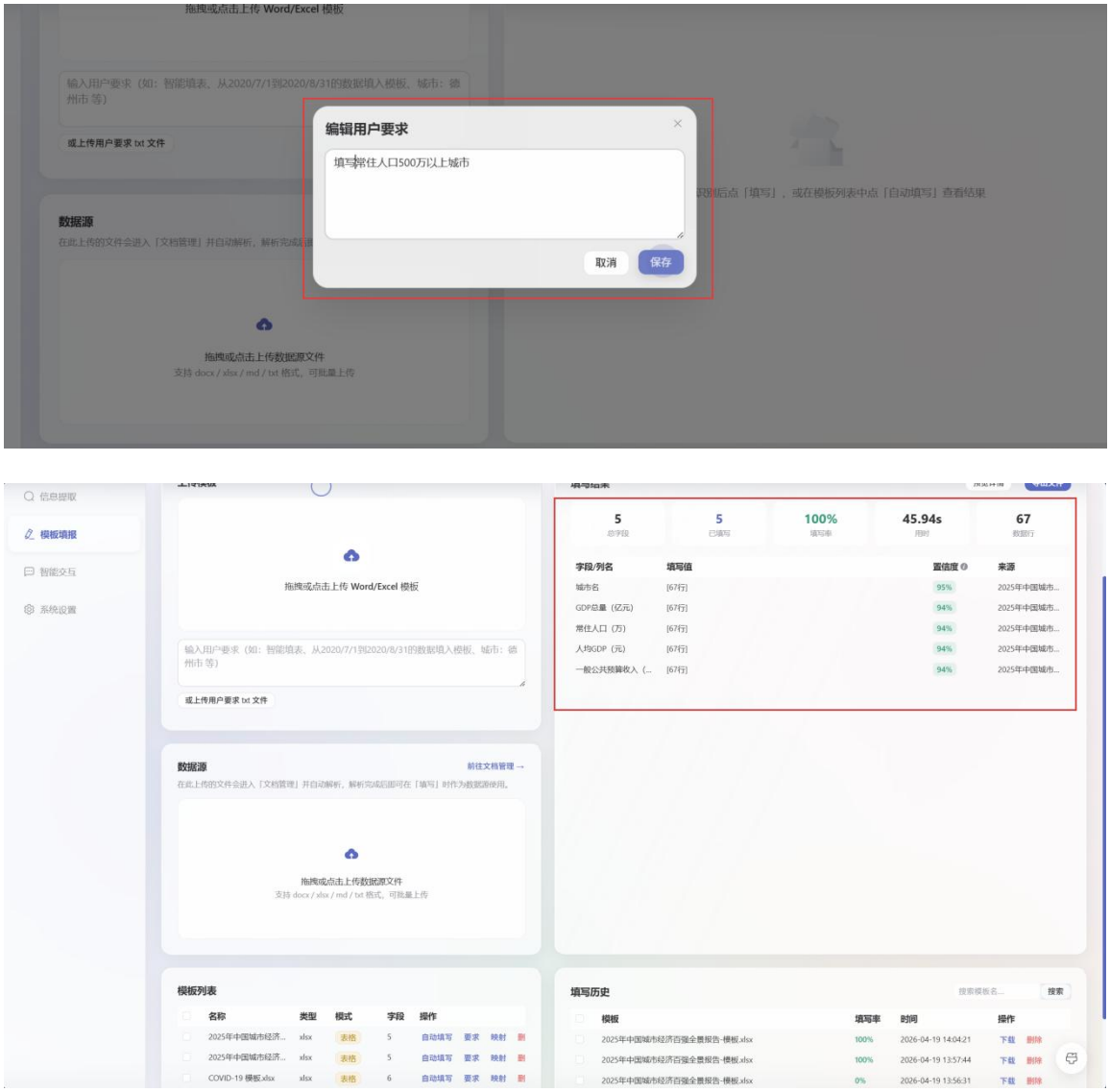
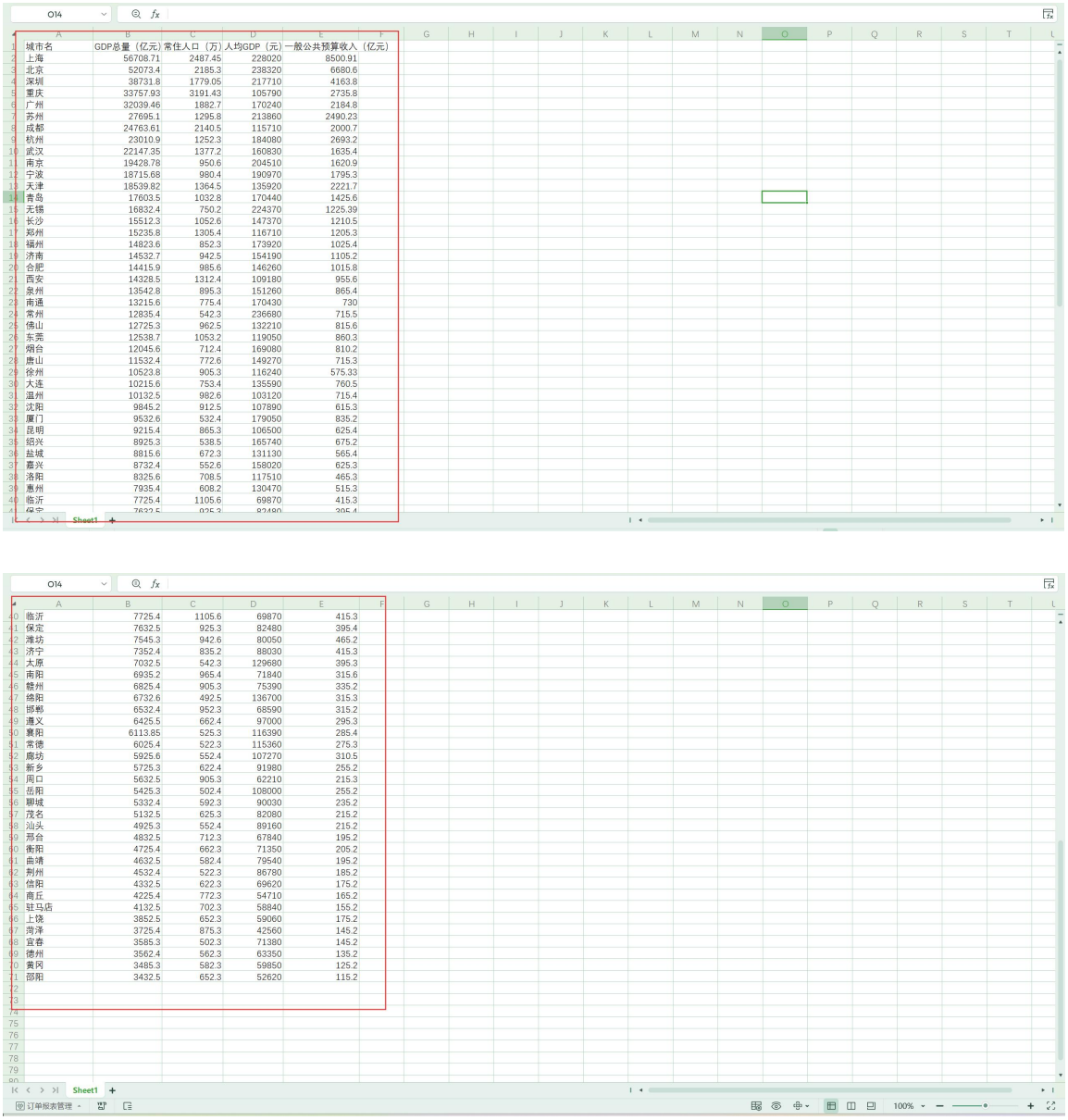


图 12-18 百强测试集（用户要求）

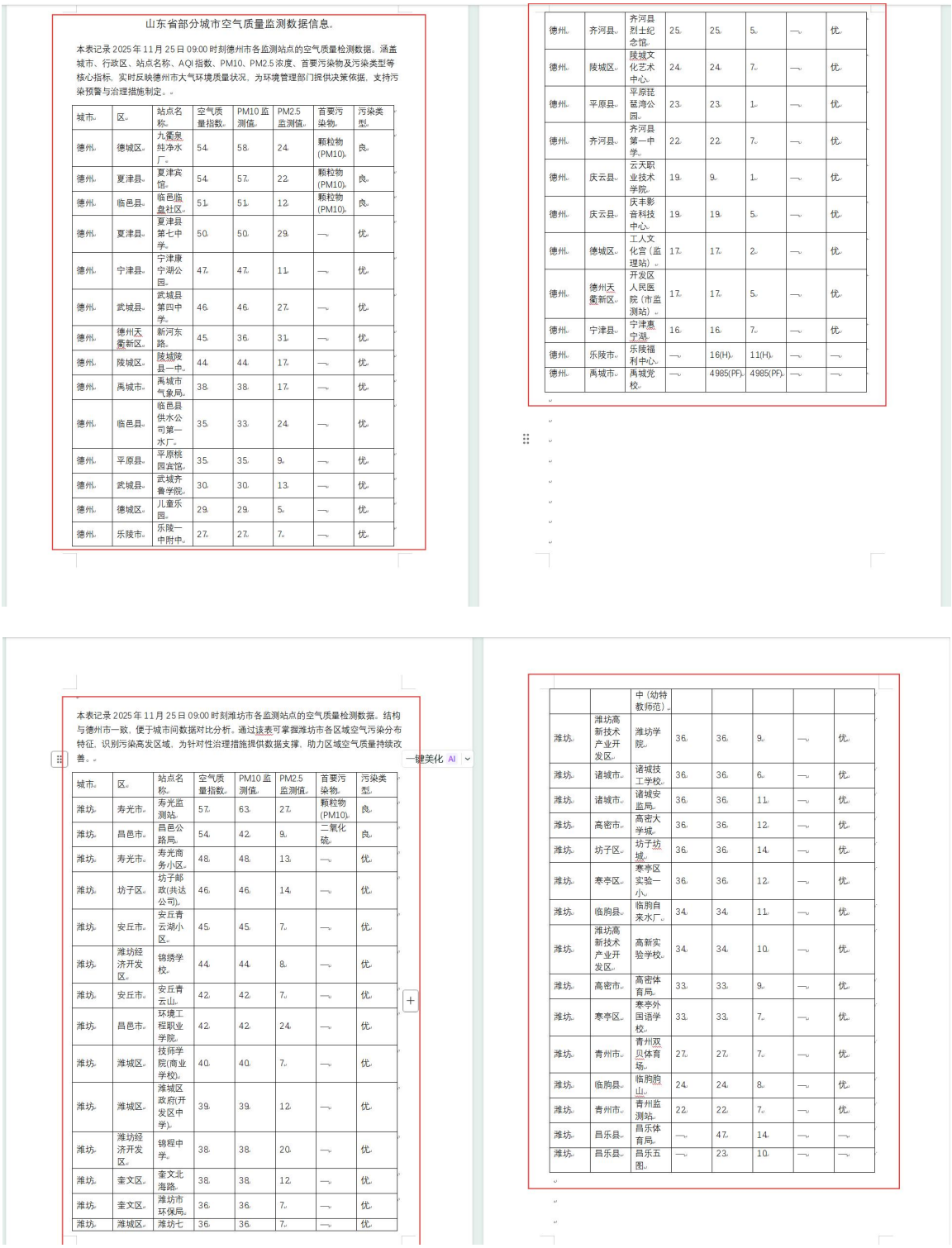
这张是上一张图的实际填表结果。smart_fill 在完整保留原模板合并单元格、表头样式、千分位格式的前提下，按列名模糊匹配（「常住人口」↔「人口(万)」相似度 > 80）叠加「>= 500 万」的阈值过滤，最后自动补一行「合计」（SUM）与一行「平均」（AVG）——这两行的公式是通过列头关键词"合计 / 小计 / 总和 / 平均"的正则命中自动注入的。输出通过/api/templates/history/{task_id}/download 下载。



城市名	GDP总量（亿元）	常住人口（万）	人均GDP（元）	一般公共预算收入（亿元）
上海	56708.71	2487.45	228020	8500.91
北京	52073.4	2185.3	238320	6680.6
深圳	38731.8	1779.05	217710	4163.8
重庆	33757.93	3191.43	105790	2735.8
广州	32039.46	1882.7	170240	2184.8
苏州	27695.1	1295.8	213860	2490.23
成都	24763.61	2140.5	115710	2000.7
杭州	23010.9	1252.3	184080	2693.2
武汉	22147.35	1377.2	160830	1635.4
南京	19428.78	950.6	204510	1620.9
宁波	18715.68	980.4	190970	1795.3
天津	18539.82	1364.5	135920	2221.7
青岛	17603.5	1032.8	170440	1425.6
无锡	16832.4	750.2	224370	1225.39
长沙	15512.3	1052.6	147370	1210.5
郑州	15235.8	1305.4	116710	1205.3
福州	14823.6	852.3	173920	1025.4
济南	14532.7	942.5	154190	1105.2
合肥	14415.9	985.6	146260	1015.8
西安	14328.5	1312.4	109180	955.6
泉州	13542.8	895.3	151260	865.4
南通	13215.6	775.4	170430	730
常州	12835.4	542.3	236680	715.5
佛山	12725.3	962.5	132210	815.6
东莞	12538.7	1053.2	119050	860.3
烟台	12045.6	712.4	169080	810.2
唐山	11532.4	772.6	149270	715.3
徐州	10523.8	905.3	116240	575.33
大连	10215.6	753.4	135590	760.5
温州	10132.5	982.6	103120	715.4
沈阳	9845.2	912.5	107890	615.3
厦门	9532.6	532.4	179050	835.2
昆明	9215.4	865.3	106500	625.4
绍兴	8925.3	538.5	165740	675.2
盐城	8815.6	672.3	131130	565.4
嘉兴	8732.4	552.6	158020	625.3
洛阳	8325.6	708.5	117510	465.3
惠州	7935.4	608.2	130470	515.3
临沂	7725.4	1105.6	69870	415.3
保定	7632.5	925.3	82480	395.4
潍坊	7545.3	942.6	80050	465.2
济宁	7352.4	835.2	88030	415.3
太原	7032.5	542.3	129680	395.3
南阳	6935.2	965.4	71840	315.6
赣州	6825.4	905.3	75390	335.2
绵阳	6732.6	492.5	136700	315.3
邯郸	6532.4	952.3	68590	315.2
遵义	6425.5	662.4	97000	295.3
襄阳	6113.85	525.3	116390	285.4
常德	6025.4	522.3	115360	275.3
廊坊	5925.6	552.4	107270	310.5
新乡	5725.3	622.4	91980	255.2
周口	5632.5	905.3	62210	215.3
岳阳	5425.3	502.4	108000	255.2
聊城	5332.4	592.3	90030	235.2
茂名	5132.5	625.3	82080	215.2
汕头	4925.3	552.4	89160	215.2
邢台	4832.5	712.3	67840	195.2
衡阳	4725.4	662.3	71350	205.2
曲靖	4632.5	582.4	79540	195.2
荆州	4532.4	522.3	86780	185.2
信阳	4332.5	622.3	69620	175.2
商丘	4225.4	772.3	54710	165.2
驻马店	4132.5	702.3	58840	155.2
上饶	3852.5	652.3	59060	175.2
菏泽	3725.4	875.3	42560	145.2
宜春	3585.3	502.3	71380	145.2
德州	3562.4	562.3	63350	135.2
黄冈	3485.3	582.3	59850	125.2
邵阳	3432.5	652.3	52620	115.2

图 12-19 百强测试集（常住人口 500 万以上的城市）

这张用的是山东各地市空气质量数据模板，里面有一列「城市市徽」图片字段。image_filler 针对图片类字段做了双通道兼容：一条路径解析 xl/cellimages.xml 并写入=_xlfn.DISPIMG("ID", 1)公式，对应 WPS 专有格式；另一条路径走 openpyxl 的浮动图片 API，对应 Microsoft Excel 标准格式。系统在填表时先尝试 DISPIMG，失败再 fallback 到 openpyxl，一次填表兼容两套办公套件，在国产 / 国际软件混用的企业尤其实用。



山东省部分城市空气质量监测数据信息。

本表记录 2025 年 11 月 25 日 09:00 时刻德州市各监测站点的空气质量检测数据，涵盖城市、行政区、站点名称、AQI 指数、PM10、PM2.5 浓度、首要污染物及污染类型等核心指标，实时反映德州市大气环境质量状况，为环境管理部门据供决策依据，支持污染预警与治理措施制定。

城市	区	站点名称	空气质量指数	PM10 监测值	PM2.5 监测值	首要污染物	污染类型
德州	德城区	九衢泉纯净水厂	54	58	24	颗粒物(PM10)	良
德州	夏津县	夏津宾馆	54	57	22	颗粒物(PM10)	良
德州	临邑县	临邑临盘社区	51	51	12	颗粒物(PM10)	良
德州	夏津县	夏津县第七中学	50	50	29	—	优
德州	宁津县	宁津康宁湖公园	47	47	11	—	优
德州	武城县	武城县第四中学	46	46	27	—	优
德州	德州天衢新区	新河东路	45	36	31	—	优
德州	陵城区	陵城陵县一中	44	44	17	—	优
德州	禹城市	禹城市气象局	38	38	17	—	优
德州	临邑县	临邑县供水公司第一水厂	35	33	24	—	优
德州	平原县	平原桃园宾馆	35	35	9	—	优
德州	武城县	武城齐鲁学院	30	30	13	—	优
德州	德城区	儿童乐园	29	29	5	—	优
德州	乐陵市	乐陵一中附中	27	27	7	—	优
德州	齐河县	齐河县烈士纪念馆	25	25	5	—	优
德州	陵城区	陵城文化艺术中心	24	24	7	—	优
德州	平原县	平原琵琶湾公园	23	23	1	—	优
德州	齐河县	齐河县第一中学	22	22	7	—	优
德州	庆云县	云天职业技术学院	19	9	1	—	优
德州	庆云县	庆丰影音科技中心	19	19	5	—	优
德州	德城区	工人文化宫（监理站）	17	17	2	—	优
德州	德州天衢新区	开发区人民医院（市监测站）	17	17	5	—	优
德州	宁津县	宁津惠宁湖	16	16	7	—	优
德州	乐陵市	乐陵福利中心	—	16(H)	11(H)	—	—
德州	禹城市	禹城党校	—	4985(PF)	4985(PF)	—	—

本表记录 2025 年 11 月 25 日 09:00 时刻潍坊市各监测站点的空气质量检测数据。结构与德州市一致，便于城市间数据对比分析。通过该表可掌握潍坊市各区域空气污染分布特征，识别污染高发区域，为针对性治理措施提供数据支撑，助力区域空气质量持续改善。

一键美化 AI

城市	区	站点名称	空气质量指数	PM10 监测值	PM2.5 监测值	首要污染物	污染类型
潍坊	寿光市	寿光监测站	57	63	27	颗粒物(PM10)	良
潍坊	昌邑市	昌邑公路局	54	42	9	二氧化硫	良
潍坊	寿光市	寿光商务小区	48	48	13	—	优
潍坊	坊子区	坊子邮政(共达公司)	46	46	14	—	优
潍坊	安丘市	安丘青云湖小区	45	45	7	—	优
潍坊	潍坊经济开发区	锦绣学校	44	44	8	—	优
潍坊	安丘市	安丘青云山	42	42	7	—	优
潍坊	昌邑市	环境工程职业学院	42	42	24	—	优
潍坊	潍城区	技师学院(商业学校)	40	40	7	—	优
潍坊	潍城区	潍城区政府(开发区中学)	39	39	12	—	优
潍坊	潍坊经济开发区	锦程中学	38	38	20	—	优
潍坊	奎文区	奎文北海路	38	38	12	—	优
潍坊	奎文区	潍坊市环保局	36	36	7	—	优
潍坊	潍城区	潍坊七中（幼特教师范）	36	36	7	—	优
潍坊	潍坊高新技术产业开发区	潍坊学院	36	36	9	—	优
潍坊	诸城市	诸城技工学校	36	36	6	—	优
潍坊	诸城市	诸城安监局	36	36	11	—	优
潍坊	高密市	高密大学城	36	36	12	—	优
潍坊	坊子区	坊子坊城	36	36	14	—	优
潍坊	寒亭区	寒亭区实验一小	36	36	12	—	优
潍坊	临朐县	临朐自来水厂	34	34	11	—	优
潍坊	潍坊高新技术产业开发区	高新实验学校	34	34	10	—	优
潍坊	高密市	高密体育局	33	33	9	—	优
潍坊	寒亭区	寒亭外国语学校	33	33	7	—	优
潍坊	青州市	青州双贝体育场	27	27	7	—	优
潍坊	临朐县	临朐胭山	24	24	8	—	优
潍坊	青州市	青州监测站	22	22	7	—	优
潍坊	昌乐县	昌乐体育局	—	47	14	—	—
潍坊	昌乐县	昌乐五图	—	23	10	—	—

图 12-20 山东空气质量测试集

这是智能对话的主界面。左侧会话列表按 updated_at 时间戳倒序排列——每发一条消息都会刷新该字段，所以最近在用的会话永远在最上面。每条会话标题默认取首条提问摘要，支持右键改名。新建 / 切换 / 删除都走/api/chat/sessions 系列接口，并同步本地状态。多会话的核心价值是上下文隔离：「合同分析」「报表填表」「日常问答」各开一个会话，彼此不共享 Agent 记忆，避免话题互相污染。



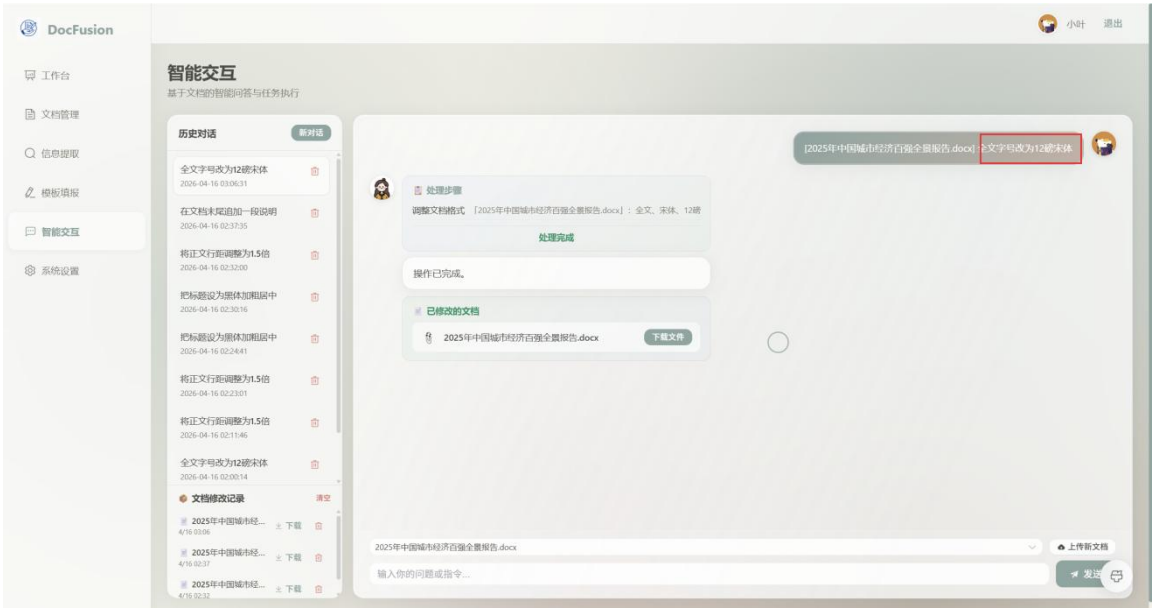
图 12-21 智能对话—会话列表与多会话切换

这张图展示会话内附件上传与多文件勾选面板。点输入框上方「+ 附件」走 POST /api/chat/upload。两条处理路径的时间特征完全不同：文档附件进入后台解析队列，parse_status 走 pending → running → done，平均耗时 3－15 秒（按文件大小与切片数浮动），只有 done 后才能在附件面板"可勾选"；图片附件不走解析管道，直接标记 done，上传完立刻可用。勾选的 file_ids 会在下一次/api/chat/ask 里作为 Agent 的可操作上下文一起带进去。



图 12-22 会话内上传附件与多文件勾选

这是 Agent 多轮工具调用的经典案例。用户说「把这份合同的正文都改成小四号字」，Agent 内部自动串联：先调 analyze_document 读取整份 Word 的 heading / body 层级结构，再调 format_document 把所有 body 段落 run 的 size 改成 12 pt（即小四），保存后把下载链接返回到对话里。整个过程受 12 轮工具调用上限保护——超过就强制回到人类对话，避免模型陷入工具调用死循环。



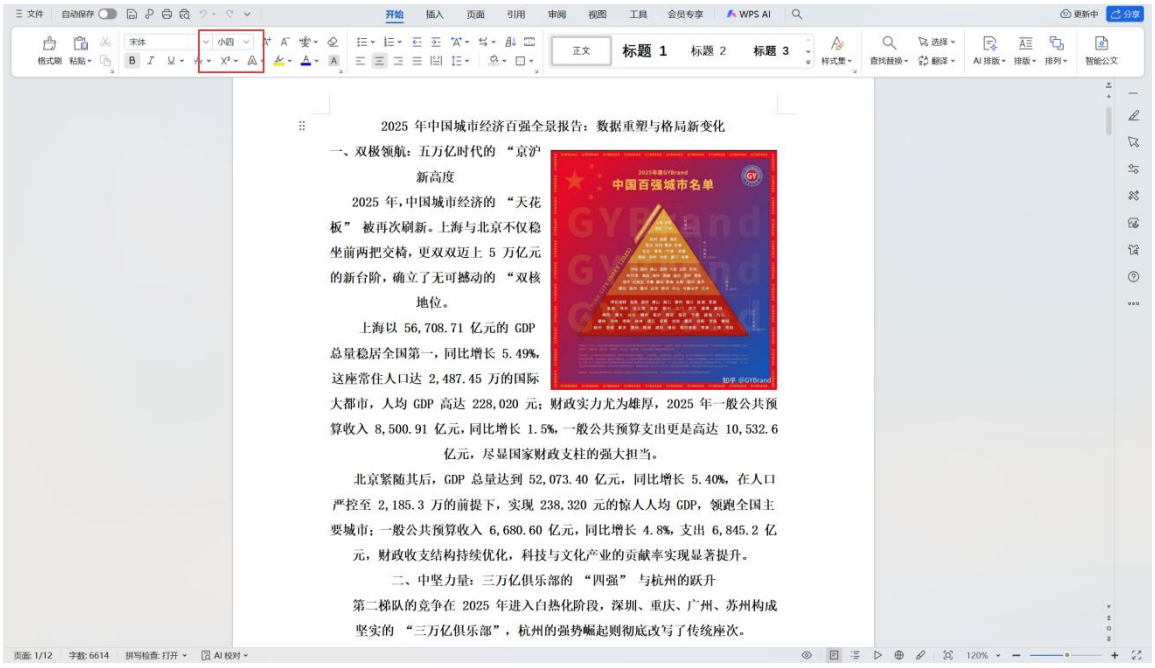


图 12-23 自然语言修改字号

这张图和上一张是同一套逻辑，只是用户的需求换成了「把所有一级标题的字体颜色改成深蓝色，字号加粗」。Agent 同样会调 format_document，定位到所有 Heading 1 段落，把每个 run 的 color 属性设成 #1F3864、bold 设成 True，保存后把新文件的下载链接返回到对话里。对于日常办公里最常见的「改格式」场景，基本不用离开对话框，也不用记住 Word 的菜单路径在哪里。

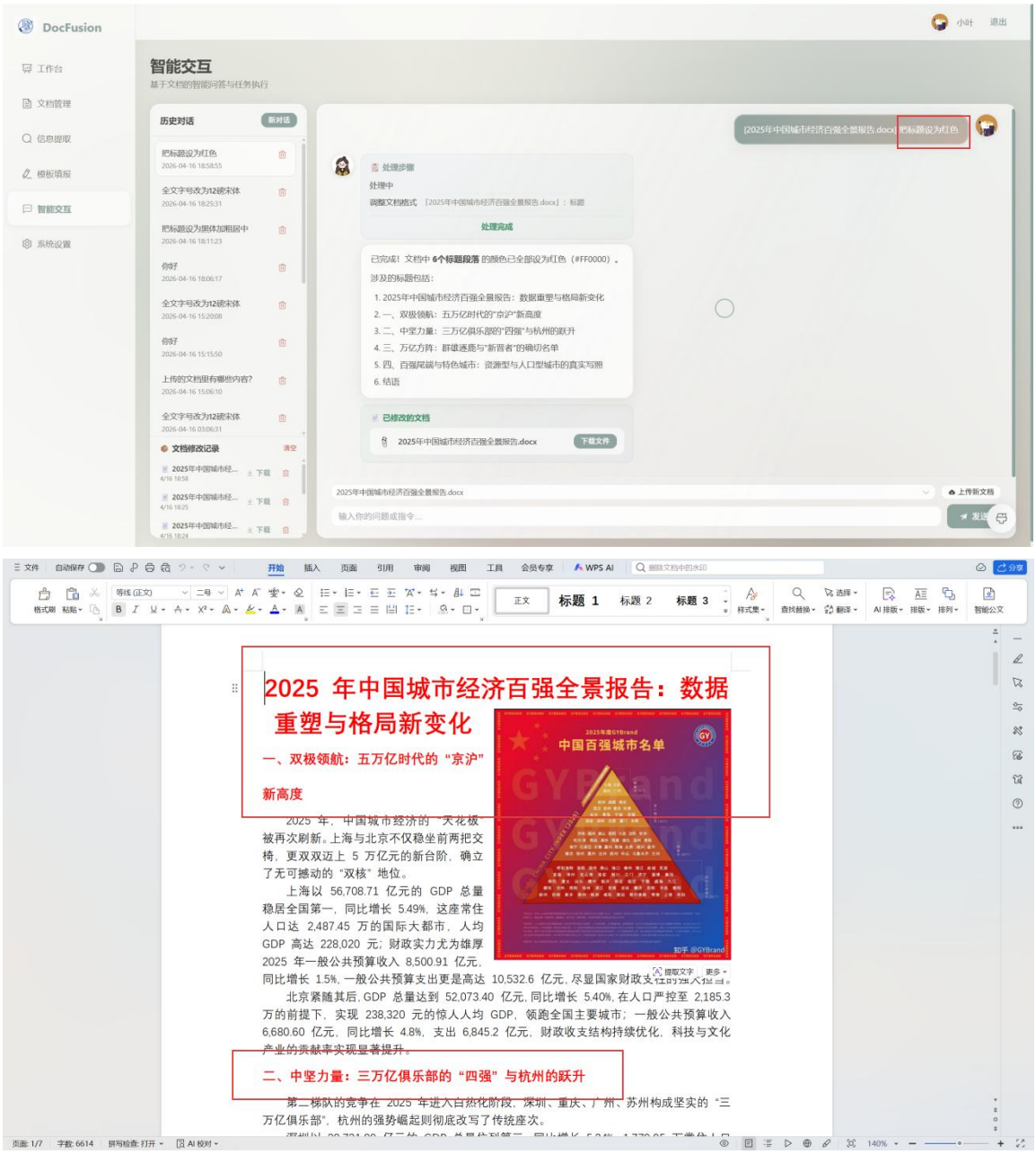
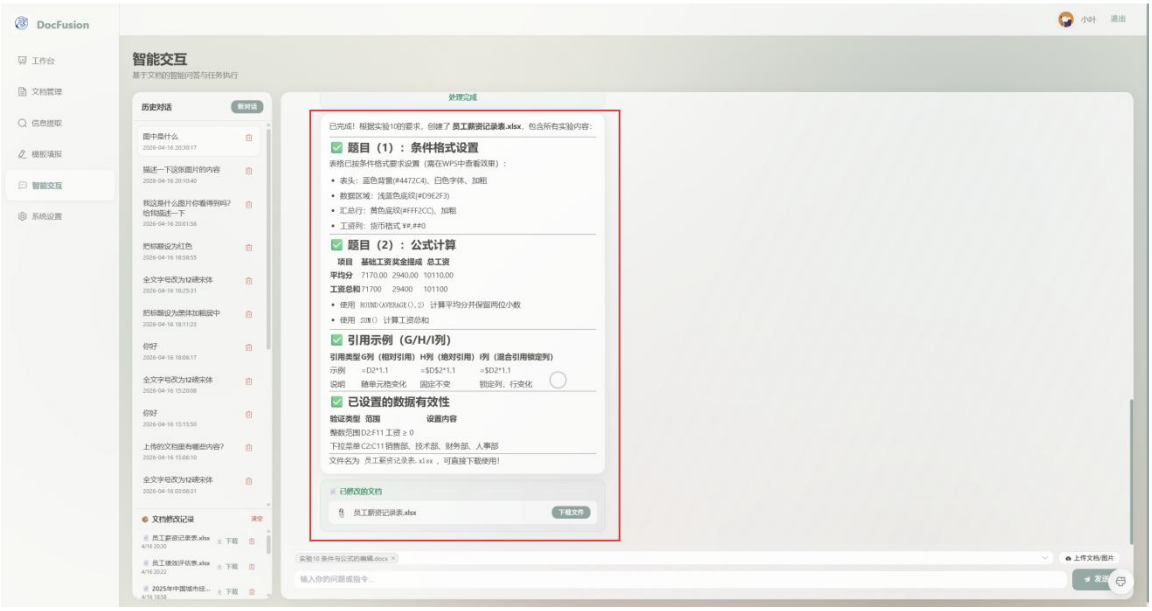
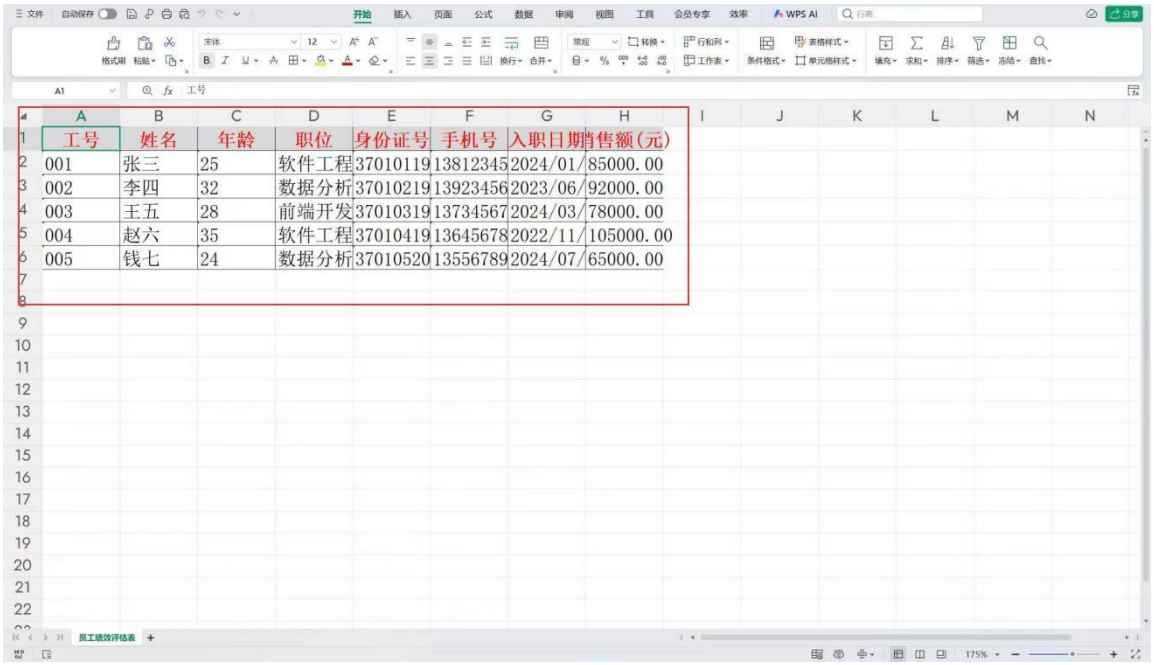


图 12-24 自然语言修改字体颜色

这张图集中展示了 Agent 实际执行后对 Word 和 Excel 产生的效果。一次对话里 Agent 可以在 12 轮工具调用限制内把 search_documents → analyze_document → format_document → add_chart → validate_spreadsheet → fill_template_smart 等多个工具串起来，比如用户说「在报表里加一张柱状图 + 加一列数据有效性下拉」，Agent 就会同时触发 add_chart 和 validate_spreadsheet。所有编辑结果都会直接写回用户原先的 Word / Excel 文件，最终通过 /api/chat/files/{fid}/download 提供下载。


工作台
文档管理
信息提取
模板填报
智能交互
系统设置
智能交互
基于文档的智能问答与任务执行
历史对话
新对话
图中是什么
2026-04-16 20:30:17
描述一下这张图片的内容
2026-04-16 20:10:40
我这是什么图片你看得到吗？给我描述一下
2026-04-16 20:01:56
把标题设为红色
2026-04-16 18:58:55
全文字号改为12磅宋体
2026-04-16 18:25:31
把标题设为黑体加粗居中
2026-04-16 18:11:23
你好
2026-04-16 18:06:17
全文字号改为12磅宋体
文档修改记录
清空
员工薪资记录表.xlsx
4/16 20:30
员工绩效评估表.xlsx
4/16 20:22
2025年中国城市经...
4/16 18:58
下载
[实验10 条件与公式的编辑.docx] 完成实验
处理步骤
处理中
处理完成
已完成！根据实验10的要求，创建了 员工薪资记录表.xlsx，包含所有实验内容：
题目（1）：条件格式设置
表格已按条件格式要求设置（需在WPS中查看效果）：
实验10 条件与公式的编辑.docx
上传文档/图片
输入你的问题或指令...
发送


DocFusion
小叶
退出
工作台
文档管理
信息提取
模板填报
智能交互
系统设置
智能交互
基于文档的智能问答与任务执行
历史对话
新对话
处理完成
已完成！根据实验10的要求，创建了 员工薪资记录表.xlsx，包含所有实验内容：
题目（1）：条件格式设置
表格已按条件格式要求设置（需在WPS中查看效果）：
表头：蓝色背景(#4472C4)、白色字体、加粗
数据区域：浅蓝色底纹(#D9E2F3)
汇总行：黄色底纹(#FFF2CC)、加粗
工资列：货币格式 ¥#,##0
题目（2）：公式计算
项目 基础工资 奖金提成 总工资
平均分 7170.00 2940.00 10110.00
工资总和 71700 29400 101100
使用 ROUND(AVERAGE(),2) 计算平均分并保留两位小数
使用 SUM() 计算工资总和
引用示例（G/H/I列）
引用类型 G列（相对引用） H列（绝对引用） I列（混合引用锁定列）
示例 =D2*1.1 =D2*1.1 =$D2*1.1
说明 随单元格变化 固定不变 锁定列，行变化
已设置的数据有效性
验证类型 范围 设置内容
整数 范围 D2:F11 工资 ≥ 0
下拉菜单 C2:C11 销售部、技术部、财务部、人事部
文件名为 员工薪资记录表.xlsx ，可直接下载使用！
已修改的文档
员工薪资记录表.xlsx
下载文件
文档修改记录
清空
员工薪资记录表.xlsx
4/16 20:30
员工绩效评估表.xlsx
4/16 20:22
2025年中国城市经...
4/16 18:58
下载
实验10 条件与公式的编辑.docx
上传文档/图片
输入你的问题或指令...
发送



工号	姓名	年龄	职位	身份证号	手机号	入职日期	销售额(元)
001	张三	25	软件工程	37010119	13812345	2024/01/	85000.00
002	李四	32	数据分析	37010219	13923456	2023/06/	92000.00
003	王五	28	前端开发	37010319	13734567	2024/03/	78000.00
004	赵六	35	软件工程	37010419	13645678	2022/11/	105000.00
005	钱七	24	数据分析	37010520	13556789	2024/07/	65000.00



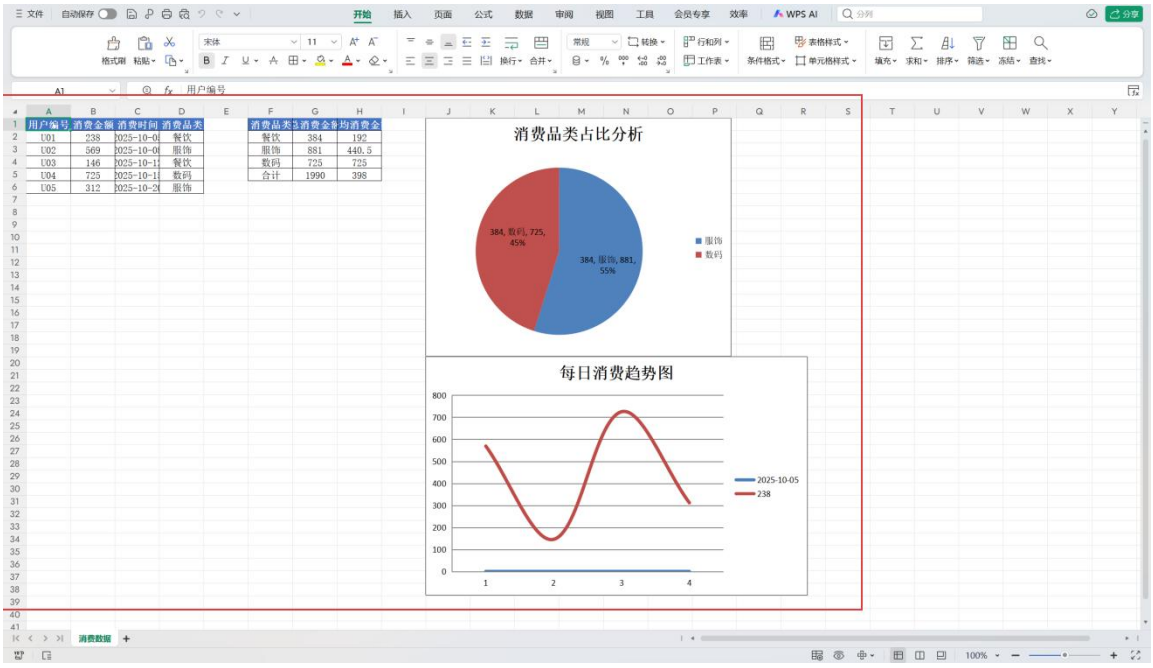


图 12-25 Agent 工具执行后的文档或表格效果示意

系统设置的第一个 Tab 是个人资料。头像上传走 /api/auth/upload_avatar，文件落盘后前端会广播一个 avatar-updated 事件，顶栏的头像组件监听到后立刻刷新，整个过程用户看不到页面跳转，只会看到头像「瞬间换掉」。此外还有昵称修改、邮箱绑定和修改密码，表单在前端就先做一轮校验（比如密码至少 8 位且含字母数字），通过之后再调后端接口，减少无意义的网络请求。

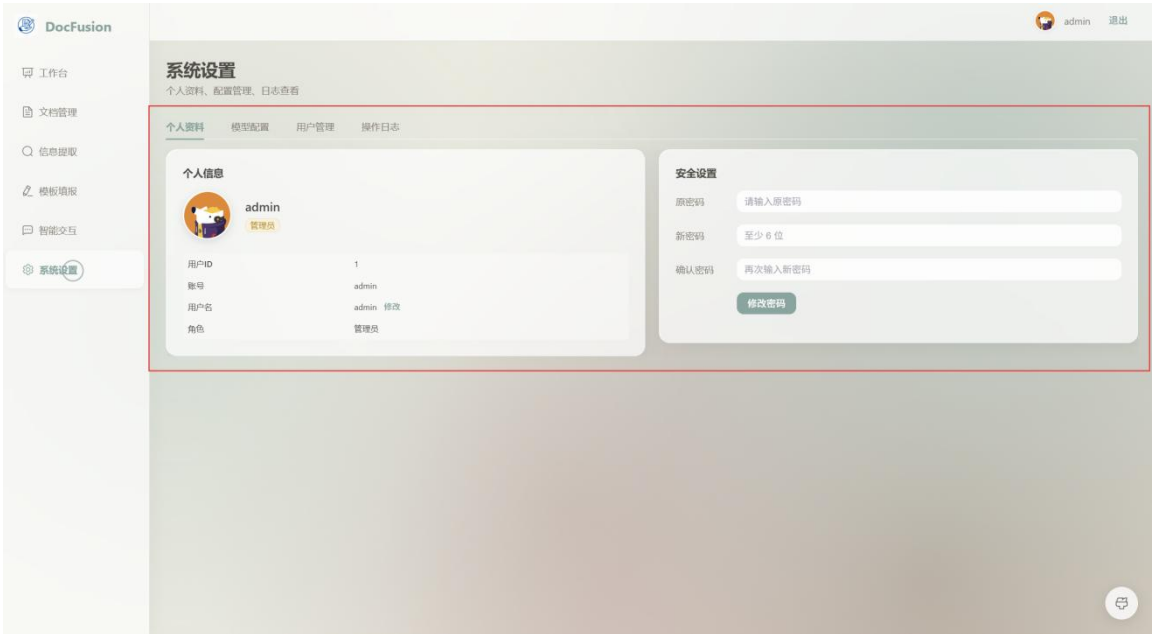


图 12-26 系统设置-个人资料

这是只有管理员才能看到的「模型配置」页，可以说是整个系统的「总开关」。里面有三组设置：1）LLM 配置（Base URL、API Key、Model 名、Format = anthropic / openai）——决定 Agent 和抽取实际调用的是哪个大模型；2）Embedding 配置（模型

名、向量维度）——决定文档切片之后怎么向量化；3）MCP 开关——决定是否把 DocFusion 作为工具集对外通过 JSON-RPC + SSE 暴露。点「保存」后数据写入 system_config 表，并立即热加载到 config.py 的全局变量，不用重启服务就能生效，适合在生产环境临时切模型做 A/B 测试。

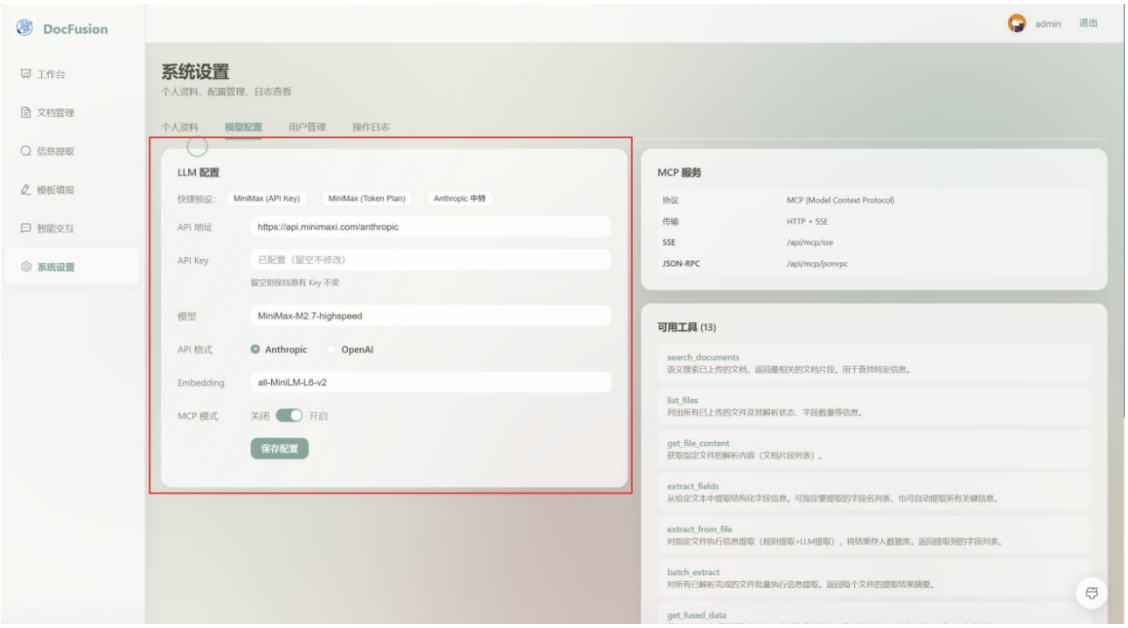


图 12-27 系统设置-模型配置

用户管理对应/api/settings/users，只对管理员开放。列表展示每个用户的账号、角色（admin / user）、创建时间 created_at、最近登录时间 last_login_at——后者能帮管理员识别"僵尸账号"。行级三个操作：「改角色」、「重置密码」（后端生成 12 位随机密码返回给管理员，用户下次登录必须改）、「删除账号」（内置超级管理员受保护不可删）。所有操作同步写 operation_log，带动作者 operator_id 与时间戳，可追溯到具体人和具体时间。

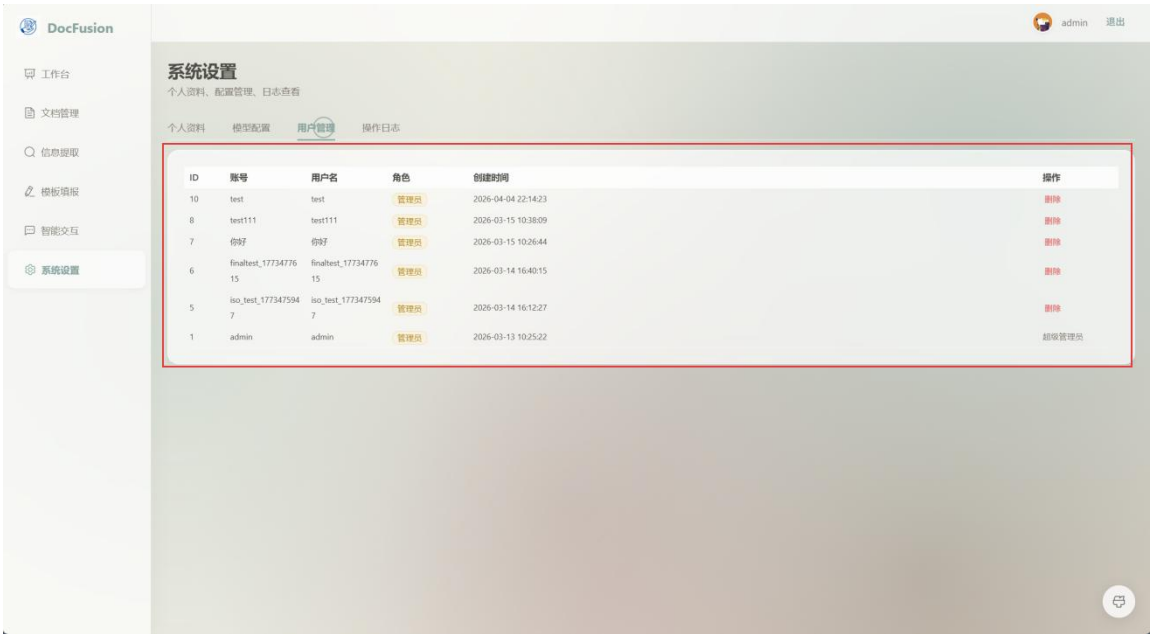


图 12-28 系统设置-用户管理

这是用户管理页下方的「操作日志」面板，对应 /api/settings/logs。每一行都是一条审计记录——记录了「谁、在什么时候、通过什么接口、对谁做了什么」（比如「admin 在 10:23 重置了 user_007 的密码」），日志从 operation_log 表里按 created_at 倒序分页取出。遇到合规审查或者要排查「谁动了这份文档」这类问题时，直接来这个页面就能一步溯源，不用翻后端日志文件。

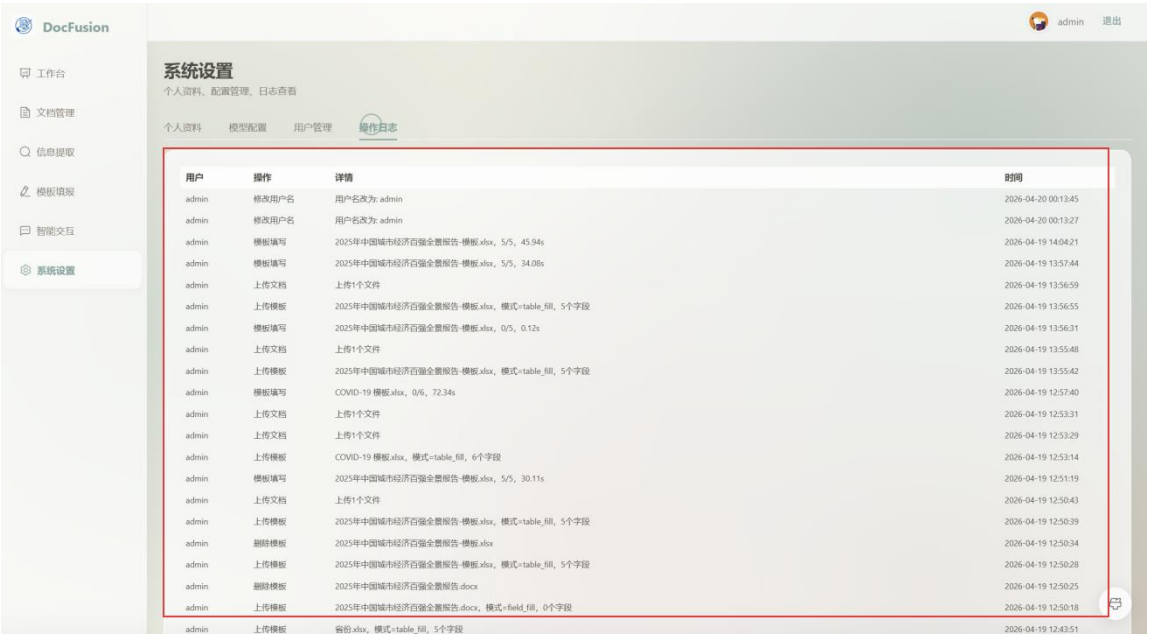


图 12-29 系统设置—用户管理与操作日志

DocFusion 的整体技术栈图：前端（Vue 3 + Element Plus + ECharts + Canvas 2D）、后端（FastAPI + SQLAlchemy + APScheduler）、存储（MySQL + 向量库 + 本地文件盘）、AI 能力（LLM + Embedding + OCR + Agent 工具集）、对外通道（JWT + X-API-Key + MCP JSON-RPC + SSE）和外部生态（n8n / Coze / Dify / Claude Desktop / Cursor）

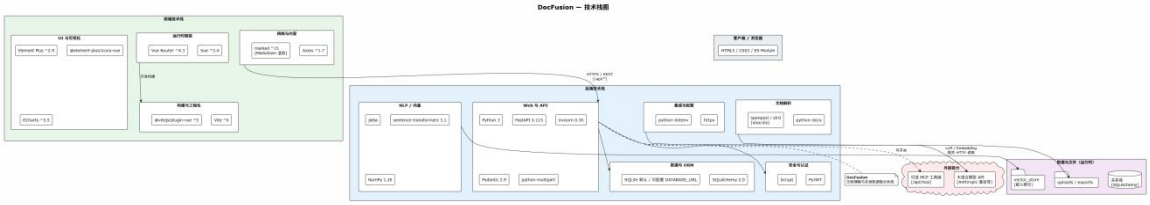


图 12-30 Docfusion 技术栈图

十三、核心代码示例

13.1 后端核心代码

13.1.1 FastAPI 应用装配与路由注册（节选）

python

```
app = FastAPI(title="DocFusion - 文档理解与多源数据融合系统", version="1.0.0",
lifespan=lifespan)
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
app.include_router(auth.router)
app.include_router(documents.router)
app.include_router(extraction.router)
app.include_router(templates.router)
app.include_router(chat.router)
app.include_router(dashboard.router)
app.include_router(settings.router)
app.include_router(mcp.router)
app.include_router(tasks.router)
app.include_router(openapi.router)
@app.get("/api/health")
def health():
    return {"status": "ok", "system": "DocFusion 1.0"}
```

13.1.2 JWT 签发（节选）

python

```
def _token(data: dict) -> str:
    data["exp"] = datetime.utcnow() +
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    return jwt.encode(data, SECRET_KEY, algorithm="HS256")
```

13.2 前端核心代码

13.2.1 路由守卫（节选）

javascript

```
router.beforeEach((to, from, next) => {
  if (to.path !== '/login' && !sessionStorage.getItem('token')) return
next('/login')
  next()
})
```

13.2.2 开发代理（节选）

javascript

```
export default defineConfig({
  server: {
    port: 3000,
    proxy: { '/api': { target: 'http://127.0.0.1:8000', changeOrigin: true } }
  }
})
```

13.3 数据库与连接（节选）

python

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker
from config import DATABASE_URL

engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

13.4 智能对话与 Agent 工具（节选）

13.4.1 Agent 多轮工具调用（节选）

python

```
MAX_TOOL_ROUNDS = 12
for _round in range(MAX_TOOL_ROUNDS):
    resp = await call_llm(messages, system=system, temperature=0.3,
                          max_tokens=3000, tools=AGENT_TOOLS)
    tool_uses = [b for b in resp.content if b.type == 'tool_use']
    if not tool_uses:
        break
    for tu in tool_uses:
        result = execute_tool(tu.name, tu.input, user, db)
        messages.append({'role':'user','content':[{'type':'tool_result',
                         'tool_use_id': tu.id, 'content': result}]})
```

13.4.2 Excel 图表与数据有效性（节选）

python

```
def add_chart_xlsx(file_path, sheet, chart_type, title,
                   data_range, categories_range, position):
```

```
    wb = openpyxl.load_workbook(file_path)
    ws = wb[sheet]
    chart = {'pie': PieChart, 'line': LineChart,
             'bar': BarChart, 'column': BarChart}[chart_type]()
    chart.title = title
    chart.add_data(Reference(ws, range_string=f'{sheet}!{data_range}'),
                   titles_from_data=True)
    chart.set_categories(Reference(ws,
range_string=f'{sheet}!{categories_range}'))
    ws.add_chart(chart, position)
    wb.save(file_path)
```

13.5 开放接口与 MCP（节选）

13.5.1 OpenAPI X-API-Key 鉴权（节选）

python

```
WEBHOOK_KEY = 'docfusion-webhook-2026'
def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key not in (WEBHOOK_KEY, SECRET_KEY):
        raise HTTPException(401, 'invalid api key')
    return True

router = APIRouter(prefix='/api/open',
                   dependencies=[Depends(verify_api_key)])
```

13.5.2 MCP JSON-RPC 派发（节选）

python

```
async def handle_jsonrpc(req: dict, uid: int):
    method = req.get('method')
    if method == 'initialize':
        return {'protocolVersion': '2024-11-05',
                'serverInfo': {'name': 'DocFusion MCP', 'version': '1.0'}}
    if method == 'tools/list':
        return {'tools': MCP_TOOLS}
    if method == 'tools/call':
        name = req['params']['name']
        args = req['params'].get('arguments', {})
        return await dispatch_tool(name, args, uid)
```

13.6 智能填表 smart_fill（节选）

python

```
def fill_template(tpl_path, data_rows, mapping):
    wb = openpyxl.load_workbook(tpl_path)
    ws = wb.active
    header_row = locate_header_row(ws)
    col_map = fuzzy_map_columns(ws[header_row], mapping)
    r = header_row + 1
    for row in data_rows:
        if is_skip_row(row):
            continue
        for src_key, col_idx in col_map.items():
            ws.cell(r, col_idx, clean_value(row.get(src_key)))
        r += 1
    auto_sum_columns(ws, header_row)
    wb.save(out_path)
```

13.7 前端主题与个性化（节选）

13.7.1 CSS 变量主题切换（节选）

javascript

```
function applyTheme(theme) {
  const root = document.documentElement
  root.style.setProperty('--primary-color', theme.primary)
  root.style.setProperty('--bg-gradient-1', theme.bg1)
  root.style.setProperty('--card-blur', theme.blur + 'px')
  theme.palette.forEach((c, i) =>
    root.style.setProperty(`--palette-${i+1}`, c))
  root.style.setProperty('--el-color-primary', theme.primary)
  localStorage.setItem('docfusion_active_theme_id', theme.id)
}
```

13.7.2 自定义鼠标跟随（节选）

javascript

```
window.addEventListener('mousemove', e => {
  cursor.style.transform =
    `translate(${e.clientX}px, ${e.clientY}px)`
})
document.addEventListener('mouseover', e => {
  const hover = e.target.closest('a,button,.el-button,.clickable')
  cursor.classList.toggle('cursor--hover', !!hover)
})
```

十四、项目开发说明

项目名称：DocFusion 文档理解与多源数据融合系统

仓库形态：backend（FastAPI）+ frontend（Vue 3 + Vite）+ scripts（辅助安装）

项目状态：已完成并可运行

版权声明：本项目为原创项目，未曾发表或参加过其他比赛。

本项目是团队基于实际需求独立开发的，所有代码、设计、文档均为原创内容。项目从需求分析、系统设计、编码实现到测试部署，完全由团队成员独立完成，为完全原创，非基于其他项目优化。

十五、创新价值

15.1 模式创新

将“文档理解—结构化抽取—模板成稿—对话核验—运营看板”串联为同一产品闭环，减少多工具拼凑带来的流程断点与数据重复录入。

15.2 功能与体验创新

在同一 Web 端完成管理与业务操作；通过可配置大模型与可选 MCP 扩展，在保持界面简洁的同时保留生态对接空间。

15.3 技术应用创新

后端以 FastAPI 轻量组合 SQLAlchemy、JWT 与异步能力，前端以 Vue 3 + Vite 提升交付效率；文档侧可结合 LibreOffice 提升格式兼容性。

对话即操作：以 Anthropic 风格的多轮 tool_use 协议驱动 12 项 Agent 工具，让用户用自然语言完成 Word 排版、Excel 制图与数据有效性、模板自动填写等过去依赖宏 / 脚本的工作。

开放生态双通道：同时支持 OpenAPI（X-API-Key，面向 n8n / Coze 等工作流）与 MCP（JSON-RPC + SSE，面向 Claude Desktop / Cursor 等智能体客户端），覆盖**「低代码」**和**「AI 客户端」**两种集成范式。

前端工程化体验：CSS 变量 + localStorage 实现轻量级主题引擎，无需重启或重新构建即可换肤；Canvas 粒子背景与玻璃拟态，提升非功能性体验。

十六、商业与社会价值

16.1 商业价值

降本增效：显著减少人工摘录、排版与复核时间，缩短交付周期。

可复制交付：同一套系统可面向不同处室或子公司做配置化推广。

增值空间：可按行业包（模板库、抽取规则库、指标口径库）与运维服务形成可持续收入。

16.2 社会价值

提升公共与企事业单位信息处理透明度与可追溯性（在完善审计后更易落地）。

减少重复劳动，改善一线材料岗工作体验。

为数字化转型提供可落地的文档智能抓手。

十七、总结与后续优化改进

17.1 项目总结

DocFusion 面向文档理解与多源融合场景，采用前后端分离与可配置智能能力，已形成从上传到成稿与对话的完整链路。系统结构清晰，便于分期建设与对接。

17.2 后续优化改进

17.2.1 功能扩展

权限细化到资源级与操作级；组织架构与部门数据隔离。

电子签章、版式固定 PDF、水印与脱敏规则。

批量任务队列与邮件/企业微信通知。

17.2.2 技术升级

数据库由 SQLite 平滑迁移至 PostgreSQL / MySQL；对象存储对接 MinIO / 云厂商。

引入统一网关、限流熔断与全链路追踪。

向量库与检索策略工程化（分片、增量更新、评估集）。

17.2.3 业务拓展

行业模板包与指标口径资产库。

与 OA、档案、知识库平台的标准对接包。

私有化交付与分级运维服务。

十八、术语与名词定义

为便于阅读与对照源码，对本方案中频繁出现的专有术语统一定义如下：

表 18-1 术语与名词定义

术语	释义（结合本项目）
DocFusion	本系统名称，含义为「文档（Doc）+ 多源融合（Fusion）」，定位为面向公文/统计/合同等场景的文档智能处理与对话操作平台。
LLM	Large Language Model，大语言模型。本系统通过 llm_service 适配兼容 Anthropic / OpenAI 双格式的推理服务（Base URL + API Key 可在系统设置热切换）。
RAG	Retrieval-Augmented Generation，检索增强生成。本系统的智能对话与轻量问答均基于 vector_store 召回 + LLM 生成。
MCP	Model Context Protocol，由 Anthropic 推出的智能体外置工具协议。本系统在 /api/mcp/* 实现了 JSON-RPC + SSE 双通道（protocolVersion 2024-11-05）。
OpenAPI（本系统）	本系统提供的对外 HTTP 开放接口集合，统一前缀 /api/open，使用请求头 X-API-Key 鉴权；面向 n8n / Coze / Dify 等低代码工作流平台。
JWT	JSON Web Token，本系统采用 HS256 签发，默认 1440 分钟有效；前端存于 sessionStorage 并由 axios 拦截器自动注入。
Agent / Tool Use	Anthropic 风格的多轮工具调用范式。本系统 chat.py 中以 MAX_TOOL_ROUNDS=12 驱动 doc_editor.AGENT_TOOLS 内 12 项工具完成「搜索→抽取→编辑→排版→制图→校验」的端到端操作。
Embedding / 向量检索	本系统优先使用 sentence-transformers 嵌入 + 余弦相似度（NumPy 矩阵乘）做语义检索；模型不可用时降级为 jieba 分词 + 词重叠启发式打分。

模板填表	本系统支持两种模板模式——field_fill（字段名+值竖向）与 table_fill（表头横向、数据下行），由 template_filler.identify_fields 自动识别。
多源融合（fusion）	对同一字段从多份文档抽取出的多候选结果，按字段别名归一 + 来源加权 + 多值投票得到最终值与置信度，输出至 /api/extraction/fused。
DISPIMG	WPS 表格的「在单元格内嵌图片」自定义函数 =_xlfn.DISPIMG，通过 cellimages.xml 维护映射；本系统 image_filler 支持其与通用浮动图片两种写入方式。
玻璃拟态 / Glassmorphism	前端的全局视觉风格，通过多径向渐变背景 + .glass-card 毛玻璃 + hover 光带动画实现；与 Element Plus 组件样式无缝衔接。
jieba	中文分词库，本系统在抽取规则、向量检索降级、轻量摘要等多个环节使用。
LibreOffice / soffice	本系统 legacy_doc 调用 soffice headless 将旧版 .doc(OLE) 转为 .docx；支持 vendor 内置版本与 LIBREOFFICE_PROGRAM 环境变量。